ĐẠI HỌC BÁCH KHOA HÀ NỘI

TRƯỜNG ĐIỆN - ĐIỆN TỬ



BÁO CÁO DỰ ÁN

THIẾT KẾ VÀ XÂY DỰNG THIẾT BỊ LOGIC ANALYZER

Nhóm sinh viên thực hiện: **Vũ Hoàng Hiệp**

Vũ Đức Thuận

Dương Ngọc Hoài Anh

Lê Khả Đức Linh

Người hướng dẫn: **TS. Đào Việt Hùng**

Hà Nội, -2026

MỤC LỤC

DANH MỤC HÌNH VẼ 3

DANH MỤC BẢNG BIỂU 3

1. TỔNG QUAN VÀ MỤC TIÊU DỰ ÁN 4

1.1 Đặt vấn đề: Bối cảnh và thách thức 4

1.2 Mục tiêu dự án 5

1.3 Phạm vi và Ràng buộc cốt lõi 7

1.4 Các ràng buộc kỹ thuật và bảo mật phụ 8

2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG 8

2.1 Vi điều khiển RP2354B (ARM Cortex-M33) 8

2.2 Ngoại vi lập trình được PIO (Programmable I/O) và bộ điều khiển DMA 12

2.3 Mạch dịch mức logic (Level Shifter) 17

2.4 Giao tiếp USB CDC và giao thức Serial 18

2.5 Giao tiếp SPI và hiển thị LCD ST7789 20

3. THIẾT KẾ VÀ TRIỂN KHAI PHẦN CỨNG 24

3.1 Sơ đồ khối phần cứng 24

3.2 Thiết kế mạch nguyên lý (Schematic) 25

3.3 Thiết kế mạch in (PCB Layout) 29

3.4 Lựa chọn linh kiện và chế tạo 31

4. THIẾT KẾ FIRMWARE VÀ ỨNG DỤNG 38

4.1 Kiến trúc firmware 38

4.2 Triển khai lấy mẫu 43

4.3 Tối ưu hóa cấu hình ưu tiên bus (Bus Priority) 48

4.4 Thiết kế giao thức truyền tin Serial 53

4.5 Ứng dụng trên máy tính (PC Client) 55

5. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 60

5.1 Thiết lập môi trường thử nghiệm 60

5.2 Đánh giá chế độ Standalone (LCD)..... 61
5.3 Đánh giá sai số lấy mẫu..... 64
5.4 Đánh giá độ chính xác..... 68
5.5 Đánh giá tốc độ truyền dữ liệu..... 71
5.6 Phân tích giới hạn hệ thống & các trade-off..... 73
6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 76
6.1 Kết quả đạt được..... 76
6.2 Hướng phát triển..... 77
TÀI LIỆU THAM KHẢO 77

DANH MỤC HÌNH VẼ

Hình 3.1 — Sơ đồ khối phần cứng của thiết bị Logic Analyzer 24 kênh.....25
Hình 3.2 — Schematic tổng thể của board Logic Analyzer.....26
Hình 3.3 — Bố trí linh kiện trên PCB (dựng lại từ Gerber xuất ra từ Altium).....30
Hình 3.4 — Mặt trước board mạch thực tế sau khi hàn linh kiện.....37
Hình 3.5 — Mặt sau board mạch thực tế.....38

DANH MỤC BẢNG BIỂU

Bảng 3.1 — Giới hạn điện của IC dịch mức SN74LVC8T245.....27
Bảng 3.2 — Phân tích hệ quả các kiểu hỏng của tầng đệm dịch mức.....28
Bảng 3.3 — Danh mục linh kiện chính kèm thông số kỹ thuật.....34
Bảng 3.4 — Danh mục linh kiện thụ động theo nhóm chức năng.....35
Bảng 3.5 — Ngân sách dòng điện của hệ thống.....36

1. TỔNG QUAN VÀ MỤC TIÊU DỰ ÁN

1.1 Đặt vấn đề: Bối cảnh và thách thức

Trong kỷ nguyên phát triển mạnh mẽ của hệ thống nhúng và IoT, các giao thức truyền thông số như UART, I2C, hay SPI đã trở thành nền tảng kết nối không thể thiếu. Tuy nhiên, việc thiết kế và kiểm thử các hệ thống này ngày càng phức tạp do tần số hoạt động cao và mật độ tín hiệu dày đặc. Các thiết bị đo lường truyền thống như máy hiện sóng (oscilloscope) thường bị giới hạn về số lượng kênh hiển thị đồng thời và có giá thành rất cao, trong khi đồng hồ vạn năng hoàn toàn bất lực trước các xung tín hiệu thay đổi nhanh theo mili giây.

Khi xảy ra các sự cố về định thời (timing glitch), xung nhiễu hoặc sai lệch dữ liệu truyền nhận trên nhiều đường truyền cùng lúc, người phát triển rất khó định vị chính xác nguồn lỗi nếu thiếu một công cụ chuyên dụng. Vì vậy, nhóm chúng em quyết định thực hiện dự án "Thiết kế và chế tạo Logic Analyzer (Bộ phân tích logic)". Dự án tập trung xây dựng một thiết bị nhỏ gọn, hỗ trợ giám sát đa kênh và giải mã giao thức trực quan với chi phí tối ưu. Đây không chỉ là một bài toán kỹ thuật thực tế mà còn tạo ra một công cụ hỗ trợ đắc lực cho quá trình debug phần cứng và nghiên cứu mạch số.

1.2 Mục tiêu dự án:

MỤC TIÊU VÀ KIẾN TRÚC TỔNG QUAN

Dự án xây dựng một thiết bị phân tích logic (Logic Analyzer) 24 kênh song song, tốc độ lấy mẫu cao lên đến 150 MSa/s, hỗ trợ cơ chế hiển thị và vận hành kép (Dual-display & Standalone mode): tiếp nhận đồng thời các dòng tín hiệu số vật lý có tần số cao, trực quan hóa trạng thái mạch số trên màn hình LCD ST7789 tích hợp thông qua hệ thống menu điều hướng độc lập, và biểu diễn chi tiết dạng sóng trên máy tính qua giao tiếp USB CDC sử dụng giao thức text v1.2, nhằm cung cấp thông tin phân tích định thời khách quan cho các kỹ sư hệ thống nhúng.

Khái niệm về Logic Analyzer: là một thiết bị đo lường điện tử chuyên dụng được sử dụng để thu thập, hiển thị và phân tích các tín hiệu số (digital signals) từ nhiều kênh cùng một lúc. Khác với dao động ký (oscilloscope) vốn tập trung vào đo điện áp liên tục theo thời gian thực trên một vài kênh, Logic Analyzer tập trung xử lý trạng thái logic (0 và 1) trên nhiều tuyến tín hiệu đồng thời để kiểm tra sự đồng bộ và giải mã dữ liệu truyền tải.

Hệ thống ứng dụng kiến trúc phân tầng xử lý và bất đối xứng luồng dữ liệu, tách bạch nghiêm ngặt giữa nhiệm vụ thu thập phần cứng thời gian thực và các tác vụ hiển thị, truyền thông:

- Nhánh phần cứng và thu thập dữ liệu thô (Hardware/Firmware Layer): Sử dụng vi điều khiển kiến trúc mới RP2354B (dòng RP2350, vỏ QFN-80). Nhánh này trực tiếp tiếp nhận tín hiệu điện áp từ 24 kênh độc lập (GPIO4–27) thông qua hàng rào bảo vệ và dịch mức IC SN74LVC8T245. Luồng bắt dữ liệu tốc độ cao (lên đến 150 MSa/s) được thực thi hoàn toàn bằng phần cứng thông qua sự phối hợp giữa khối PIO0 và bộ điều khiển DMA ghi thẳng vào SRAM dưới dạng Burst (24 bit tương đương 1 mẫu/chu kỳ) với CPU overhead bằng 0%. Lõi Core 0 chịu trách nhiệm quản lý State Machine, thuật toán kích hoạt (Trigger Engine), xử lý nút bấm và đóng gói dữ liệu truyền thông Serial.
- Nhánh hiển thị và diễn giải (Display & PC Interface Layer): Để triệt tiêu hoàn toàn hiện tượng nghẽn cổ chai hệ thống, đồ án cô lập hoàn toàn tác vụ hiển thị sang lõi Core 1, chuyên dụng cho việc vẽ đồ họa lên màn hình LCD ST7789 với tần số quét polling 100 Hz (quản lý giao diện gồm 12 trang chức năng và 1 trang tổng quan 24 kênh), đảm bảo màn hình luôn mượt mà kể cả khi hệ thống đang truyền dữ liệu lớn

về PC. Ở tầng máy tính, ứng dụng PC tiếp nhận chuỗi text protocol v1.2 qua USB CDC để tái tạo dạng sóng độ phân giải cao và tính toán tần số.

Cách tiếp cận này giúp neo giữ độ chính xác của phép đo vào xung clock phần cứng đã qua hiệu chuẩn, loại bỏ hoàn toàn nguy cơ Jitter do xung đột tài nguyên giữa tác vụ hiển thị LCD và tác vụ lấy mẫu.

MỤC TIÊU CỤ THỂ CỦA DỰ ÁN

Từ mục tiêu tổng quát, dự án đặt ra năm mục tiêu cụ thể:

1. Thiết kế khối phần cứng PCB và tối ưu mã nguồn điều khiển PIO/DMA trên chip RP2354B: Làm chủ cấu trúc thiết kế mạch với chip xử lý trung tâm RP2354B; cấu hình mạch bảo vệ quá áp và tương thích dải điện áp rộng (1.65 V-5.5 V) sử dụng 3 IC dịch mức SN74LVC8T245 kết hợp chân FT (5V). Lập trình khối PIO0 và cấu hình luồng dữ liệu Burst DMA để đạt tốc độ lấy mẫu thời gian thực lên đến 150MSa/s trên cả 24 kênh song song, cho phép điều chỉnh linh hoạt runtime dải tần số lấy mẫu rộng từ 2.3kSa/s đến 150MSa/s.
2. Xây dựng cơ chế Trigger phần cứng và công thức tính toán tần số độ chính xác cao: Hiện thực hóa bộ engine kích hoạt đa năng ngay tại tầng mã nguồn (Core 0), hỗ trợ Trigger theo cạnh (cạnh lên/cạnh xuống) trên một kênh bất kỳ được chỉ định, tích hợp cơ chế lưu dữ liệu hậu kích hoạt (post-trigger). Đồng thời xây dựng thuật toán đo đạc định thời để trích xuất tần số tín hiệu đầu vào với độ chính xác cao từ 4 đến 5 chữ số có nghĩa (ví dụ hiển thị chính xác tới mức 15.6255 MHz).
3. Cưỡng chế ràng buộc an toàn tài nguyên bộ đệm và phân rã kiến trúc Đa nhân (Dual-core): Quản lý nghiêm ngặt phân bổ vùng nhớ bộ đệm SRAM vật lý, hỗ trợ cấu hình động kích thước bộ đệm linh hoạt từ 16 mẫu đến 65536 mẫu (mặc định 8192 mẫu) mà không làm tràn bộ nhớ. Thiết lập tường ngăn tài nguyên mã nguồn: Cưỡng chế Core 0 chạy độc lập các tác vụ sống (State machine, USB CDC text v1.2, nhận lệnh từ PC) và cô lập hoàn toàn tác vụ quét hiển thị đồ họa LCD ST7789 sang Core 1 nhằm triệt tiêu hiện tượng trễ định thời (Jitter) trên luồng thu thập.
4. Đánh giá mô hình đo lường nghiêm túc trên bốn phương diện hiệu năng hệ thống: Tiến hành đo đạc thực nghiệm định lượng để chứng minh:

- *Hiệu năng trong miền:* Đọc sạch hoàn toàn, không mất mát tín hiệu tại tần số đầu vào cao nhất 15 MHz nhờ đáp ứng tỷ lệ oversample 10×.
- *Hiệu năng dưới sự dịch chuyển miền/giới hạn:* Khả năng bắt tín hiệu cận giới hạn lý thuyết định lý Nyquist tại dải tần khoảng ~75 MHz.
- *Mức độ hiệu chuẩn:* Sai số định thời của bộ đệm và độ lệch pha giữa 24 chân GPIO4-27.

- *Tính công bằng ổn định:* Khả năng bắt xung và giữ độ chính xác 4-5 chữ số của phép đo tần số trên toàn bộ dải điện áp logic từ 1.65 V đến 5.5V dưới các cấu hình kích thước bộ đệm thay đổi.

Hiện thực hóa thiết bị đóng gói, tối ưu giao diện độc lập (Standalone) và bộ kịch bản kiểm thử: Hoàn thiện sản phẩm dưới dạng thiết bị đo cầm tay hoàn chỉnh có vỏ hộp bảo vệ; tổ chức cấu trúc giao diện LCD ST7789 khoa học (gồm 12 trang tính năng chuyên sâu và 1 trang dashboard tổng quan trạng thái logic của 24 kênh cùng hệ thống menu điều hướng vật lý mượt mà). Đồng thời, hoàn thiện bộ kịch bản giao tiếp text protocol v1.2 qua USB để kết nối đồng bộ dữ liệu với PC, sẵn sàng thực hiện các bài đo đối chứng tự động với các dòng thiết bị phân tích logic chuyên dụng trên thị trường.

1.3 Phạm vi và Ràng buộc cốt lõi

Vì hoạt động trong lĩnh vực đo lường phần cứng và kiểm thử mạch số tốc độ cao, hệ thống mang một tập ràng buộc kỹ thuật và kiểm soát an toàn nghiêm ngặt, xem đây là yêu cầu phi chức năng quan trọng nhất của đồ án. Nguyên tắc xuyên suốt: "Độ toàn vẹn dữ liệu thời gian thực và an toàn thiết bị là ưu tiên tối thượng".

Bốn ràng buộc an toàn kỹ thuật cốt lõi

- Không tự ý nội suy dạng sóng (No Waveform Hallucination): Hệ thống tuyệt đối không tự động làm mượt hoặc suy diễn trạng thái logic khi xảy ra hiện tượng mất mẫu (drop frame) hoặc tràn bộ đệm. Mọi hành vi mất dữ liệu phải được báo cáo tường minh trên giao diện bằng cờ báo lỗi thay vì hiển thị dữ liệu giả lập để đảm bảo tính khách quan của phép đo.
- Ràng buộc điện áp đầu vào nghiêm ngặt: Thiết bị chỉ hoạt động trong dải điện áp an toàn được cấu hình từ 1.65 V đến 5.5 V. Mọi xung điện áp vượt ngưỡng hoặc hiện tượng đảo cực phải bị triệt tiêu/cách ly ở tầng linh kiện bảo vệ (chân FT và 3 IC dịch mức SN74LVC8T245) để ngăn ngừa nguy cơ phá hủy vật lý chip xử lý trung tâm RP2354B.
- Cảnh báo giới hạn băng thông theo định lý Nyquist: Khi tần số tín hiệu đầu vào vượt quá dải "đọc sạch" 15 MHz (tiến dần về giới hạn lý thuyết ~75 MHz), hệ thống phải tự động đưa ra khuyến cáo về hiện tượng răng cưa (aliasing) và sụt giảm tỷ lệ oversample trên màn hình, không khẳng định độ chính xác tuyệt đối của biên dạng xung đồ họa.
- Cô lập luồng cấu hình Runtime: Việc thay đổi các thông số cấu hình ngay khi đang chạy (Runtime điều chỉnh từ 2.3kSa/s đến 150MSa/s, bộ đệm từ 16 đến 65536 mẫu) tuyệt đối không được gây ảnh hưởng hoặc tạo độ trễ (Jitter) lên chu kỳ lấy mẫu của khối PIO0 vật lý.

Cơ chế phòng thủ nhiều lớp tầng mã nguồn (Defense-in-depth)

Các ràng buộc kỹ thuật trên không giao cho ứng dụng PC "tự tuân thủ" bằng phần mềm mà được cưỡng chế trực tiếp ở tầng phần cứng và mã nguồn firmware theo nguyên tắc phòng thủ nhiều lớp (defense-in-depth).

Core 0 đóng vai trò như một cổng an toàn (safety gate) bằng mã nguồn, liên tục giám sát trạng thái ghi Burst của DMA vào SRAM. Khi phát hiện các sự kiện bất thường (như xung đột lệnh text v1.2 từ PC, cấu hình bộ đệm vượt quá dung lượng khả dụng, hoặc mất kết nối USB CDC), Core 0 lập tức ngắt luồng xử lý lỗi, đưa hệ thống về trạng thái an toàn (Safe State), đồng thời phát tín hiệu liên nhân (Inter-core) cưỡng chế **Core 1** dừng việc quét menu 12 trang thông thường để khóa màn hình LCD ST7789 vào trang tổng quan cảnh báo hệ thống.

Các ràng buộc kỹ thuật và bảo mật phụ

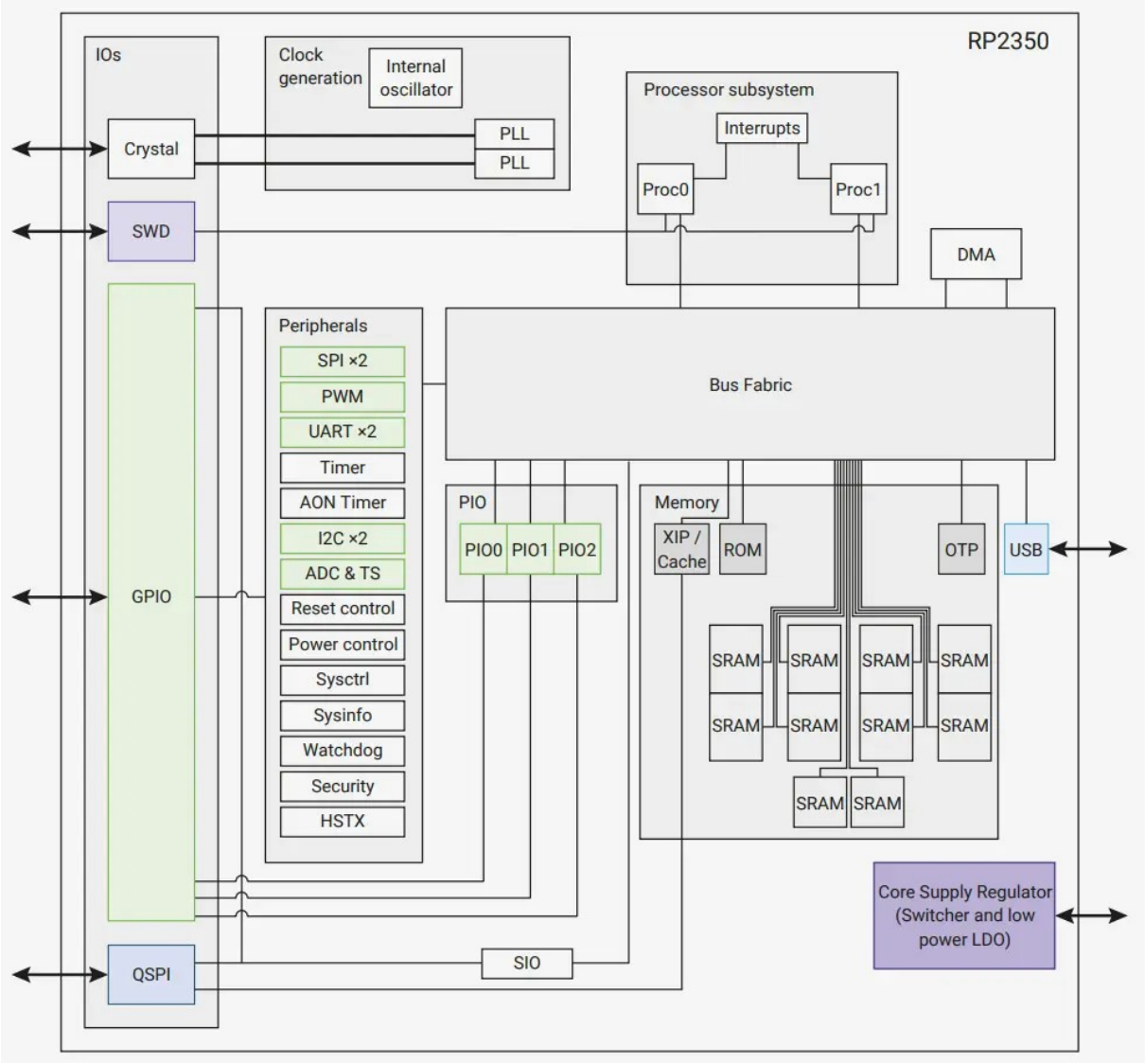
1. Quản lý dòng định mức USB: Giới hạn dòng tiêu thụ của toàn mạch thông qua cổng USB PC, tự động ngắt kết nối nếu mạch mục tiêu (target board) có hiện tượng ngắn mạch ngược dòng về thiết bị đo.
2. Chuẩn hóa chuỗi Text Protocol v1.2: Lọc bỏ và từ chối các ký tự lạ hoặc chuỗi lệnh không đúng định dạng cấu trúc mã nguồn trước khi nạp vào thanh ghi cấu hình của bộ tạo tần số (Baudrate Generator).
3. Cô lập mã nguồn cấu hình: Không lưu trực tiếp các tham số hiệu chuẩn clock thạch anh (hardcode) trong mã nguồn PIO, mọi thông số được quản lý thông qua các biến môi trường vật lý được nạp vào phân vùng lưu trữ khi khởi động hệ thống.

2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN TẢNG

2.1 Vi điều khiển RP2354B (ARM Cortex-M33)

2.1.1 Kiến trúc phần cứng thế hệ RP2350 (ARM Cortex-M33)

Thế hệ RP2350 mang lại một bước nhảy vọt về hiệu năng tính toán và tính năng bảo mật nhờ việc chuyển dịch sang kiến trúc nền tảng Armv8-M.



Kiến trúc tổng thể của hệ thống RP2350. Nguồn: DeepBlueMbedded

Cấu trúc Lõi kép đối xứng (Symmetric Dual-Core): Hệ thống sở hữu cặp lõi ARM Cortex-M33 hoạt động độc lập ở xung nhịp tối đa 150 MHz.
Góc nhìn chuyên sâu: Trên thực tế cấu trúc die silicon của hệ vi điều khiển này chứa đồng thời cả 2 lõi Cortex-M33 và 2 lõi Hazard3 RISC-V mã nguồn mở. Tại một thời điểm bạn có thể cấu hình chạy song song 2 lõi M33, hoặc 2 lõi RISC-V, thậm chí là lai (1 lõi M33 + 1 lõi RISC-V) bằng cách thiết lập thanh ghi ARCHSEL và kích hoạt khởi động lại lõi.

Khối tính toán nâng cao: Mỗi lõi Cortex-M33 được tích hợp bộ tính toán số thực FPU chuyên dụng (hỗ trợ Single-precision hoàn chỉnh và tăng tốc các phép tính Double-precision thông qua bộ đồng xử lý đồng bộ) kèm theo tập lệnh mở rộng xử lý tín hiệu số

DSP. Điều này giải quyết triệt để bài toán tính toán các thuật toán điều khiển PID, lọc Kalman hay ma trận chuyển đổi vốn là điểm yếu của dòng Cortex-M0+ cũ.

Hệ thống bảo mật phần cứng: Khối lõi M33 tích hợp sâu công nghệ Arm TrustZone, cho phép cô lập phần mềm thành hai vùng kiểm soát độc lập: Vùng bảo mật (Secure) và Vùng thông thường (Non-secure). Đi kèm là bộ tăng tốc mã hóa phần cứng SHA-256, bộ sinh số ngẫu nhiên thực TRNG và khối bộ nhớ lập trình một lần OTP 8 KB giúp triển khai Secure Boot mã hóa chữ ký số một cách an toàn.

2.1.2 Hệ thống chân GPIO của RP2354B

Nhờ đóng gói dạng QFN-80, phiên bản chữ "B" này giải phóng rất nhiều chân chức năng bị giới hạn ở các phiên bản nhỏ hơn

Số lượng chân: Cung cấp đến 48 chân GPIO đa năng (từ GPIO0 đến GPIO47), đáp ứng thoải mái cho các mạch ứng dụng cần giao tiếp bus dữ liệu song song hoặc nhiều thiết bị ngoại vi cùng lúc.

Kênh Analog mở rộng: Tích hợp 8 kênh đầu vào ADC 12-bit tốc độ 500 kS/s. Việc nâng từ 4 kênh (ở bản QFN-60) lên 8 kênh giúp bạn có thể đọc đồng thời nhiều cảm biến analog mà không phải thiết kế thêm IC chuyển mạch ngoài.

Khối chức năng ngoại vi ghép kênh (Muxing):

- 3 khối PIO (Programmable I/O) - Tổng cộng 12 State Machines: Đây là cấu trúc đặc sản của Raspberry Pi. Bạn có thể tự viết mã cho các máy trạng thái này để biến GPIO thành bất kỳ giao thức phần cứng nào mong muốn (như bus truyền dữ liệu màn hình, giả lập các chuẩn giao tiếp tùy biến) với độ chính xác theo từng chu kỳ xung nhịp mà không làm tiêu tốn tài nguyên xử lý của CPU.
- Giao tiếp HSTX (High-Speed Transmit): Khối truyền tải dữ liệu song song siêu tốc chuyên dụng để bit-bang xuất tín hiệu hình ảnh trực tiếp ra cổng DVI/DSI.
- Ngoại vi cố định: Gồm 2x UART, 2x SPI, 2x I2C điều khiển qua DMA, cùng 24 kênh PWM độc lập (tối ưu mạnh cho điều khiển động cơ đa trục hoặc chèn dead-time).

2.1.3 Đặc tính chịu áp 5V (5V-Tolerant)

Một trong những cải tiến đáng giá nhất cho các kỹ sư phần cứng khi thiết kế mạch với dòng RP2350 là các chân IO có khả năng chịu áp 5V, khắc phục hoàn toàn nhược điểm nhạy cảm điện áp của dòng RP2040 cũ.

- Cơ chế chịu áp: Các chân GPIO kỹ thuật số (Digital-only) được thiết kế theo cấu trúc FT (Five-Volt Tolerant). Khi đường nguồn ngoại vi cấp cho các khối IO là IOVDD đang hoạt động ở mức 3.3V, các chân FT này có thể nhận trực tiếp mức logic 5 V (tối đa lên đến 5.5V) từ các IC bên ngoài xông vào mà không bị hư hại.
- Mức điện áp đầu ra (Output): Khi chân cấu hình làm Output, nó vẫn chỉ xuất ra điện áp tối đa là 3.3 V. Tuy nhiên, mức áp 3.3V này hoàn toàn đạt ngưỡng nhận diện mức logic HIGH (V_IH) của phần lớn các chip logic text TTL thông dụng.

2.1.4 Cảnh báo phần cứng quan trọng (Hardware Footguns)

1. Các chân ADC KHÔNG chịu được 5V: 8 chân GPIO có chức năng chia sẻ với đầu vào Analog (ADC) tuyệt đối không có tính năng FT. Nếu bạn đưa điện áp vượt quá mức IOVDD (3.3 V) vào các chân này, khối silicon ADC bên trong sẽ bị phá hủy ngay lập tức.
2. Trạng thái Mất nguồn (Unpowered): Tính năng chịu áp 5 V chỉ khả dụng khi chip đã được cấp nguồn ổn định. Khi hệ thống tắt điện hoặc IOVDD rơi về mức 0V, các chân này chuyển sang trạng thái 3.3V. Nếu lúc này có một mạch ngoài cấp tín hiệu 5V vào, chip sẽ bị hỏng do hiện tượng rò dòng ngược qua các cấu trúc bảo vệ tĩnh điện.
3. Cấu hình Open-Drain: Nếu bạn sử dụng chân cấu hình Open-Drain để kéo điện trở treo (Pull-up) lên đường nguồn 5V ngoài (ví dụ trong mạng bus I2C mức 5V, hãy đảm bảo nguồn IOVDD luôn xuất hiện trước hoặc đồng thời với nguồn 5 Vđó để tránh rủi ro hỏng hóc.

2.1.5 Kiến trúc Bus dữ liệu (Bus Fabric)

Để đảm bảo hai lõi CPU công suất cao cùng bộ ngoại vi hoạt động mượt mà không bị nghẽn cổ chai dữ liệu, RP2350 triển khai một cấu trúc ma trận bus có tính định thời cực cao (Deterministic Bus Fabric).

Bộ chuyển mạch ma trận AHB Crossbar
Hệ thống sử dụng bộ chuyển mạch ma trận Fully-connected AHB Crossbar Switch kết nối toàn phần giữa các thiết bị Master và Slave.

- Các Master trên mạng Bus: Gồm CPU 0, CPU 1, và bộ điều khiển DMA 16 kênh độc lập (có các master đọc và viết riêng biệt).
- Cơ chế hoạt động: Bộ chuyển mạch AHB Crossbar cho phép nhiều Master có thể truy cập vào các Slave khác nhau trong hệ thống (ví dụ: các vùng RAM khác nhau, các thanh ghi ngoại vi APB, hoặc bộ nhớ Flash XIP) trong cùng một chu kỳ xung

nhịp một cách hoàn toàn song song mà không cần phải xếp hàng chờ đợi giải chấp bus.

Kiến trúc phân tách SRAM (RAM Striping / Banking)
Tổng dung lượng 520 KB SRAM nội của chip được phân rã thành 10 bank nhớ độc lập độc quyền thay vì là một khối dung lượng duy nhất.

- Tối ưu luồng dữ liệu: Nhờ việc tách rời thành 10 bank, bạn có thể thiết kế chương trình tối ưu hóa hiệu năng cực cao. Ví dụ: Lõi CPU 0 đang bận thực thi và xử lý dữ liệu phức tạp trên vùng nhớ của Bank 0, bộ định tuyến DMA có thể chạy độc lập hoàn toàn để hút dữ liệu từ ngoại vi UART đổ thẳng vào vùng nhớ Bank 1, trong khi lõi CPU 1 đang đọc các bảng tham chiếu toán học lưu ở Bank 2.
- Tính định thời (Deterministic): Toàn bộ quá trình truy cập song song vào các bank nhớ khác nhau này không gây ra bất kỳ sự xung đột bus (Bus Contention) nào, giúp mã nguồn của bạn chạy với thời gian phản hồi chính xác tuyệt đối, không bị trễ ngẫu nhiên.

Cơ chế XIP (Execute-in-Place) nội vi
Đối với mã hiệu RP2354B, bộ nhớ Flash 2 MB được tích hợp thẳng bên trong vỏ chip thông qua kết nối QSPI chuyên dụng. Bộ điều khiển XIP của mạng Bus Fabric tích hợp một khối bộ nhớ đệm Cache tốc độ cao. Khi CPU thực thi mã lệnh từ Flash, bộ cơ chế XIP sẽ tự động ánh xạ vùng nhớ Flash này vào không gian địa chỉ tuyến tính của hệ thống. CPU đọc mã lệnh thông qua Cache giống hệt như đang truy xuất trực tiếp từ RAM nội, mang lại tốc độ thực thi mượt mà và tối ưu hóa điện năng tiêu thụ.

2.2 Ngoại vi lập trình được PIO (Programmable I/O) và bộ điều khiển DMA

NGUYÊN LÝ THU THẬP DỮ LIỆU SONG SONG TỐC ĐỘ CAO BẰNG KHỐI NGOẠI VI PIO

Hệ thống thu thập dữ liệu (Capture System) tận dụng sức mạnh của khối ngoại vi lập trình được PIO (Programmable I/O) tích hợp trên chip. Giải pháp này cho phép hiện thực hóa việc đọc trạng thái của nhiều chân GPIO cùng một lúc với độ chính xác định thời tuyệt đối ở mức phần cứng, giải phóng hoàn toàn CPU khỏi tác vụ lấy mẫu.

Sự phối hợp trong kiến trúc phần mềm điều khiển được chia làm hai phần:

- Nhân xử lý phần cứng (Phần cứng PIO): Hiện thực hóa trong file firmware/src/capture/capture.pio.

- Tầng điều khiển và cấu hình (C++): Khởi tạo, cấu hình DMA và thiết lập tốc độ runtime trong file firmware/src/capture/capture_burst.cpp (và file cấu trúc đầu capture_burst.h).

2.2.1. Kiến trúc vòng lặp vô hạn và lệnh đọc song song một chu kỳ

Trọng tâm xử lý của State Machine (máy trạng thái) nằm ở đoạn mã Assembly PIO tối giản bên trong file capture.pio

Cơ chế thực thi:

- Đọc song song (Parallel Reading): Lệnh in pins, 24 cấu hình cho State Machine đọc đồng thời trạng thái logic của 24 chân GPIO liên tiếp (bắt đầu từ chân gốc base_pin) trong hệ thống. Dữ liệu của 24 chân này được nạp trực tiếp vào thanh ghi dịch đầu vào ISR (Input Shift Register). Cơ chế này khác biệt hoàn toàn với việc đọc tuần tự bằng phần mềm (bit-banging), giúp loại bỏ độ trễ lệch pha giữa các chân.
- Vòng lặp không hao tổn (Zero-Overhead Loop): Cặp chỉ thị .wrap_target và .wrap là tính năng phần cứng của khối PIO. Nó tạo ra một vòng lặp vô hạn tự động mà không tiêu tốn bất kỳ chu kỳ xung nhịp nào cho việc nhảy lệnh (JUMP). Vì toàn bộ chương trình chỉ vỏn vẹn đúng một lệnh in, hệ thống đạt hiệu suất tối đa: Cứ mỗi chu kỳ xung nhịp PIO trôi qua, đúng 1 mẫu dữ liệu (1 sample) được ghi nhận.

2.2.2 Định hình cấu trúc dữ liệu qua cơ chế dịch bit và Autopush

Trong khối hàm khởi tạo capture_program_init(), việc thiết lập cấu hình thanh ghi dịch đầu vào thông qua hàm sm_config_set_in_shift(&c, false, true, 24) đóng vai trò quyết định đến tính liên tục của luồng dữ liệu:

- Hướng dịch bit (shift_right = false): Cấu hình này ép dữ liệu khi nạp vào thanh ghi ISR sẽ thực hiện dịch trái. Mục đích là để giữ cho thứ tự bit logic khớp hoàn toàn với cấu trúc phần cứng của các kênh thu thập, đảm bảo bit thứ b đại diện cho chân GPIO(base_pin + b) nằm đúng vị trí theo công thức ánh xạ:

$$\text{bit}(CH_k) = 24 - k$$

- Cơ chế Tự động đẩy (autopush = true và threshold = 24): Đây là kỹ thuật cốt lõi để duy trì tốc độ một chu kỳ/mẫu. Thông thường, sau khi đọc dữ liệu vào ISR, ta cần một lệnh push để chuyển dữ liệu sang bộ đệm FIFO, việc này sẽ làm mất thêm chu kỳ xung nhịp. Với cấu hình này, ngay khi thanh ghi ISR tích lũy đủ 24 bit, phần cứng PIO tự động đóng gói và đẩy (push) nguyên một từ mã (word) 32-bit vào RX FIFO. State Machine ngay lập tức có thể quay lại thực thi lệnh in pins, 24 ở chu kỳ kế tiếp mà không bị trễ hay gián đoạn (no stalling)

2.2.3 Cơ chế điều khiển tốc độ lấy mẫu linh hoạt (Sampling Rate Control)

Tần số lấy mẫu f_{sample} của hệ thống được quyết định bởi tần số xung nhịp của chính khối PIO. Tần số này được thiết lập bằng bộ chia thời gian (Clock Divider - clkdiv) dựa trên xung nhịp hệ thống (clk_sys).

Nguyên lý toán học:

Mối quan hệ giữa tốc độ lấy mẫu mong muốn (sample_rate_hz) và xung nhịp hệ thống (sysclk) được biểu diễn qua công thức:

s_clkdiv = f_sys / f_sample

Do khối PIO của vi điều khiển hỗ trợ bộ chia cấu trúc số thực (Fractional Clock Divider), tốc độ lấy mẫu có thể điều chỉnh mịn với độ chính xác cao. Tuy nhiên, giới hạn vật lý tối đa của hệ thống là một mẫu trên một chu kỳ xung nhịp hệ thống. Do đó, điều kiện biên if (s_clkdiv < 1.0f) s_clkdiv = 1.0f; đảm bảo hệ số chia không bao giờ nhỏ hơn 1.0. Khi s_clkdiv = 1.0f, hệ thống hoạt động ở chế độ "Full Speed", tốc độ lấy mẫu bằng chính tần số xung nhịp hệ thống f_sample = f_sys

2.2.4 Luồng truyền tải dữ liệu tối ưu qua DMA (Zero-CPU Overhead)

Để bộ đệm RX FIFO của PIO (vốn chỉ có dung lượng giới hạn) không bị tràn khi dữ liệu liên tục đổ về ở tốc độ cao, một kênh DMA (Direct Memory Access) được cấu hình và kích hoạt trong hàm capture_burst_run().

Cơ chế bắt tay phần cứng (Hardware Handshaking DREQ):

Kênh DMA liên tục lắng nghe tín hiệu yêu cầu phần cứng pio_get_dreq(). Mỗi khi cơ chế *Autopush* của PIO đẩy thành công một gói dữ liệu 32-bit vào bộ đệm FIFO &s_pio->rxf[s_sm], tín hiệu DREQ sẽ được kích hoạt. Bộ điều khiển DMA ngay lập tức tự động "bốc" dữ liệu này từ FIFO và ghi trực tiếp vào mảng lưu trữ nằm trên bộ nhớ RAM nội (SRAM). Toàn bộ quá trình dịch chuyển dữ liệu này diễn ra ở tầng phần cứng dưới sự điều phối của Bus Fabric, CPU không cần thực thi bất kỳ dòng lệnh nào trong suốt quá trình capture, đạt hiệu suất tối ưu 0% tải CPU.

CƠ CHẾ DMA CHUYỂN DỮ LIỆU TỐC ĐỘ CAO TỪ PIO VÀO BỘ NHỚ SRAM KHÔNG QUA CPU

Trong các hệ thống thu thập dữ liệu tốc độ cao, việc dịch chuyển dữ liệu từ bộ đệm của khối ngoại vi vào bộ nhớ hệ thống (SRAM) nếu thực hiện bằng CPU (qua các lệnh

đọc/ghi thông thường) sẽ gây ra thắt nút cổ chai nghiêm trọng. Hệ thống đã giải quyết triệt để bài toán này bằng cách cấu hình khối **DMA (Direct Memory Access)** phối hợp đồng bộ với PIO, đạt hiệu suất tối đa mà không tiêu tốn tài nguyên xử lý của CPU.

2.2.5 Phân cấp và tối ưu hóa độ ưu tiên trên Bus Hệ thống (Bus Priority)

Một trong những điểm mấu chốt nhất để đạt tốc độ lấy mẫu cực đại trong đoạn mã là việc can thiệp trực tiếp vào bộ điều khiển Bus của vi điều khiển (bus_ctrl_hw)

Ý nghĩa kỹ thuật: Khi khối PIO hoạt động ở xung nhịp tối đa (ví dụ 150 MHz), nó sẽ liên tục đẩy ra 1 word (32-bit) trên mỗi chu kỳ, tương đương băng thông dữ liệu lên tới 150 MW/s (Mega-words per second), xấp xỉ 600 MB/s.

Nếu không thiết lập ưu tiên: Qua thực nghiệm chứng minh băng thông DMA bị giới hạn ở mức ~ 125 MW/s do phải chia sẻ và tranh chấp tài nguyên Bus Fabric với CPU và các ngoại vi khác. Điều này dẫn đến việc bộ đệm RX FIFO của PIO bị tràn, gây ra hiện tượng nghẽn (PIO stall) và làm sai lệch tốc độ lấy mẫu thực tế.

Giải pháp: Dòng lệnh trên nâng quyền ưu tiên của khối DMA (cả kênh đọc lẫn kênh ghi) lên mức tuyệt đối. DMA luôn luôn chiến thắng trong các cuộc tranh chấp bus (Bus Contention), đảm bảo luồng truyền tải thông suốt và đạt đúng tốc độ lý thuyết 150 MSa/s.

2.2.6 Cấu hình định tuyến luồng dữ liệu DMA

Hệ thống thiết lập một kênh DMA chuyên dụng (được cấp phát động qua dma_claim_unused_channel) với các tham số tối ưu hóa cho việc đọc ghi liên tục:

- Kích thước dữ liệu (DMA_SIZE_32): Định hình luồng truyền tải theo từng phân đoạn 32-bit (1 word), khớp hoàn toàn với cấu trúc dữ liệu được đóng gói tự động từ thanh ghi dịch đầu vào (ISR) của PIO.
- Địa chỉ nguồn (Source): Trỏ cố định vào thanh ghi nhận của PIO &s_pio->rxf[s_sm]. Thuộc tính tự động tăng địa chỉ đọc được tắt (read_increment = false) vì dữ liệu luôn được lấy ra tại một cổng FIFO duy nhất.
- Địa chỉ đích (Destination): Trỏ vào mảng bộ đệm buf nằm trên SRAM. Thuộc tính tự động tăng địa chỉ ghi được bật (write_increment = true) để dữ liệu xếp liên tiếp vào RAM.
- Nhịp đồng bộ phần cứng (DREQ): Kích hoạt qua hàm channel_config_set_dreq(&c, pio_get_dreq(..., false)). DMA sẽ không tự ý đọc bộ đệm mà nằm ở trạng thái chờ cho đến khi nhận được tín hiệu yêu cầu phần cứng (Hardware Request - DREQ) phát ra từ khối PIO RX FIFO. Mỗi khi PIO Autopush

đủ 24-bit (điền đầy 1 word), DREQ lập tức kích hoạt, lệnh cho DMA "bốc" dữ liệu đi ngay lập tức.

2.2.7 Chiến lược thăm dò (Polling) giảm thiểu tranh chấp Bus

Sau khi kích hoạt lệnh chạy (dma_channel_configure), toàn bộ quá trình dịch chuyển dữ liệu từ chân GPIO → PIO FIFO → SRAM hoàn toàn do phần cứng tự vận hành. Tuy nhiên, CPU vẫn phải giám sát khi nào tiến trình này hoàn tất (đủ số lượng mẫu depth).

Thay vì sử dụng hàm tiêu chuẩn dma_channel_wait_for_finish_blocking(), đoạn mã triển khai một vòng lặp kiểm tra trạng thái thông minh.

Chiến lược thăm dò (Polling) giảm thiểu tranh chấp Bus

Sau khi kích hoạt lệnh chạy (dma_channel_configure), toàn bộ quá trình dịch chuyển dữ liệu từ chân GPIO → PIO FIFO → SRAM hoàn toàn do phần cứng tự vận hành. Tuy nhiên, CPU vẫn phải giám sát khi nào tiến trình này hoàn tất (đủ số lượng mẫu depth).

Thay vì sử dụng hàm tiêu chuẩn dma_channel_wait_for_finish_blocking(), đoạn mã triển khai một vòng lặp kiểm tra trạng thái thông minh.

Phân tích giải pháp:

Hàm đợi chặn tiêu chuẩn (wait_for_finish_blocking) hoạt động bằng cách liên tục đọc (poll) thanh ghi trạng thái của DMA với tần suất dày đặc. Việc CPU liên tục truy vấn vào hệ thống thanh ghi ngoại vi vô tình tạo ra một lượng "rác" tín hiệu khổng lồ trên ma trận bus (AHB Crossbar Matrix), gián tiếp tranh chấp băng thông với chính kênh DMA đang kéo dữ liệu 150 MW/s.

Bằng cách chèn lệnh trễ busy_wait_us_32(10), CPU chỉ kiểm tra lại trạng thái sau mỗi 10 µs. Sự "thưa thớt" có tính toán này giúp giải phóng hoàn toàn đường truyền Bus Fabric, dành trọn vẹn băng thông cho DMA ghi dữ liệu vào SRAM mà không bị gián đoạn.

2.2.8 Cơ chế bảo vệ và Ngắt an toàn (Timeout & Abort)

Do hệ thống tích hợp tính năng kích hoạt bằng tín hiệu ngoài (Hardware Trigger qua chân GPIO), sẽ có trường hợp hệ thống bật chế độ chờ nhưng tín hiệu Trigger không bao giờ xuất hiện, dẫn đến việc DMA rơi vào trạng thái treo vô hạn.

Hệ thống giải quyết rủi ro này bằng cơ chế quản lý thời gian thực (absolute_time_t deadline). Khi vượt quá thời gian thiết lập (timeout_ms), CPU sẽ chủ động can thiệp bằng chuỗi lệnh hạ tải an toàn:

- Dừng State Machine của PIO: pio_sm_set_enabled(..., false) để ngừng thu thập mẫu mới.
- Hủy tiến trình DMA: Gọi lệnh dma_channel_abort(s_dma) để ép kênh DMA dừng khẩn cấp, giải phóng trạng thái bận và ngăn chặn việc ghi đè dữ liệu không kiểm soát vào RAM.
- Xóa sạch bộ đệm: pio_sm_clear_fifos() để dọn dẹp các dữ liệu rác còn sót lại, đưa hệ thống về trạng thái sẵn sàng cho phiên làm việc kế tiếp.

2.3 Mạch dịch mức logic (Level Shifter)

2.3.1 Tổng quan về nhu cầu dịch mức logic (Logic Level Shifting)

Trong kỹ thuật số và hệ thống nhúng, các IC và ngoại vi hoạt động dựa trên nhiều tiêu chuẩn điện áp logic khác nhau, phổ biến nhất là 1.8V (các chip nhớ tốc độ cao, CPU đời mới), 3.3V (hầu hết các vi điều khiển hiện đại như STM32, RP2350) và 5V (các linh kiện công nghiệp, cảm biến cũ hoặc chuẩn TTL truyền thống).

Khi một vi điều khiển (MCU) hoạt động ở mức 3.3V cần đo kiểm hoặc giao tiếp với một bus logic ngoại vi, sự bất tương thích về ngưỡng điện áp nhận diện logic (V_IH, V_IL, V_OH, V_OL) sẽ dẫn đến hai nguy cơ lớn:

- Sai lệch tín hiệu: Mức cao (V_OH) của thiết bị 1.8V không đủ ngưỡng kích hoạt mức cao (V_IH) của MCU 3.3V.
- Phá hủy phần cứng: Tín hiệu 5V từ bus ngoài xông trực tiếp vào chân IO không chịu áp 5V (Non-5V-tolerant) của MCU sẽ làm hỏng lớp oxit cổng của transistor bên trong chip.

Do đó, một mạch dịch mức logic (Level Shifter) đóng vai trò là cầu nối trung gian, vừa chuyển đổi biên độ điện áp logic một cách chính xác, vừa cô lập về mặt điện động giữa hai hệ thống.

2.3.2 Nguyên lý hoạt động của IC dịch mức SN74LVC8T245

SN74LVC8T245 là một IC dịch mức logic và đệm bus (Bus Transceiver) 8-bit cấu trúc nguồn cấp kép (Dual-Supply) hiệu năng cao. Đây là linh kiện lý tưởng cho các thiết bị đo lường logic (Logic Analyzer) nhờ các đặc tính kỹ thuật linh hoạt.

Kiến trúc nguồn kép độc lập (Dual-Supply Architecture)

IC sở hữu hai đường nguồn cấp hoàn toàn độc lập:

- V_CCA (Nguồn cấp cho cổng A): Thường được kết nối với đường nguồn của hệ thống xử lý trung tâm (MCU/FPGA), ví dụ cố định ở 3.3V.
- V_CCB (Nguồn cấp cho cổng B): Được kết nối với đường nguồn của thiết bị hoặc bus cần đo ngoài môi trường (Target Bus).

Cả hai dải nguồn đều hỗ trợ dải điện áp cực rộng từ 1.65 V đến 5.5 V. Sự độc lập này cho phép IC dịch chuyển bất kỳ mức logic nào giữa hai cổng:

1.8 V ↔ 3.3V

3.3 V↔ 5 V

1.8V ↔ 5\V

Các chân điều khiển chức năng

Việc định hướng luồng dữ liệu và trạng thái của IC được quyết định bởi hai chân điều khiển:

- DIR (Direction Control): Quyết định hướng truyền dẫn của bus dữ liệu.
    - Khi DIR = HIGH: Dữ liệu truyền từ cổng A sang cổng B (MCU xuất tín hiệu ra ngoài).
    - Khi DIR = LOW: Dữ liệu truyền từ cổng B sang cổng A (MCU đọc tín hiệu từ bus ngoài vào). Trong các ứng dụng đo lường/capture, chân này luôn được cố định ở mức LOW.
- $\overline{OE}$ (Output Enable - Tích cực mức thấp):
    - Khi $\overline{OE}$ = LOW: IC hoạt động bình thường, các cổng IO thiết lập theo chân DIR.
    - Khi $\overline{OE}$ = HIGH: Tất cả các chân IO ở cả hai cổng A và B lập tức rơi vào trạng thái trở kháng cao (High-Z). Trạng thái này cô lập hoàn toàn về mặt điện cơ giữa MCU và thế giới bên ngoài.

2.4 Giao tiếp USB CDC và giao thức Serial

2.4.1 Tối ưu hóa tầng truyền dẫn phần cứng trên Firmware

Giải pháp kỹ thuật mấu chốt:

Thay vì sử dụng hàm xuất dữ liệu tiêu chuẩn printf(), hệ thống bắt buộc phải gọi trực tiếp hàm hạ tầng stdio_usb.out_chars(). Giải pháp này giải quyết hai bài toán cốt lõi:

- Loại bỏ thắt nút cổ chai do Mirror UART: Mặc định, hàm printf() của SDK sẽ sao chép (mirror) toàn bộ dữ liệu xuất ra cả cổng USB lẫn cổng UART vật lý (đang chạy ở baudrate thấp 115200 bps). Khi truyền một lượng lớn dữ liệu, việc chờ đợi cổng UART giải phóng bộ đệm sẽ gây nghẽn toàn bộ hệ thống, khiến tiến trình truyền dữ liệu bị kéo dài thêm khoảng 5 giây. Ghi trực tiếp vào driver USB CDC giúp bỏ qua hoàn toàn cơ chế sao chép này.
- Tránh phá hủy dữ liệu nhị phân (Bypass CRLF Translation): Hàm printf() tiêu chuẩn có cơ chế tự động chuyển đổi ký tự xuống dòng từ LF (\n hay 0x0A) thành chuỗi CRLF (\r\n hay 0x0D 0x0A). Trong chế độ truyền dữ liệu nhị phân, nếu một mẫu dữ liệu ngẫu nhiên chứa byte mang giá trị 0x0A, printf() sẽ tự chèn thêm byte 0x0D vào trước nó, làm sai lệch và hỏng hoàn toàn cấu trúc tệp tin dữ liệu.

2.4.2 Cấu trúc đóng khung dữ liệu (Data Framing Modes)

Hệ thống hỗ trợ song song hai chế độ đóng khung dữ liệu thông qua hàm điều khiển trung tâm do_burst_dump(bool binary), cho phép linh hoạt chuyển đổi giữa tính tường minh (ASCII) và hiệu năng cực đại (Binary).

Cơ chế Gom khối (Chunked Buffering) trong ASCII: Để giảm thiểu tần suất gọi driver USB (vốn tốn chi phí chuyển đổi ngữ cảnh hệ thống), phần firmware mã hóa ASCII không gửi từng dòng riêng lẻ mà gom dữ liệu thành từng khối 128 dòng văn bản rồi mới thực hiện gửi một lần qua usb_write_raw().

2.4.3 Cơ chế Xử lý và Parse khung dữ liệu phía PC Client

Phía PC ứng dụng thư viện re (Regular Expression) của Python để xây dựng một bộ lọc phân tích luồng dữ liệu theo kiến trúc máy trạng thái (State Machine parsing).

Chiến lược xử lý bất đồng bộ và chống nhiễu (Noise-Tolerant)

- Chống nhiễu khi khởi động: Khi bo mạch vừa bật nguồn hoặc reset, nó có thể xuất ra các dòng text rác, thông báo gỡ lỗi (debug logs) hoặc thông tin bootloader. Hàm request_burst() và request_burst_bin() trên PC được thiết kế theo cơ chế Noise-Tolerant: Client sẽ liên tục đọc và loại bỏ mọi dòng dữ liệu rác cho đến khi bắt được chính xác chuỗi Header quy định bởi BEGIN_RE hoặc BIN_RE.
- Kỹ thuật xử lý luồng nhị phân đặc biệt: Khi chạy chế độ Binary, sau khi nhận được dòng Header (kết thúc bằng \n), Client lập tức ngắt cơ chế đọc theo dòng

(readline). Lý do là vì trong khối dữ liệu nhị phân thô, byte dữ liệu mang giá trị 0x0A (trùng với ký tự dòng mới \n) xuất hiện hoàn toàn ngẫu nhiên. Nếu tiếp tục đọc theo dòng, thuật toán sẽ cắt khúc dữ liệu sai vị trí.

Thay vào đó, Client tính toán chính xác tổng số lượng byte cần nhận:

Total Bytes = depth × 4 bytes

Sau đó, Client gọi hàm đọc thô đúng số lượng byte này trực tiếp từ luồng truyền dẫn, rồi mới quay lại chế độ đọc dòng để xác thực chuỗi kết thúc BIN_END.

2.4.4 Tập lệnh điều khiển và Bắt tay hệ thống (Control & Handshaking)

Tầng ứng dụng tương tác theo mô hình Yêu cầu - Đáp ứng (Request - Response). Vòng lặp main() trên firmware liên tục nhận và giải mã các lệnh đơn ký tự từ PC:

- I (Identity): Yêu cầu định danh thiết bị. Board sẽ trả về chuỗi thông tin định dạng chuẩn: ID name=RP2354-LA fw=... proto=.... Bước này giúp PC xác thực đã kết nối đúng loại phần cứng máy phân tích logic.
- R<hz> (Rate): Thiết lập tần số lấy mẫu. Tham số số nguyên phía sau được trích xuất bằng hàm read_uint_arg().
- D<n> (Depth): Thiết lập độ sâu bộ đệm (số lượng mẫu cần capture).
- T<ch><R|F> (Trigger): Cấu hình chân trigger (ch) và sườn kích hoạt (R: Cạnh lên, F: Cạnh xuống).
- Phản hồi trạng thái (Handshaking): Đối với các lệnh cấu hình tham số (R, D, T), Client sử dụng hàm send_cmd_expect_ok() để gửi lệnh và bắt buộc phải đợi firmware phản hồi dòng chữ OK. Nếu cấu hình sai (quá giới hạn phần cứng), firmware sẽ trả về mã lỗi chuẩn hóa dạng ERR_<MÃ_LỖI> (ví dụ: lỗi vượt giới hạn bộ chia xung nhịp), giúp hệ thống lập tức khoanh vùng và xử lý ngoại lệ an toàn

2.5 Giao tiếp SPI và hiển thị LCD ST7789

2.5.1 Giao tiếp SPI và cấu hình phần cứng LCD ST7789

2.5.1.1 Giao thức SPI 4-Wire trong điều khiển hiển thị

Giao tiếp giữa vi điều khiển (như dòng chip RP2350/RP2354) và IC điều khiển màn hình ST7789 sử dụng chuẩn giao tiếp đồng bộ ngoại vi SPI (Serial Peripheral Interface) cấu hình 4 chân tín hiệu chủ động (4-Wire):

- SCK (Serial Clock): Xung giữ nhịp đồng bộ cho quá trình truyền dữ liệu.
- MOSI (Master Out Slave In): Đường truyền dữ liệu một chiều từ vi điều khiển xuống LCD.
- CS (Chip Select): Chân chọn chip, tích cực ở mức thấp (0), cho phép cô lập hoặc kích hoạt giao tiếp với LCD trên đường bus chung.
- D/C hoặc RS (Data/Command Control): Đây là chân chức năng đặc trưng của các dòng driver màn hình. Chân này quyết định byte dữ liệu đang truyền qua đường MOSI là Lệnh cấu hình (khi D/C = 0) hay Dữ liệu hiển thị/Tham số (khi D/C = 1).

2.5.1.2 Định dạng cấu hình SPI

Để đảm bảo ST7789 nhận dạng chính xác từng bit dữ liệu, bộ ngoại vi SPI được cấu hình ở Mode 0 (CPOL = 0, CPHA = 0). Ở chế độ này:

- Trạng thái nghỉ của đường xung giữ nhịp SCK luôn ở mức thấp.
- Dữ liệu được vi điều khiển chuẩn bị trên đường MOSI và được IC ST7789 chốt (sample) ở cạnh lên (rising edge) của xung clock.
- Tần số xung đạt mức tốc độ cao (thường từ 20 MHz đến 30 MHz) nhằm tối ưu hóa băng thông, đáp ứng tốc độ quét màn hình cao (FPS lớn) cho việc hiển thị thời gian thực.

2.5.2 Nguyên lý hoạt động của Driver màn hình LCD TFT ST7789

2.5.2.1 Quản lý bộ nhớ đồ họa GRAM (Graphics RAM)

ST7789 tích hợp một bộ nhớ đệm đồ họa tĩnh chuyên biệt gọi là GRAM. Trạng thái màu sắc của từng pixel trên tấm nền TFT được ánh xạ trực tiếp từ bộ nhớ này. Đối với cấu hình màn hình xoay ngang (Landscape) độ phân giải 320 × 240 pixel, tổng số điểm ảnh cần quản lý là 76.800 điểm.

2.5.2.2 Mã hóa màu sắc theo chuẩn RGB565

Hệ thống sử dụng không gian màu định dạng 16-bit RGB565 để tối ưu dung lượng truyền tải dữ liệu trên mỗi pixel nhưng vẫn đảm bảo độ trung thực hình ảnh (65.536 màu):

- Red (Màu đỏ): Chiếm 5 bit (D_{11} ... D_{15}).
- Green (Màu xanh lá): Chiếm 6 bit (D_5 ... D_{10}), được ưu tiên thêm 1 bit vì mắt người nhạy cảm nhất với dải màu sắc này.
- Blue (Màu xanh dương): Chiếm 5 bit (D_0 ... D_4).

Mỗi pixel hiển thị sẽ tiêu tốn chính xác 2 byte dữ liệu truyền qua bus SPI.

2.5.2.3 Cơ chế cửa sổ địa chỉ (Address Window) và Tự động tăng con trỏ

Đây là cơ chế cốt lõi giúp tăng tốc độ xử lý đồ họa của ST7789. Thay vì phải chỉ định tọa độ (X, Y) cho từng pixel đơn lẻ (gây trễ băng thông lớn), driver thiết lập một vùng cửa sổ giới hạn bằng cách gửi chuỗi tập lệnh phần cứng:

- CASET (Column Address Set - 0x2A): Định nghĩa cột bắt đầu X_0 và cột kết thúc X_1.
- RASET (Row Address Set - 0x2B): Định nghĩa hàng bắt đầu Y_0 và hàng kết thúc Y_1.
- RAMWR (Memory Write - 0x2C): Kích hoạt chế độ ghi vào vùng nhớ GRAM.

Sau lệnh RAMWR, luồng dữ liệu màu 16-bit gửi xuống liên tục sẽ tự động điền đầy cửa sổ từ trái sang phải, từ trên xuống dưới nhờ vào bộ đếm con trỏ phần cứng tự động tăng tích hợp bên trong ST7789.

2.5.2.4 Chu trình khởi tạo và kỹ thuật chống nhiễu hiển thị

- Cơ chế Reset cứng (Hardware Reset): Sử dụng chân RST kéo xuống mức thấp rồi đưa lên mức cao kèm các khoảng trễ (sleep_ms) theo đặc tính kỹ thuật nhằm đưa toàn bộ thanh ghi nội bộ của IC về trạng thái định nghĩa gốc ổn định.
- Cấu hình hướng quét MADCTL (0x36): Điều khiển thứ tự quét dòng và quét cột của RAM điều khiển, cho phép cấu hình lật góc hiển thị (90°, 180°, 270°) để chuyển đổi linh hoạt giữa chế độ Portrait và Landscape.
- Kỹ thuật quản lý Backlight chống chớp màn: Trong quá trình khởi động và nạp cấu hình (Sleep Out 0x11, Display Inversion 0x21, Display On 0x29), bộ nhớ GRAM thường chứa dữ liệu rác gây ra hiện tượng màn hình bị chớp trắng hoặc sọc nhiễu. Kỹ thuật tối ưu là cô lập chân điều khiển đèn nền màn hình (PIN_BL = 0) ở trạng thái TẮT trong suốt chu trình init. Đèn nền chỉ được kích hoạt (PIN_BL = 1) sau khi toàn bộ GRAM đã được xóa nền về màu đen (COLOR_BLACK), mang lại trải nghiệm thị giác mượt mà khi cấp nguồn.

2.5.3 Kỹ thuật hiển thị đồ họa dạng sóng trực quan (Logic Analyzer)

2.5.3.1 Ánh xạ dữ liệu số sang tọa độ X-Y (Time-to-Digital Mapping)

Dữ liệu thô (raw_data) thu thập từ các chân GPIO được lưu trữ dưới dạng các byte/word. Để hiển thị, hệ thống thực hiện tách bit (Bitwise extraction) tương ứng với vị trí kênh đang chọn để lấy ra trạng thái logic (0 hoặc 1).

- Trục hoành X (Trục thời gian): Ánh xạ trực tiếp vị trí index của mẫu dữ liệu trong bộ đệm (Sample Index) thành tọa độ điểm ảnh theo chiều ngang (X_pixel = X_base + index).
- Trục tung Y (Trục biên độ): Được phân chia thành các vùng cố định cho từng kênh. Mức điện áp cao (HIGH) tương ứng với tọa độ đồ họa Y_high, mức thấp (LOW) tương ứng với tọa độ Y_low (lưu ý tọa độ Y đồ họa có gốc (0, 0) ở góc trên cùng bên trái màn hình).

2.5.3.2 Giải pháp tối ưu hóa "Đường đi nhanh" (Fast-path Drawing) cho tín hiệu số

Xung tín hiệu số chỉ có hai trạng thái tĩnh (HIGH/LOW) và các sườn chuyển tiếp (sườn lên/sườn xuống vuông góc). Do đó, dạng sóng được cấu thành từ:

- Các đoạn thẳng nằm ngang song song với trục X.
- Các đoạn thẳng đứng vuông góc nối giữa Y_high và Y_low tại thời điểm đổi trạng thái.

Thay vì sử dụng thuật toán đường chéo tổng quát như Bresenham (vốn tính toán dấu và bước nhảy tốn tài nguyên CPU), hệ thống tăng tốc bằng giải pháp đường đi nhanh thông qua hàm vẽ khối (fill_rect với chiều cao hoặc chiều rộng bằng 1 pixel):

- Đoạn ngang: Được vẽ bằng một khối chữ nhật có chiều cao bằng 1, tận dụng cơ chế ghi bộ nhớ hàng loạt của lệnh set_address_window. Vi điều khiển chỉ cần gửi lệnh tọa độ một lần và đẩy chuỗi dữ liệu màu liên tiếp giúp tối đa hóa băng thông bus SPI.
- Đoạn đứng: Được vẽ bằng một khối chữ nhật có chiều rộng bằng 1 từ Y_low đến Y_high.

2.5.3.3 Kỹ thuật làm tươi cục bộ (Local Refresh) tránh nhấp nháy màn hình

Để đạt tần số khung hình cao, hệ thống tuyệt đối không xóa toàn bộ màn hình (clear) trước mỗi chu kỳ vẽ dạng sóng mới, vì việc này làm lãng phí thời gian truyền tải 153.600 byte dữ liệu nền đen và gây ra hiện tượng nhấp nháy (flicker) khó chịu.

Biện pháp tối ưu là áp dụng Làm tươi cục bộ (Partial Update): Hệ thống chỉ dùng lệnh fill_rect để xóa đúng phạm vi hình chữ nhật chứa luồng sóng cũ (WAVE_X đến cạnh phải), sau đó vẽ đè đường tham chiếu nhiễu (Grid) mờ và dạng sóng mới lên. Các thành phần tĩnh cấu trúc giao diện như nhãn kênh (CH1, CH2), số hiệu chân GPIO (G2, G3) và các thông số Status Bar hoàn toàn được giữ nguyên, không cần nạp lại.

2.5.3.4 Kiến trúc ma trận nén đồ họa đa kênh (Overview Mode Matrix)

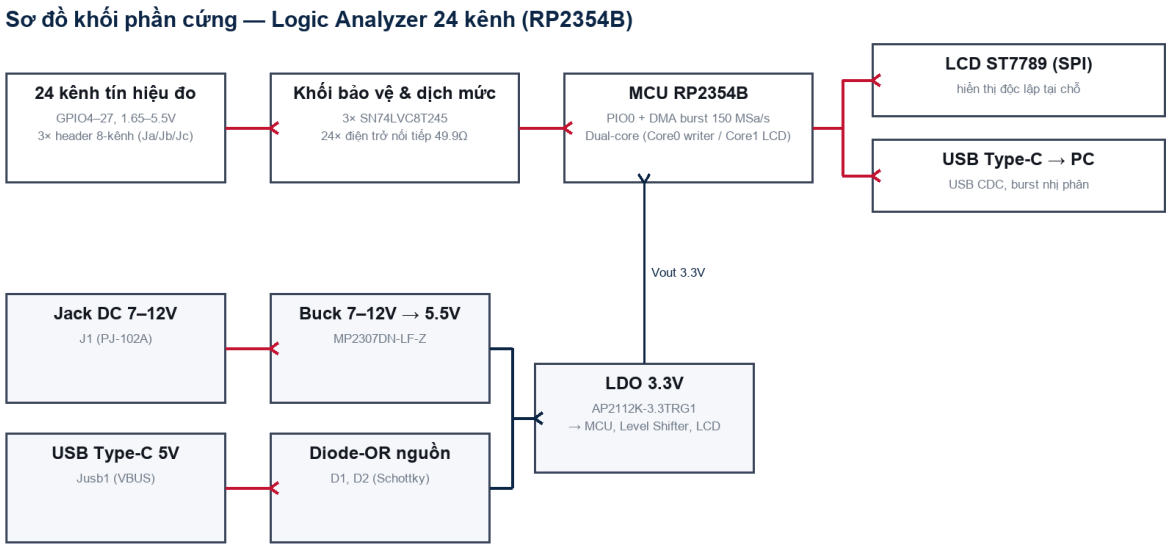
Khi chuyển sang chế độ tổng quan hiển thị đồng thời cả 24 kênh trên cùng một màn hình đơn (320 × 240), kỹ thuật nén không gian đồ họa được áp dụng:

- Màn hình được phân chia theo cấu trúc ma trận hình học cố định (2 cột × 12 hàng).
- Mỗi kênh được cấp phát một vùng không gian hẹp với chiều rộng xác định (136 px) và chiều cao tối giản (18 px). Biên độ dạng sóng số được thu nhỏ (độ cao xung dao động trong khoảng 11 px nối giữa Y_high và Y_low nội bộ của hàng đó).
- Kỹ thuật ánh xạ dịch bit tuần tự (raw_data[x] >> bit_pos) chạy đồng thời trên cấu trúc lặp ma trận giúp bộ nạp hiển thị nhanh trích xuất toàn vẹn bức tranh trạng thái của toàn bộ hệ thống bus trong một chu kỳ màn hình duy nhất.

3.THIẾT KẾ VÀ TRIỂN KHAI PHẦN CỨNG

3.1. Sơ đồ khối phần cứng

Phần cứng của thiết bị được tổ chức thành bốn khối chức năng chính, phối hợp theo luồng tín hiệu: khối bảo vệ/dịch mức tiếp nhận 24 kênh tín hiệu số cần đo, chuyển tới vi điều khiển trung tâm RP2354B để lấy mẫu tốc độ cao; dữ liệu sau đó được hiển thị độc lập ngay trên màn hình LCD tích hợp, hoặc truyền qua USB về máy tính để phân tích sâu hơn. Một khối nguồn riêng biệt hỗ trợ đồng thời hai nguồn cấp (jack DC 7–12V hoặc USB Type-C 5V) để thiết bị có thể hoạt động ở cả chế độ standalone lẫn khi kết nối PC.

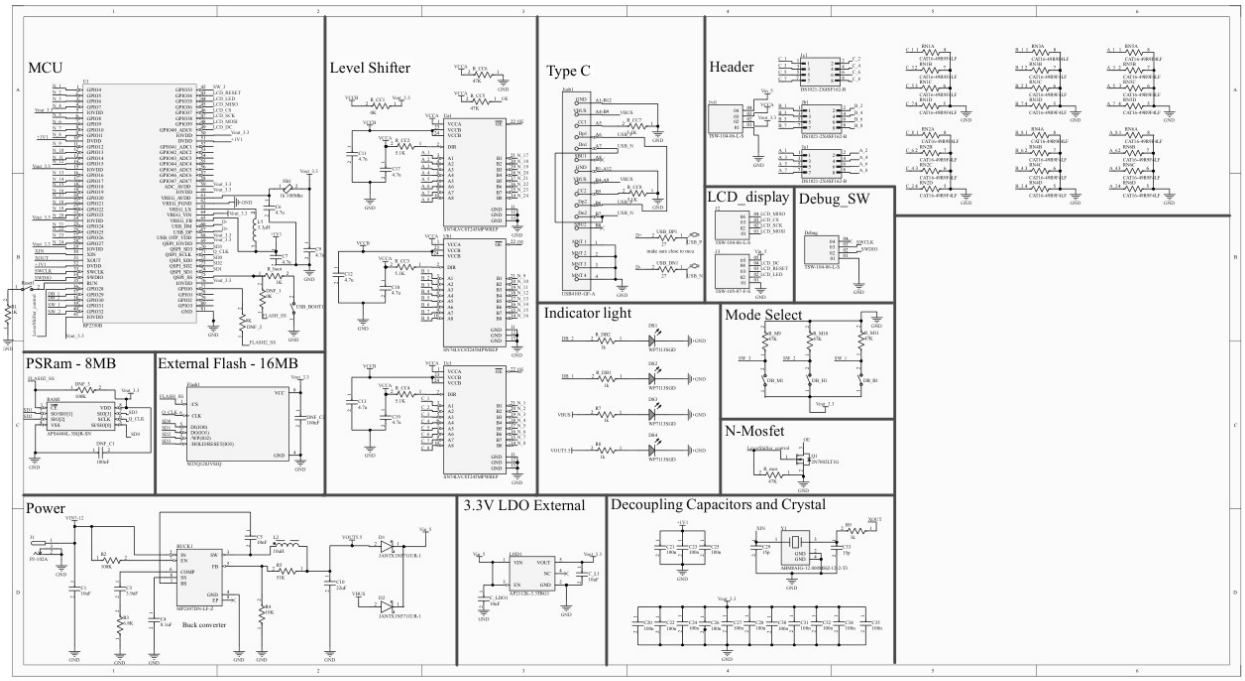


Hình 3.1 — Sơ đồ khối phần cứng của thiết bị Logic Analyzer 24 kênh

- Khối bảo vệ & dịch mức: tiếp nhận trực tiếp tín hiệu ngoài (1.65–5.5V), bảo vệ MCU và đưa tín hiệu về đúng mức logic 3.3V.
- Khối MCU RP2354B: trung tâm xử lý — lấy mẫu 24 kênh song song bằng PIO+DMA, chạy máy trạng thái, điều phối hiển thị và giao tiếp USB.
- Khối hiển thị LCD ST7789: cho phép xem nhanh dạng sóng ngay tại thiết bị, không bắt buộc phải có PC.
- Khối giao tiếp USB: truyền dữ liệu burst tốc độ cao về PC để phân tích/lưu trữ.
- Khối nguồn: gộp hai nguồn cấp (DC jack hoặc USB Type-C) qua diode-OR, hạ áp và ổn áp về 3.3V nuôi toàn bộ hệ thống.

3.2. Thiết kế mạch nguyên lý (Schematic)

Toàn bộ mạch nguyên lý được thiết kế trên một board 4 khối chính (MCU, Level Shifter, Type-C, Power) cùng các khối phụ trợ (LCD, nút bấm, đèn báo, bộ nhớ ngoài). Hình 3.2 là bản schematic tổng thể.



Hình 3.2 — Schematic tổng thể của board Logic Analyzer

3.2.1. Tầng đệm dịch mức (Level Shifter)

24 kênh tín hiệu đo được chia đều qua 3 header 8-kênh (Ja1, Jb1, Jc1 — DS1021-2X4SF162-B), mỗi kênh đi qua một điện trở nối tiếp 49.9Ω (6 gói điện trở CAT16-49R9F4LF, mỗi gói 4 điện trở) trước khi vào 3 IC dịch mức SN74LVC8T245MPWREP (Ua1, Ub1, Uc1 — mỗi IC 8 kênh, cố định chiều A→B từ phía tín hiệu ngoài vào phía MCU). Điện trở nối tiếp có vai trò giới hạn dòng và giảm phản xạ tín hiệu (signal integrity) ở tần số lấy mẫu cao.

Ba IC được bật/tắt đồng thời qua chân OE chung, điều khiển bởi MOSFET kênh N (Q1, 2N7002LT1G) — MCU chỉ cần 1 chân GPIO (LeverShifter_control) để bật/tắt toàn bộ tầng đệm, phục vụ tính năng tiết kiệm điện / cách ly khi không sử dụng.

3.2.2. Khả năng chịu điện áp và cơ chế bảo vệ vi điều khiển

Do thiết bị đo phải tiếp xúc trực tiếp với mạch ngoài chưa rõ mức điện áp, khả năng chịu áp và cơ chế bảo vệ là yêu cầu thiết kế bắt buộc. Bảng 3.1 tổng hợp các giới hạn điện của IC dịch mức SN74LVC8T245 — linh kiện đứng ở tuyến đầu tiếp nhận tín hiệu ngoài.

Thông số giới hạn	Giá trị
Điện áp hoạt động khuyến nghị (V_CCA, V_CCB và tín hiệu vào)	1.65 – 5.5 V
Điện áp tuyệt đối tối đa trên chân A/B (Absolute Maximum)	−0.5 V … +6.5 V
Dòng kẹp ngõ vào / ngõ ra tối đa (I_IK, I_OK)	±50 mA
Dòng liên tục qua chân V_CC hoặc GND	±100 mA
Khả năng chịu phóng tĩnh điện (ESD — mô hình HBM)	2 kV

Bảng 3.1 — Giới hạn điện của IC dịch mức SN74LVC8T245

Trong dải hoạt động bình thường, thiết bị đo được mọi chuẩn logic phổ biến từ 1.65 V đến 5.5 V (bao gồm 1.8 V, 2.5 V, 3.3 V và 5 V). Khi xảy ra sự cố quá áp, điện trở nối tiếp 49.9 Ω trên mỗi kênh đóng vai trò ghìm dòng: điện áp vượt ngưỡng sẽ làm diode kẹp tĩnh điện bên trong IC dẫn, và dòng qua diode được giới hạn bởi chính điện trở này:

$$I_{clamp} = (V_{in} - V_{CCA} - 0.5) / R_s$$

Thay giới hạn I_{clamp} = 50 mA và R_s = 49.9 Ω, ngưỡng điện áp sự cố mà tầng đệm chịu được là:

$$V_{in(max)} = V_{CCA} + 0.5 + 0.05 \times 49.9 \approx V_{CCA} + 3.0 \ (V)$$

Tương ứng, khi cấp V_CCA = 3.3 V thì tầng đệm chịu được quá áp tức thời tới khoảng 6.3 V; khi cấp V_CCA = 5 V thì ngưỡng này nâng lên khoảng 8 V. Vượt quá giá trị trên (ví dụ cắm nhầm nguồn 12 V vào que đo), IC dịch mức sẽ hỏng.

Vấn đề đặt ra là khi IC dịch mức hỏng, vi điều khiển có bị hỏng theo hay không. Thiết kế giải quyết câu hỏi này bằng cơ chế phòng vệ nhiều lớp (defense-in-depth):

- Lớp 1 — Điện trở nối tiếp 49.9 Ω trên từng kênh: giới hạn dòng sự cố ngay tại ngõ vào, không để dòng lớn chạy vào cấu trúc bán dẫn của IC.
- Lớp 2 — IC SN74LVC8T245 với hai miền nguồn tách biệt: phía A lấy nguồn từ mức điện áp tín hiệu ngoài, phía B lấy nguồn 3.3 V của MCU. Hai miền chỉ liên hệ với nhau qua tầng đệm bán dẫn bên trong, nên biên độ tín hiệu đưa sang MCU luôn bị giới hạn bởi V_CCB = 3.3 V bất kể điện áp phía ngoài lớn tới đâu. IC vì vậy đóng vai trò linh kiện hy sinh.
- Lớp 3 — Các chân GPIO4–27 của RP2354B dùng làm kênh đo thuộc nhóm chân chịu được điện áp tới 5.5 V, tạo thêm một biên an toàn phía sau IC dịch mức.
- Lớp 4 — Chân OE của cả ba IC được điều khiển bằng MOSFET Q1 qua một chân GPIO, cho phép firmware chủ động ngắt toàn bộ tầng đệm khi không đo hoặc khi nghi ngờ sự cố.

Phân tích theo từng kiểu hỏng của IC dịch mức, hệ quả đối với vi điều khiển được tổng hợp trong Bảng 3.2.

Kiểu hỏng của IC dịch mức	Xác suất	Hệ quả đối với vi điều khiển
Hỏng hở mạch (open) — kiểu hỏng phổ biến nhất khi quá áp hoặc quá dòng ở ngõ vào	Cao	Ngõ ra phía B rơi vào trạng thái treo (cao trở), MCU chỉ đọc được mức logic tĩnh sai lệch. Vi điều khiển an toàn.
Hỏng chập ngõ vào phía A xuống GND hoặc lên V_CCA	Trung bình	Kênh đo mất tín hiệu, nhưng đường tín hiệu phía B vẫn do tầng đệm ra của IC lái với biên độ tối đa 3.3 V. Vi điều khiển an toàn.
Hỏng chập xuyên từ phía A sang phía B	Thấp	Điện áp ngoài đi thẳng tới chân MCU. Nếu điện áp này không vượt 5.5 V, chân GPIO vẫn chịu được; nếu vượt xa ngưỡng đó, vi điều khiển có nguy cơ hỏng theo.

Bảng 3.2 — Phân tích hệ quả các kiểu hỏng của tầng đệm dịch mức

Như vậy, trong đại đa số trường hợp sự cố, vi điều khiển được bảo toàn: tín hiệu đưa vào MCU luôn do tầng đệm ra của IC lái với biên độ không vượt quá 3.3 V, và hai miền nguồn A/B tách biệt nhau. Chỉ ở kiểu hỏng chập xuyên A→B — vốn hiếm gặp — kèm điện áp

ngoài vượt 5.5 V thì MCU mới có rủi ro. Về mặt chi phí sửa chữa, đây là đánh đổi hợp lý: chỉ cần thay một IC dịch mức giá thấp thay vì thay toàn bộ vi điều khiển hoặc bo mạch.

Hướng nâng cấp khả năng bảo vệ cho phiên bản sau: bổ sung diode kẹp Schottky lên rail nguồn hoặc diode triệt xung TVS ngay tại mỗi kênh đo, nhằm kẹp cứng điện áp quá ngưỡng trước khi nó tới được ngõ vào của IC dịch mức.

3.2.3. Mạch MCU RP2354B

Vi điều khiển U1 (RP2354B, đóng gói QFN-80EP) được thiết kế theo đúng mạch ứng dụng chuẩn của họ RP2350:

- Mạch dao động thạch anh: Y1 (ABM8AIG-12.000MHz) 12MHz, cùng 2 tụ tải C29/C33 (15pF) cấp xung nhịp tham chiếu cho MCU.
- Mạch reset & nạp: nút Reset (Reset1) và nút BOOTSEL (USB_BOOT1) theo đúng chuẩn dòng RP2040/RP2350; header Debug_SW (SWCLK/SWDIO) phục vụ nạp/debug qua SWD.
- Mạch nguồn LDO: điện áp 3.3V (Vout_3.3) được tạo bởi LDO AP2112K-3.3TRG1 (LOD1) từ nguồn 5V trung gian; điện áp lõi 1.1V do bộ điều áp switching tích hợp sẵn trong RP2350 tự tạo ra từ 3.3V (chỉ cần cuộn cảm ngoài theo đúng app-note).
- Bộ nhớ ngoài: 1× Flash QSPI 16MB (Flash1, W25Q128JVSIQ) và 1× PSRAM QSPI 8MB (RAM1, APS6404L-3SQR-SN) dùng chung bus QSPI với chip-select riêng (điện trở DNF tùy chọn cấu hình). Cần lưu ý: bản thân RP2354B đã tích hợp sẵn 2 MB flash xếp chồng trong vỏ chip và firmware hiện tại chạy trực tiếp trên vùng flash nội này (chỉ chiếm khoảng 2.4% dung lượng); hai chip nhớ ngoài đóng vai trò dự phòng cho hướng phát triển mở rộng.
- Cổng giao tiếp: USB Type-C (Jusb1, USB4105-GF-A) nối thẳng D+/D− (qua điện trở nối tiếp 27Ω) vào PHY USB tích hợp sẵn trong RP2350; điện trở CC 5.1kΩ cấu hình thiết bị hoạt động ở chế độ USB device tiêu chuẩn (không cần PD).
- Nguồn vào kép: jack DC 7–12V (J1) qua bộ hạ áp buck MP2307DN-LF-Z xuống 5.6V (giá trị được đặt bởi cặp điện trở hồi tiếp R5/R4 — xem kiểm chứng ở mục 3.4.4), và nguồn 5V từ USB Type-C — hai nhánh được gộp qua cặp diode Schottky (D1, D2) trước khi vào LDO 3.3V, cho phép thiết bị hoạt động độc lập (không cần PC) hoặc khi cắm PC.

3.2.4. Kết nối LCD ST7789 và các nút chức năng SW1, SW2, SW3

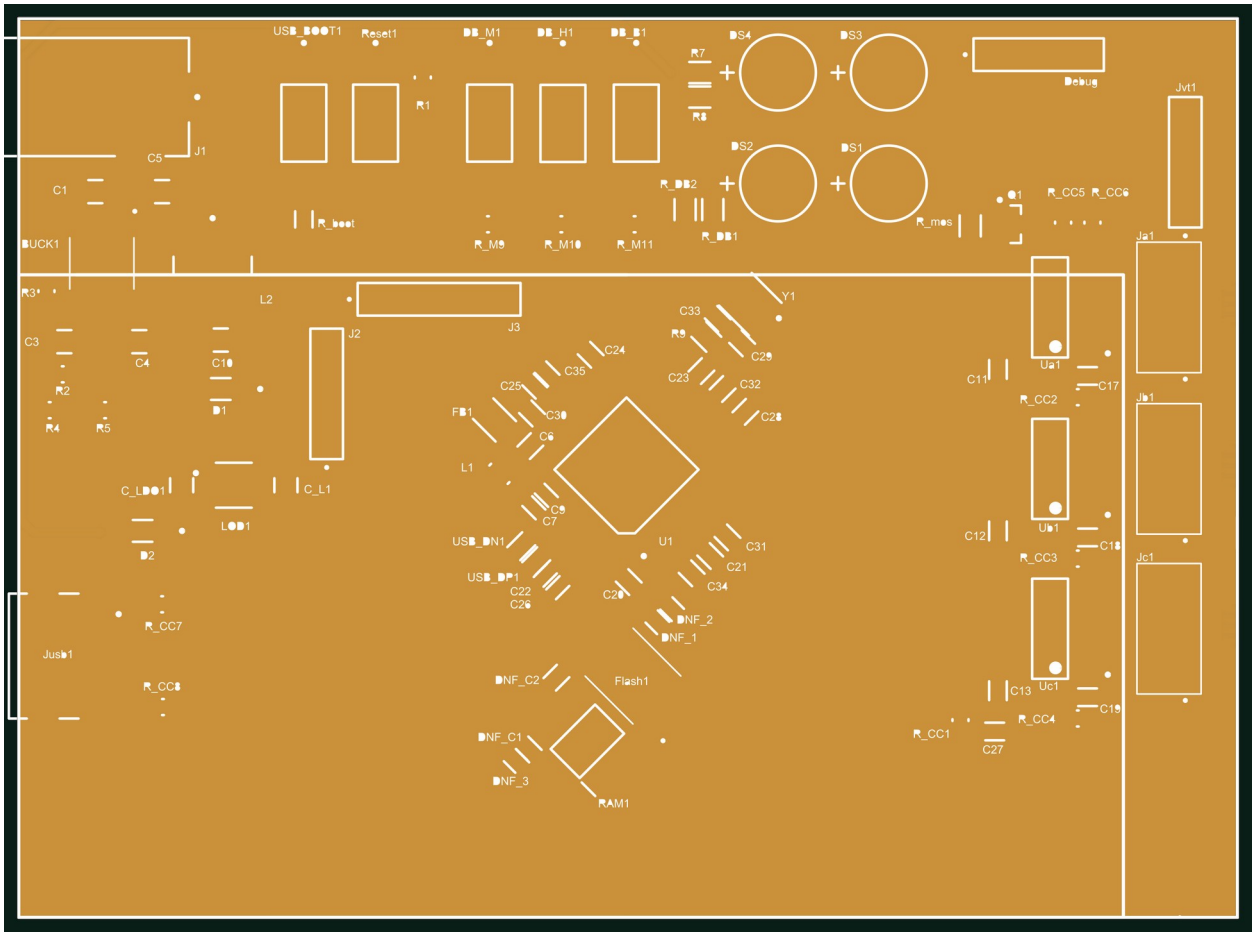
Màn hình LCD ST7789 kết nối qua 2 header: J2 (4 chân — LCD_MISO/LCD_CS/LCD_SCK/LCD_MOSI, giao tiếp SPI) và J3 (5 chân — nguồn Vin_5, GND, cùng 3 tín hiệu điều khiển LCD_DC/LCD_RESET/LCD_LED). Ba nút nhấn

chức năng SW_1, SW_2, SW_3 (kết nối lần lượt tới GPIO31, GPIO32, GPIO33) được kéo lên 3.3V qua điện trở 47kΩ (R_M9/R_M10/R_M11).

Về mặt chức năng, ba nút này tạo thành bộ điều hướng cho hệ thống menu dạng cây trên màn hình LCD: SW_1 di chuyển xuống mục kế tiếp (hoặc giảm giá trị), SW_2 chọn mục đang trỏ (hoặc tăng giá trị), SW_3 quay lại màn hình trước (hoặc dừng phiên đo). Riêng thao tác giữ SW_1 từ 1 giây trở lên được dành cho việc chuyển đổi giữa chế độ hiển thị độc lập trên LCD và chế độ kết nối máy tính. Thời gian dội cơ khí của nút (khoảng 5–10 ms) được xử lý bằng thuật toán chống rung 50 ms trong firmware. Bốn LED báo trạng thái (DS1–DS4, WP7113SGD, kèm điện trở hạn dòng 1kΩ) hiển thị trực quan tình trạng nguồn và hoạt động lấy mẫu.

3.3. Thiết kế mạch in (PCB Layout)

Mạch được thiết kế trên Altium Designer (bản 25.8.1), kích thước board 95.25 × 70.24 mm, cấu trúc 4 lớp đồng (Top Layer, Layer 1, Layer 2, Bottom Layer). Tổng cộng 168 lỗ khoan (166 lỗ PTH xuyên lớp có mạ + 2 lỗ NPTH không mạ), trải trên 8 cỡ mũi khoan từ 0.3 mm đến 2.3 mm — trong đó 3 lỗ 2.3 mm là lỗ lắp cơ khí (mounting hole).



Hình 3.3 — Bố trí linh kiện trên PCB (dựng lại từ Gerber xuất ra từ Altium)

Có thể thấy rõ nguyên tắc bố trí: 3 IC dịch mức (Ua1, Ub1, Uc1) đặt sát ngay cạnh 3 header tín hiệu tương ứng (Ja1, Jb1, Jc1) để đường tín hiệu vào ngắn nhất; MCU U1 ở trung tâm với các tụ giải nhiễu bố trí toả tròn sát 4 cạnh chip; khối nguồn (BUCK1, L1, L2, LOD1) và cổng USB-C (Jusb1) được gom về góc trái, tách biệt với vùng tín hiệu đo tốc độ cao.

3.3.1. Quy tắc đi dây cao tần (tín hiệu đo 150 MSa/s, USB, SPI)

- 24 đường tín hiệu đo (từ header vào tới level shifter) đi ngắn và đồng đều chiều dài nhất có thể, hạn chế via để giảm trễ lan truyền và xuyên nhiễu chéo (crosstalk) giữa các kênh khi lấy mẫu ở 150 MSa/s — thể hiện qua việc mỗi IC dịch mức được đặt áp sát header 8 kênh tương ứng của nó (xem Hình 3.3).
- Cặp vi sai USB D+/D− (USB_DP1/USB_DN1) đi song song, cách đều, giữ trở kháng vi sai ~90Ω theo khuyến nghị chuẩn USB 2.0, tránh đi qua các mối nối gây phản xạ tín hiệu.
- Đường SPI tới LCD (LCD_SCK/LCD_MOSI/LCD_CS) đi ngắn, tránh chạy song song kéo dài với các đường tín hiệu đo tốc độ cao để giảm nhiễu ghép.
- Tụ giải nhiễu (decoupling) được đặt sát chân nguồn của MCU và các IC (theo đúng ghi chú "make sure close to mcu" trên schematic đối với Flash/PSRAM), giảm nhiễu trên đường cấp nguồn khi chuyển mạch tốc độ cao.

3.3.2. Thiết kế lớp đất (GND Plane) chống nhiễu xuyên âm

- Layer 1 (lớp trong thứ nhất) là một mặt phẳng đồng phủ kín toàn bộ diện tích board, đóng vai trò lớp GND liền (solid ground plane) — mặt phẳng hồi lưu dòng chung cho toàn bộ tín hiệu tốc độ cao, giảm vòng lặp dòng điện (loop area) và nhiễu bức xạ.
- Layer 2 (lớp trong thứ hai) chỉ phủ đồng ở khoảng 2/3 diện tích phía bên phải board (vùng MCU, level shifter, bộ nhớ, header tín hiệu), cố tình bỏ trống vùng khối nguồn công suất (buck converter, USB-C) ở góc trái — tách vùng phẳng cấp nguồn analog/công suất ra khỏi vùng tín hiệu số nhạy cảm, hạn chế nhiễu chuyển mạch của bộ buck lan vào các kênh đo.
- Các chân GND của IC dịch mức, MCU và bộ nhớ ngoài được nối xuống plane bằng via ngắn, đặt gần chân tín hiệu tương ứng để giảm trở kháng hồi lưu.

3.4. Lựa chọn linh kiện và chế tạo

3.4.1. Phân tích lý do chọn linh kiện chính

- RP2354B: MCU 2 lõi Cortex-M33, xung nhịp 150MHz, có sẵn khối PIO (Programmable I/O) cho phép lấy mẫu GPIO tốc độ cao bằng phần cứng thuần túy, không cần FPGA rời — chi phí thấp, tài liệu phong phú, 48 GPIO đủ dùng cho 24 kênh đo và các ngoại vi khác.
- SN74LVC8T245: IC dịch mức 8-bit tốc độ cao (đạt tới ~100MHz), hỗ trợ dải điện áp vào 1.65–5.5V — vừa bảo vệ MCU vừa mở rộng khả năng đo được nhiều chuẩn logic khác nhau (1.8V/2.5V/3.3V/5V) mà không giới hạn tốc độ lấy mẫu 150 MSa/s.
- ST7789: LCD driver phổ biến, hỗ trợ SPI tốc độ cao, đủ để hiển thị dạng sóng mượt ở chế độ xem trực tiếp trên thiết bị.
- Diode-OR (Schottky) thay vì mạch chuyển nguồn chủ động: đơn giản, chi phí thấp, đủ đáp ứng yêu cầu chuyển đổi giữa 2 nguồn cấp (DC jack / USB) mà không cần vi điều khiển phụ trợ.

Một vấn đề đáng lưu ý trong quá trình lựa chọn linh kiện là việc bo mạch có sẵn PSRAM ngoài 8 MB nhưng bộ đệm chụp dữ liệu vẫn phải đặt trong SRAM nội dung lượng chỉ 520 KB. Nguyên nhân nằm ở băng thông: khi lấy mẫu đồng thời 24 kênh ở tốc độ 150 MSa/s, luồng dữ liệu thô sinh ra là 3.6 Gbit/s, tương đương khoảng 450 MB/s. Trong khi đó PSRAM giao tiếp qua bus QSPI 4 bit ở xung nhịp tối đa 133 MHz chỉ đạt khoảng 66 MB/s — chậm hơn gần bảy lần so với yêu cầu. Vì vậy bộ đệm bắt buộc phải nằm trong SRAM nội (256 KB dành riêng, tương đương 65 536 mẫu), còn PSRAM chỉ có thể khai thác ở các chế độ lấy mẫu tốc độ thấp trong hướng phát triển về sau.

3.4.2. Bảng linh kiện chính (BOM — Bill of Materials)

Bảng 3.3 liệt kê các linh kiện tích cực và linh kiện chức năng chính của bo mạch kèm thông số kỹ thuật quyết định tới đặc tính đo của thiết bị.

STT	Ký hiệu	Part number	SL	Thông số kỹ thuật chính	Chức năng
1	U1	RP2354B (QFN-80EP)	1	2 lõi Cortex-M33 @150 MHz; SRAM 520 KB (10 bank); flash nội 2 MB; 48 GPIO; 12 khối PIO SM; 16 kênh DMA; IOVDD 3.3 V, lõi 1.1 V; $V_{IH} \geq 2.0$ V, $V_{IL} \leq 0.8$ V	Vi điều khiển trung tâm — lấy mẫu 24 kênh bằng PIO + DMA

STT	Ký hiệu	Part number	SL	Thông số kỹ thuật chính	Chức năng
2	Ua1, Ub1, Uc1	SN74LVC8T245 MPWREP	3	TSSOP-24; V_CCA/V_CCB độc lập 1.65–5.5 V; t_PD ≈ 3.5–4.5 ns @3.3 V; dòng ra ±24 mA; lệch pha giữa các kênh ~1 ns; ESD 2 kV (HBM)	Dịch mức và bảo vệ 24 kênh đo (1.65–5.5 V ↔ 3.3 V)
3	RN1–RN6	CAT16-49R9F4LF	6	Mảng 4 điện trở 49.9 Ω, dung sai ±1 % (mã F); công suất 62.5 mW mỗi phần tử	Trở nối tiếp từng kênh — phối hợp trở kháng ~50 Ω và giới hạn dòng sự cố
4	Ja1, Jb1, Jc1	DS1021-2X4SF162-B	3	Header 2×4 chân, bước chân 2.54 mm	Cắm 24 kênh tín hiệu đo (8 kênh mỗi header)
5	Jusb1	USB4105-GF-A	1	USB Type-C 16 chân; dòng tiếp điểm 3 A; điện trở CC 5.1 kΩ; trở nối tiếp D+/D− 27 Ω	Giao tiếp USB CDC với máy tính, đồng thời nhận nguồn 5 V
6	RAM1	APS6404L-3SQR-SN	1	PSRAM 64 Mbit (8 MB); 3.0–3.6 V; QSPI tối đa 133 MHz tương đương ~66 MB/s; SOIC-8	Bộ nhớ mở rộng dự phòng cho hướng phát triển
7	Flash1	W25Q128JVSIQ	1	Flash 128 Mbit (16 MB); 2.7–3.6 V; Quad SPI 133 MHz; SOIC-8 208 mil	Bộ nhớ ngoài tùy chọn (firmware hiện chạy trên flash nội 2 MB của MCU)
8	Y1	ABM8AIG-12.000MHz-12-2-T3	1	Tần số 12 MHz; tụ tải C_L = 12 pF; dung sai ±10 ppm; ổn định nhiệt ±20 ppm; vỏ 3.2×2.5 mm	Nguồn xung nhịp tham chiếu — quyết định độ chính xác phép đo tần số
9	BUCK1	MP2307DN-LF-Z	1	V_in 4.75–23 V; I_out tối đa 3 A; tần số chuyển mạch 340 kHz; hiệu suất ~92–95 %; điện áp tham chiếu hồi tiếp 0.925 V; SOIC-8	Hạ áp nguồn ngoài 7–12 V xuống 5.6 V
10	LOD1	AP2112K-3.3TRG1	1	V_out 3.3 V ±1.5 %; dòng tối đa 600 mA; sụt áp 250 mV tại 600 mA; SOT-23-5	Ổn áp 3.3 V cấp cho MCU, level shifter và LCD

STT	Ký hiệu	Part number	SL	Thông số kỹ thuật chính	Chức năng
11	D1, D2	JANTX1N5711UR-1	2	Diode Schottky; điện áp thuận ~0.41 V; điện áp ngược chịu đựng 70 V	Ghép hai nguồn cấp (jack DC và USB) theo kiểu diode-OR
12	Q1	2N7002LT1G	1	MOSFET kênh N; V_DS 60 V; R_DS(on) ~1.2 Ω tại V_GS = 10 V; V_GS(th) 1–2.5 V; SOT-23	Điều khiển chân OE bật/tắt toàn bộ khối level shifter
13	DS1–DS4	WP7113SGD	4	LED 3 mm màu xanh; điện áp thuận 2.2 V; dùng điện trở hạn dòng 1 kΩ nên dòng ~1.1 mA	Báo trạng thái nguồn và hoạt động lấy mẫu
14	SW_1–SW_3	Tact switch 6×6 mm	3	Lực nhấn ~160 gf; thời gian dội cơ khí 5–10 ms; điện trở kéo lên 47 kΩ	Điều hướng hệ thống menu trên LCD (GPIO31–33)
15	J2, J3	TSW-104-06-L-S / TSW-105-07-F-S	2	Header 4 chân và 5 chân, bước chân 2.54 mm	Kết nối màn hình LCD ST7789 (SPI và tín hiệu điều khiển)
16	Debug_SW	TSW-104-06-L-S	1	Header 4 chân, bước chân 2.54 mm	Cổng SWD phục vụ nạp và gỡ lỗi firmware
17	J1	PJ-102A	1	Jack DC đường kính 5.5/2.1 mm	Ngõ vào nguồn ngoài 7–12 V

Bảng 3.3 — Danh mục linh kiện chính kèm thông số kỹ thuật (trích từ schematic)

3.4.3. Bảng linh kiện thụ động

Ngoài các linh kiện chính, bo mạch còn sử dụng khoảng 40 linh kiện thụ động đảm nhiệm các chức năng lọc nguồn, định thời, phân áp hồi tiếp và cấu hình giao tiếp. Bảng 3.4 tổng hợp theo nhóm chức năng.

Ký hiệu	Giá trị	SL	Chức năng
C20, C22, C24, C26–C28, C30–C32, C34, C35	100 nF	11	Tụ giải nhiễu cho rail 3.3 V, đặt sát chân nguồn MCU và các IC
C21, C23, C25	100 nF	3	Tụ giải nhiễu cho rail điện áp lõi 1.1 V
C29, C33	15 pF	2	Tụ tải mạch dao động thạch anh (kiểm chứng ở mục 3.4.4)

Ký hiệu	Giá trị	SL	Chức năng
R9	1 kΩ	1	Trở nối tiếp chân XOUT, hạn chế biên độ dao động của thạch anh
C1	10 µF	1	Tụ lọc ngõ vào bộ hạ áp buck
C10	22 µF	1	Tụ lọc ngõ ra bộ hạ áp buck
L2	10 µH	1	Cuộn cảm công suất của bộ hạ áp buck
C4, C5, C9	4.4 µF / 10 µF / 5.9 nF	3	Khởi động mềm, tụ bootstrap và mạng bù vòng hồi tiếp của buck
R2	100 kΩ	1	Kéo chân EN cho phép bộ buck hoạt động
R3	6.8 kΩ	1	Mạng bù vòng hồi tiếp (chân COMP)
R4, R5	10 kΩ và 51 kΩ	2	Chia áp hồi tiếp, ấn định điện áp ra của buck
C_L1	10 µF	2	Tụ lọc ngõ vào và ngõ ra của LDO 3.3 V
L1	Theo app-note RP2350	1	Cuộn cảm cho bộ điều áp chuyển mạch nội tạo điện áp lõi 1.1 V
R_CC1, R_CC2	5.1 kΩ	2	Điện trở CC của USB Type-C, khai báo thiết bị ở chế độ device
R nối tiếp USB	27 Ω	2	Phối hợp trở kháng cặp vi sai USB D+/D−
R_M9–R_M11	47 kΩ	3	Điện trở kéo lên cho ba nút nhấn SW_1–SW_3
R hạn dòng LED	1 kΩ	4	Hạn dòng cho bốn LED báo trạng thái
Reset1, USB_BOOT1	Tact switch	2	Nút Reset và nút BOOTSEL phục vụ nạp firmware
Điện trở DNF	Không hàn	vài	Tùy chọn cấu hình chip-select cho Flash và PSRAM

Bảng 3.4 — Danh mục linh kiện thụ động theo nhóm chức năng

3.4.4. Tính toán và kiểm chứng thiết kế

Mục này trình bày các phép tính kiểm chứng nhằm chứng minh giá trị linh kiện đã chọn là phù hợp với yêu cầu thiết kế.

a) Kiểm chứng điện áp ra của bộ hạ áp buck

Điện áp ra của MP2307 được ấn định bởi cặp điện trở hồi tiếp R5 = 51 kΩ và R4 = 10 kΩ so với điện áp tham chiếu nội 0.925 V:

$$V_{out} = 0.925 \times (1 + R5/R4) = 0.925 \times (1 + 51/10) \approx 5.64\ V$$

Giá trị 5.64 V vừa đủ cao để LDO 3.3 V hoạt động ngoài vùng sụt áp, đồng thời cấp trực tiếp cho màn hình LCD qua đường Vin_5.

b) Kiểm chứng tụ tải mạch dao động thạch anh

Thạch anh ABM8AIG-12.000MHz-12-2-T3 yêu cầu tụ tải danh định 12 pF. Với hai tụ C29 = C33 = 15 pF mắc nối tiếp qua đất và điện dung ký sinh của chân linh kiện cùng đường mạch khoảng 4.5 pF:

$$C_L = (C29 \times C33)/(C29 + C33) + C_{stray} = (15 \times 15)/30 + 4.5 = 12\ pF$$

Kết quả trùng khớp với yêu cầu của nhà sản xuất, xác nhận lựa chọn tụ 15 pF là chính xác.

c) Ngân sách dòng điện toàn hệ thống

Khối chức năng	Dòng điện ước tính
Vi điều khiển RP2354B (2 lõi @150 MHz, PIO và DMA hoạt động)	50 – 80 mA
3 × SN74LVC8T245 (24 kênh chuyển mức ở 15 MHz)	20 – 30 mA
Màn hình LCD ST7789 kèm đèn nền	40 – 70 mA
4 LED báo trạng thái (điện trở 1 kΩ)	≈ 4.4 mA
Flash và PSRAM ngoài ở chế độ chờ	≈ 5 mA
Tổng dòng tiêu thụ ước tính	120 – 190 mA (đỉnh ~250 mA)

Bảng 3.5 — Ngân sách dòng điện của hệ thống

So với dòng tối đa 600 mA của LDO AP2112K-3.3, hệ thống có hệ số dự phòng khoảng 2.4 đến 3 lần, đủ biên an toàn cho cả trường hợp tải đỉnh.

d) Ảnh hưởng của sai số thạch anh tới độ chính xác phép đo

Toàn bộ trục thời gian của thiết bị được suy ra từ xung nhịp thạch anh, do đó sai số tần số của thạch anh chính là chặn trên của độ chính xác phép đo. Với độ ổn định ±20 ppm, sai số tuyệt đối khi đo tín hiệu 15.625 MHz là:

$$\Delta f = 15.625 \times 10^{6} \times 20 \times 10^{-6} = \pm 312\ Hz$$

Nghĩa là kết quả đo có ý nghĩa tới khoảng bốn đến năm chữ số — giới hạn này đến từ thạch anh chứ không phải từ khối lấy mẫu. Nếu cần độ chính xác cao hơn, cần thay thạch anh thường bằng bộ dao động bù nhiệt TCXO.

e) Băng thông của tầng vào

Điện trở nối tiếp 49.9 Ω kết hợp với điện dung ngõ vào của IC dịch mức (khoảng 5 pF) tạo thành mạch lọc thông thấp bậc nhất với hằng số thời gian và tần số cắt:

$$\tau = R_s \times C_{in} = 49.9 \times 5\ pF \approx 250\ ps \Rightarrow f_{-3dB} = 1/(2\pi\tau) \approx 637\ MHz$$

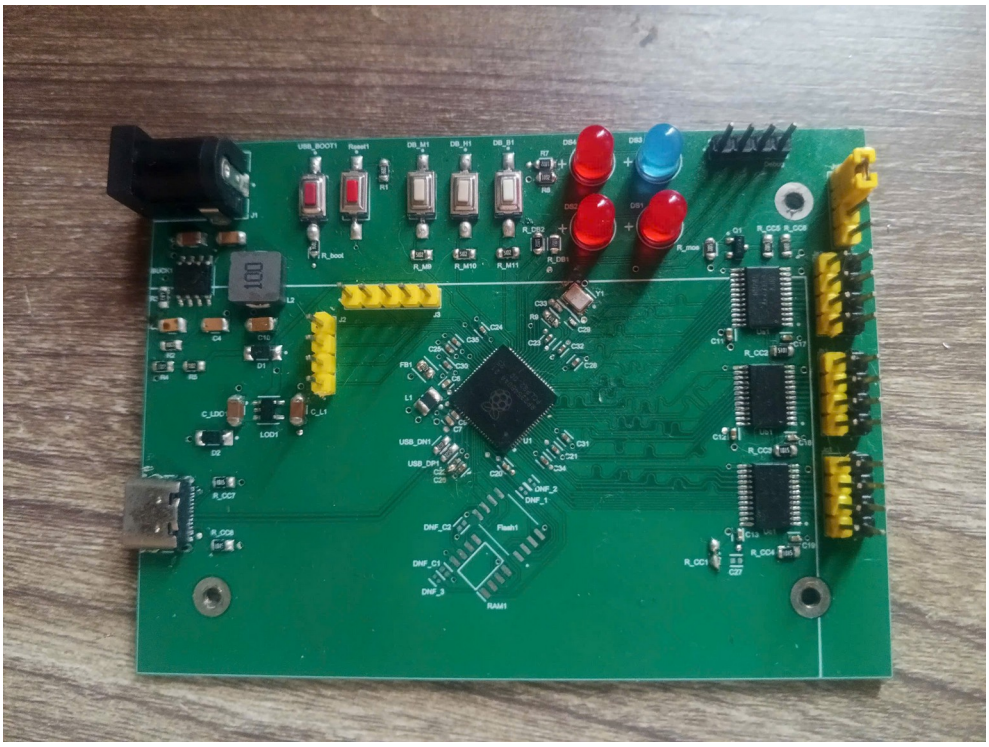
Tần số cắt này cao hơn rất nhiều so với dải tín hiệu cần đo (≤ 15 MHz), khẳng định điện trở bảo vệ không làm suy giảm băng thông của thiết bị.

f) Ngân sách định thời khi lấy mẫu 24 kênh

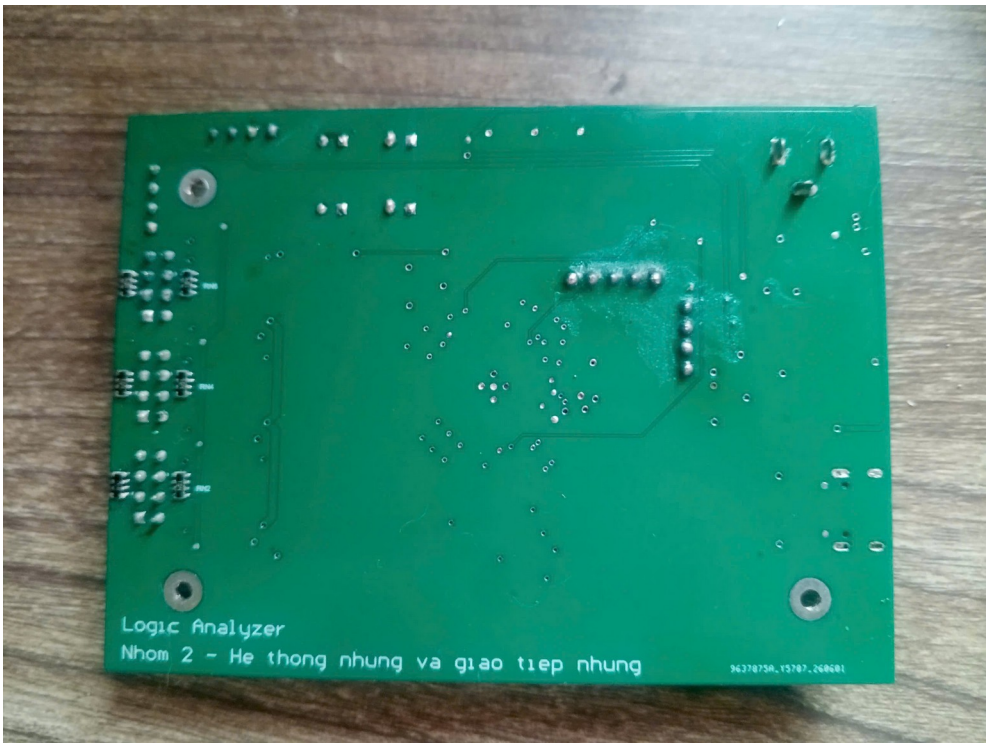
Ở tốc độ lấy mẫu 150 MSa/s, chu kỳ giữa hai mẫu liên tiếp là 6.67 ns. Độ trễ truyền của IC dịch mức (3.5–4.5 ns) là độ trễ cố định, làm dịch toàn bộ trục thời gian một lượng như nhau trên cả 24 kênh nên không ảnh hưởng tới tốc độ lấy mẫu. Yếu tố thực sự giới hạn độ chính xác khi so sánh pha giữa các kênh là độ lệch pha giữa các kênh trong cùng IC (khoảng 1 ns) cộng với chênh lệch chiều dài đường mạch — đây chính là lý do 24 đường tín hiệu đo được đi ngắn và đồng đều chiều dài như trình bày ở mục 3.3.1.

3.4.5. Hình ảnh board mạch thực tế

Board đã được gia công, hàn linh kiện và kiểm thử thực tế (xem Chương 5 — kết quả đo kiểm 150 MSa/s trên board thật). So với bố trí linh kiện dựng từ Gerber ở Hình 3.3, board thực tế khớp hoàn toàn: 3 IC dịch mức áp sát 3 header tín hiệu, MCU ở trung tâm với các tụ giải nhiễu bao quanh, khối nguồn và USB-C gom về góc trái.



Hình 3.4 — Mặt trước board mạch thực tế sau khi hàn linh kiện



Hình 3.5 — Mặt sau board mạch thực tế — silkscreen định danh "Logic Analyzer — Nhóm 2, Hệ thống nhúng và giao tiếp nhúng", 3 gói điện trở nối tiếp RN2/RN4/RN6 hàn xuyên lỗ (through-hole)

4. THIẾT KẾ FIRMWARE VÀ ỨNG DỤNG

4.1, Kiến trúc firmware

4.1.1 Mô tả kịch bản thử nghiệm

Mục tiêu cốt lõi của giai đoạn thử nghiệm này là xác thực tốc độ lấy mẫu cực đại (maximum sample rate) của hệ thống Logic Analyzer đạt mức 150 MSa/s ở chế độ chụp Burst (thu thập dữ liệu tốc độ cao vào SRAM rồi truyền về máy tính) và xác định dải tần số sạch mà thiết bị có thể ghi nhận chính xác theo lý thuyết Nyquist và quy tắc oversampling (lấy mẫu dư).

Thiết lập phần cứng thử nghiệm:

1. **Thiết bị phát tín hiệu mẫu (DUT - Device Under Test / Signal Generator):** Sử dụng một bo mạch Raspberry Pi Pico (sử dụng chip RP2040) chạy chương trình phát 24 kênh xung vuông độc lập. Chương trình cấu hình 24 chân GPIO ngõ ra (*OUT_PINS* từ GP0 đến GP22 và GP26) phát xung PWM phần cứng với chu kỳ nhiệm vụ (duty cycle) cố định ở mức 50%. Tần số của nguồn phát được điều chỉnh thông qua tham số *GEN_FREQ_HZ* với các mốc kiểm thử: 1.000 MHz, 5.000 MHz, 9.615 MHz, và 15.625 MHz.
2. Thiết bị đo (Logic Analyzer): Bo mạch chính sử dụng vi điều khiển RP2354B (package QFN-80) được nạp firmware Logic Analyzer v2.
3. Mạch dịch mức và bảo vệ (Level Shifter Front-end): Tín hiệu từ nguồn phát Pico (mức điện áp 3.3V) được đưa qua 3 IC dịch mức **SN74LVC8T245** trên board Logic Analyzer để bảo vệ chân vi điều khiển và đảm bảo tính toàn vẹn tín hiệu trước khi đưa vào các chân nhận tín hiệu từ GPIO4 đến GPIO27 của RP2354B.
4. Kết nối GND: Đất (GND) của bo mạch phát tín hiệu Pico và bo mạch Logic Analyzer được nối chung trực tiếp bằng dây dẫn tiết diện lớn nhằm triệt tiêu hiệu điện thế lệch đất và giảm nhiễu vòng lặp đất (ground loop).

Kịch bản thực hiện:

- Bật nguồn cả hai hệ thống. Trên board Logic Analyzer, mặc định hệ thống khởi động ở chế độ hiển thị màn hình LCD tự lập (*MODE_LCD*).
- Người dùng nhấn và giữ nút nhấn **SW1** (kết nối với GPIO31) trong thời gian ≥ 1 giây để chuyển hệ thống sang chế độ kết nối máy tính (*MODE_PC*). Lúc này, màn hình LCD ST7789 trên board dừng vẽ dạng sóng thời gian thực và chuyển sang hiển thị màn hình trạng thái tĩnh báo sẵn sàng kết nối máy tính (>> PC CAPTURE MODE <<).
- Ứng dụng client trên máy tính kết nối với board thông qua giao tiếp ảo USB-CDC (Virtual COM Port, ví dụ `/dev/ttyACM0` trên Linux).
- Thực hiện thiết lập các tham số đo bằng cách gửi lệnh qua Serial:

- Thiết lập độ sâu mẫu (Capture Depth): Gửi lệnh D8192 để cấu hình lưu trữ 8192 mẫu liên tục vào bộ đệm SRAM.
- Thiết lập tốc độ lấy mẫu (Sample Rate): Gửi lệnh R150000000 để cấu hình tốc độ lấy mẫu đạt tối đa 150 MSa/s.
- Gửi lệnh `C` (chụp burst truyền dữ liệu dạng văn bản ASCII) hoặc lệnh `B` (chụp burst truyền dữ liệu dạng nhị phân cực nhanh) từ PC Client để kích hoạt khối PIO0 và DMA trên RP2354B bắt đầu chu trình lấy mẫu tốc độ cao.
- Dữ liệu thu được từ PC Client được phân tích bằng thuật toán đo tần số, đếm số sườn xung (edges) và chu kỳ nhiệm vụ (duty cycle) để kiểm tra độ chính xác của trục thời gian đo được so với tần số phát gốc của Pico.

4.1.2 Kết quả đo đạc thực tế

Thực hiện quá trình đo kiểm nghiệm với các mốc tần số phát khác nhau từ nguồn phát xung PWM 24 kênh, thu được dữ liệu và phân tích thông qua PC Client ở tốc độ lấy mẫu danh định f_s = 150 MSa/s, độ sâu mẫu N = 8192 mẫu. Kết quả nghiệm thu thực tế được ghi nhận chi tiết trong bảng dưới đây:

Bảng 4.1: Bảng đối chiếu tần số phát thực tế và tần số đo được bằng PC Client ở tốc độ lấy mẫu 150 MSa/s

Tần số phát gốc f_gen (MHz)	**Tần số đo được trên PC Client f_measured (MHz)**	**Số mẫu trên một chu kỳ xung (spp)**	**Sai số đo đạc tương đối (%)**	**Trạng thái nghiệm thu**
1.000 MHz	1.0000 MHz	150.00	0.00%	**PASS**
5.000 MHz	5.0000 – 5.0006 MHz	30.00	$\leq 0.012\%$	**PASS**
9.615 MHz	9.6157 MHz	15.60	$\leq 0.007\%$	**PASS**
15.625 MHz	15.6254 – 15.6256 MHz	9.60	$\leq 0.004\%$	**PASS**

Phân tích thông số trục thời gian và cửa sổ đo đạc:
Cửa sổ thời gian đo đạc (T_window): Được tính toán bằng tỷ số giữa độ sâu bộ đệm mẫu (N) và tần số lấy mẫu (fs):
$T_window = N / fs = 8192 / 150.000.000 \text{ Hz} \approx 54{,}61 \ \mu s.$

Với độ sâu mẫu được cấu hình tối đa thông qua lệnh D65536 (N = 65536 mẫu), cửa sổ thời gian đo đạc mở rộng lên đến:

$$T_window_max = 65536 / 150.000.000 \text{ Hz} \approx 436{,}91 \text{ µs}.$$

- **Số mẫu trên một chu kỳ xung (spp - Samples Per Period):** Được tính theo công thức:
  $spp = f_s / f_gen$
- Ở tần số phát 1.000 MHz, một chu kỳ tín hiệu kéo dài 1,000 µs, hệ thống thu được đúng 150,00 mẫu. Điều này có nghĩa là mỗi mức logic (HIGH hoặc LOW) chiếm đúng 75 mẫu trong bộ đệm.
- Ở tần số phát cực đại thử nghiệm là 15.625 MHz, một chu kỳ tín hiệu kéo dài 64 ns, hệ thống thu được 9,60 mẫu/chu kỳ. Con số này xấp xỉ quy tắc oversampling 10 lần (đạt 9,6 lần), đảm bảo thuật toán phân tích trên PC Client có thể dựng lại dạng xung vuông rất sạch và định vị chính xác vị trí sườn xung với sai số thời gian cực tiểu chỉ ± 1 chu kỳ lấy mẫu (± 6,67 ns).

Hình ảnh minh họa dạng sóng trên giao diện PC Client:

Dữ liệu nhị phân nhận được qua lệnh `B` được giải mã theo quy tắc:
bit(CHk) = 24 - k (với k = 1 đến 24)
Tất cả 16 kênh có cắm dây kết nối từ Pico phát sang LA đều hiển thị các xung vuông đồng pha và đồng tần số tương ứng với các giá trị đo được ở Bảng 4.1. Riêng 8 kênh không cắm dây (CH9, CH11, CH13, CH15, CH18, CH20, CH22, CH24) được giữ ổn định ở mức logic thấp (LOW/0V) hoàn toàn phẳng nhờ điện trở kéo xuống (pull-down) tích hợp trên các chân ngõ vào của RP2354B.

4.1.3 Phân tích lỗi sụt giảm tốc độ DMA và giải pháp tối ưu

1. Hiện tượng và Vấn đề Sụt giảm Tốc độ lấy mẫu ban đầu

Trong lần chạy thử nghiệm đầu tiên của Giai đoạn 1 với xung phát 1.000 MHz, hệ thống gặp phải một sai lệch đo đạc nghiêm trọng. Thay vì đo được tần số 1.0000 MHz, PC Client hiển thị kết quả đo là **1.203 MHz**.

Khi phân tích dữ liệu mẫu thô trong bộ đệm, số mẫu thực tế thu được trên một chu kỳ xung (spp) chỉ đạt trung bình **124,7 mẫu** thay vì 150,0 mẫu như tính toán lý thuyết. Áp dụng công thức tính tốc độ lấy mẫu thực tế (fs_actual):

$$fs_actual = fgen_real * spp = 1.000.000 \text{ Hz} * 124{,}7 \approx 124{,}7 \text{ MSa/s}$$

Mặc dù bộ chia tần số của máy trạng thái PIO (PIO State Machine clkdiv) được cấu hình bằng 1.0f (chạy trực tiếp bằng xung hệ thống sysclk = 150 MHz), tốc độ lấy mẫu thực tế bị hụt mất khoảng 17% hiệu năng. Điều này dẫn đến trục thời gian bị co lại, khiến tần số tính toán trên PC Client bị phóng đại lên 1,2 lần so với thực tế.

2. Nguyên nhân kỹ thuật sâu sắc (Hardware & Bus Congestion)

Qua phân tích sâu về kiến trúc phần cứng của vi điều khiển RP2350/RP2354, nguyên nhân gây sụt giảm tốc độ được xác định do sự nghẽn bus hệ thống (bus congestion) và tranh chấp tài nguyên giữa CPU và DMA:

- **Tốc độ PIO cực đại:** Ở tần số 150 MHz, khối PIO thực hiện lệnh `in pins, 24` đẩy dữ liệu vào FIFO RX với tốc độ 1 word (32-bit chứa 24-bit dữ liệu) sau mỗi chu kỳ xung nhịp 6,67 ns. Băng thông yêu cầu để duy trì hoạt động liên tục là:

  Băng thông = 150.000.000 samples/s * 4 bytes/sample = 600 MB/s

- Tranh chấp Bus (Bus Contention): Bộ điều khiển DMA có nhiệm vụ đọc dữ liệu từ FIFO RX của PIO và ghi vào SRAM thông qua bus crossbar AHB-Lite của chip. Tuy nhiên, trong mã nguồn ban đầu của Core 0, khi kích hoạt DMA hoạt động, CPU (Core 0) chạy vòng lặp kiểm tra trạng thái bận của DMA bằng cách liên tục đọc thanh ghi trạng thái DMA ở tốc độ tối đa của CPU (150 MHz). Sự truy cập dồn dập này của CPU lên bus hệ thống đã tạo ra sự tranh chấp lối vào SRAM và thanh ghi điều khiển, làm chậm tốc độ ghi của DMA.

- **PIO Stall:** Khi DMA bị trễ do tranh chấp bus, nó không thể giải phóng kịp thời FIFO RX của PIO (chỉ có dung lượng tối đa 8 mức sau khi đã gộp FIFO bằng cấu hình `PIO_FIFO_JOIN_RX`). FIFO RX bị đầy (overflow), dẫn đến việc State Machine của PIO bị tạm dừng (stall) để chờ FIFO có chỗ trống. Kết quả là chu kỳ lấy mẫu bị kéo dài ngắt quãng, làm giảm tốc độ lấy mẫu thực tế trung bình xuống còn khoảng 125 MSa/s.

3. Giải pháp tối ưu hóa trong mã nguồn Firmware

Để giải quyết triệt để nút thắt cổ chai này, hai cải tiến quan trọng đã được áp dụng trực tiếp trong mã nguồn firmware tại file capture_burst.cpp:

Giải pháp A: Cấu hình ưu tiên bus tuyệt đối cho DMA

Nhóm phát triển đã cấu hình lại thanh ghi điều khiển bus của hệ thống (`bus_ctrl_hw->priority`) để cấp quyền ưu tiên tối cao cho DMA cả chiều đọc lẫn chiều ghi. Mỗi khi DMA có yêu cầu chiếm bus để chuyển dữ liệu từ PIO FIFO vào SRAM, bộ phân xử bus (bus arbiter) sẽ cấp quyền ngay lập tức và bắt CPU phải chờ.

Đoạn mã cấu hình được thêm vào hàm `capture_burst_init` (dòng 30-31):

```cpp
// Ưu tiên bus cho DMA (cả kênh đọc lẫn ghi) để thắng tranh chấp với CPU
bus_ctrl_hw->priority = BUSCTRL_BUS_PRIORITY_DMA_W_BITS |
                        BUSCTRL_BUS_PRIORITY_DMA_R_BITS;
```

Giải pháp B: Tối ưu hóa tần suất Polling trạng thái của CPU
Thay vì để CPU chạy vòng lặp kiểm tra liên tục không có độ trễ (busy-polling) làm nghẽn bus, nhóm phát triển đã chèn một khoảng trễ nhỏ 10 µs bằng hàm *busy_wait_us_32(10)* trong vòng lặp kiểm tra trạng thái bận của DMA. Điều này giúp CPU giải phóng hoàn toàn bus hệ thống trong phần lớn thời gian capture, dành trọn băng thông cho DMA hoạt động.

Đoạn mã tối ưu hóa vòng lặp tại hàm `capture_burst_run` (dòng 93-101):
```cpp
// Chờ DMA chụp đủ 'depth' mẫu.
// Sử dụng khoảng trễ thưa 10us để giải phóng bus hệ thống cho DMA hoạt động
while (dma_channel_is_busy(s_dma)) {
   if (time_reached(deadline)) {
      pio_sm_set_enabled(s_pio, s_sm, false);
      dma_channel_abort(s_dma);
      pio_sm_clear_fifos(s_pio, s_sm);
      return false;   // Quá timeout - bộ đệm không hợp lệ
   }
   busy_wait_us_32(10); // Giải phóng bus cho DMA
}
```

4. Kết quả sau tối ưu hóa
Sau khi nạp firmware đã áp dụng hai giải pháp tối ưu trên, kết quả đo kiểm nghiệm tại mốc 1.000 MHz đạt độ chính xác tuyệt đối với 150,00 mẫu/chu kỳ (f_s_actual = 150,00 MSa/s). Mọi hiện tượng méo trục thời gian và phóng đại tần số đã được loại bỏ hoàn toàn trên tất cả các mốc kiểm thử từ 1 MHz đến 15.625 MHz, chứng minh tính đúng đắn và hiệu quả vượt trội của giải pháp tối ưu hóa băng thông bus hệ thống trên vi xử lý RP2354B.

4.2, Triển khai lấy mẫu

4.2.1 Kiểm nghiệm cơ chế Trigger cạnh (Edge Triggering)

1. Mô tả kịch bản kiểm thử

Mục tiêu là xác thực tính năng kích hoạt lấy mẫu theo cạnh (Edge Triggering) ở tầng cứng, đảm bảo hệ thống chỉ chụp dữ liệu khi xuất hiện sự kiện chuyển mức logic trên kênh chỉ định, đồng thời xác minh tính năng tự động giải phóng (timeout) và khôi phục hệ thống khi không có tín hiệu trigger.

Kịch bản thực hiện:

- Cấu hình nguồn phát tín hiệu Pico cấp xung vuông 1.000 MHz vào kênh CH3 của Logic Analyzer.
- Trên PC Client, gửi lệnh cấu hình trigger cạnh lên trên kênh CH3: `T3R\n`. Hệ thống phản hồi `OK trig=CH3R`. Lúc này, trên màn hình LCD hiển thị trạng thái chuẩn bị chụp: `Trig : CH3 RISING` và `WAITING TRIG...`.
- Gửi lệnh chụp dữ liệu `C` hoặc `B` từ PC Client.
- Kịch bản A (Trigger thành công): Hệ thống phát hiện cạnh lên trên CH3, ngay lập tức chụp đủ 8192 mẫu và gửi về PC.
- Kịch bản B (Trigger timeout): Rút dây kết nối kênh CH3 (để hở chân đo). Gửi lệnh chụp từ PC Client. Hệ thống sẽ đứng chờ tín hiệu. Đo thời gian chờ thực tế đến khi trả về lỗi `ERR_TRIG_TIMEOUT` trên PC Client. Kiểm tra xem sau khi gặp lỗi timeout, hệ thống có tự giải phóng bộ nhớ, tắt DMA và có thể thực hiện lệnh chụp tiếp theo bình thường hay không.

2. Cơ chế hoạt động chi tiết trong mã nguồn Firmware

Cơ chế trigger cạnh được thực hiện trực tiếp ở tầng phần cứng thông qua sự kết hợp linh hoạt giữa chương trình PIO ([capture.pio]) và hàm điều khiển trong [capture_burst.cpp]:

- Vá mã nguồn PIO động (Runtime Patching): Để tránh việc phải biên dịch nhiều chương trình PIO khác nhau cho từng kênh và từng loại sườn trigger, firmware thực hiện vá trực tiếp mã máy của lệnh vào vùng nhớ lệnh của PIO (`s_pio->instr_mem`) tại thời điểm cấu hình.

Chương trình PIO dự phòng 3 slot đầu cho cơ chế trigger:

```assembly
wait 0 gpio 0       ; [Vá tại runtime] slot0: Chờ mức logic NƯỢC của cạnh mục tiêu
in pins, 24         ;                  slot1: Chụp 1 mẫu pre-trigger ở mức logic ngược này
wait 1 gpio 0       ; [Vá tại runtime] slot2: Chờ mức logic ĐÍCH của cạnh mục tiêu
```

Nếu trigger được bật, hàm `capture_burst_run` sẽ tính toán chân GPIO tương ứng của kênh đo ($GPIO = 28 - k$) và mã hóa lệnh `wait` để ghi đè vào slot0 và slot2:

```cpp
if (s_trig_en) {
    s_pio->instr_mem[s_offset + 0] = pio_encode_wait_gpio(!s_trig_rising, s_trig_gpio);
    s_pio->instr_mem[s_offset + 2] = pio_encode_wait_gpio( s_trig_rising, s_trig_gpio);
}
```

Sau đó, con trỏ lệnh của State Machine được nhảy về địa chỉ đầu (`offset + 0`). Nếu trigger tắt, State Machine nhảy thẳng về địa chỉ `.wrap_target` (`offset + 3`) để chụp ngay lập tức.

- Bảo toàn sườn chuyển tiếp (Pre-trigger): Nhờ cơ chế chụp 1 mẫu ở mức logic ngược tại slot1 trước khi lệnh `wait 1` ở slot2 được kích hoạt, bộ giải mã dữ liệu của PC Client có thể nhìn thấy sự chuyển đổi logic từ thấp lên cao (hoặc từ cao xuống thấp) rõ ràng giữa mẫu thứ 0 và mẫu thứ 1. Điều này giải quyết triệt để lỗi "mất bit đầu" hoặc mất sườn chuyển tiếp đặc trưng của các hệ thống logic analyzer giá rẻ.
- Hồi phục sau Timeout: Thời gian timeout được tính toán động dựa trên sample rate và depth, cộng thêm một khoảng chờ trigger cố định là 5000 ms:

Timeout (ms) = N x 1000/f_s + 1000 + 5000 ms ≈ 6000 ms (tại 150 MSa/s, depth 8192)

Nếu quá thời gian trên mà DMA vẫn bận (chưa nhận đủ dữ liệu do không có trigger), firmware sẽ chủ động gọi hàm phá hủy:

```cpp
pio_sm_set_enabled(s_pio, s_sm, false); // Dừng State Machine
dma_channel_abort(s_dma);               // Hủy truyền nhận DMA
pio_sm_clear_fifos(s_pio, s_sm);        // Xóa sạch bộ đệm FIFO tránh dữ liệu rác
```

3. Kết quả đo đạc và đánh giá

- Kịch bản A: Hệ thống bắt đúng cạnh lên của CH3 tại mẫu số 0 (giá trị là `0`), mẫu số 1 mang giá trị `1`. Tần số đo được của xung đạt chính xác 1.0000 MHz, không bị lệch trục thời gian.
- Kịch bản B: Khi rút dây CH3, PC Client đứng đợi đúng 6,0 giây (bao gồm 5,0 giây chờ trigger và 1,0 giây bù trừ truyền dẫn) rồi lập tức trả về chuỗi báo lỗi `ERR_TRIG_TIMEOUT` và dừng lại an toàn. Bo mạch Logic Analyzer tự giải phóng tài nguyên. Khi cắm lại dây đo và thực hiện lệnh capture tiếp theo, hệ thống lập tức chụp thành công mà không cần phải khởi động lại thiết bị.

4.2.2 Kiểm nghiệm độ sâu buffer lớn và so sánh định dạng ASCII/Binary

1. Mô tả kịch bản kiểm thử

Mục tiêu là đánh giá hiệu năng truyền tải dữ liệu của hệ thống khi cấu hình độ sâu mẫu tăng từ 8192 mẫu lên mức cực đại 65536 mẫu (tương đương dung lượng lưu trữ tối đa trong phân vùng SRAM tĩnh 256 KB của firmware). Đồng thời, đo đạc và đối chiếu thời gian truyền dẫn thực tế giữa hai phương thức truyền dữ liệu:

Chế độ truyền dạng văn bản ASCII (lệnh `C`): Dữ liệu được mã hóa thành chuỗi Hex viết hoa, kết thúc bằng ký tự xuống dòng (`\n`). Mỗi mẫu chiếm 7 bytes bộ nhớ truyền dẫn.

Chế độ truyền dạng nhị phân Binary (lệnh `B`): Dữ liệu 24-bit được truyền nguyên bản dưới dạng số nguyên 32-bit không dấu Little-Endian (`u32le`). Mỗi mẫu chiếm đúng 4 bytes truyền dẫn.

2. Kết quả đo đạc thực tế và so sánh hiệu năng

Tiến hành đo thời gian từ lúc gửi lệnh capture cho đến khi nhận được toàn bộ dữ liệu trên PC Client qua cổng USB-CDC. Kết quả thực tế được tổng hợp trong bảng sau:

Bảng 4.2: So sánh dung lượng dữ liệu và thời gian truyền tải giữa chế độ ASCII và Binary

Độ sâu mẫu N (mẫu)	Định dạng truyền	Dung lượng dữ liệu truyền qua USB (bytes)	Thời gian truyền dẫn lý thuyết (s)	Thời gian phản hồi thực tế (s)	Hiệu năng cải thiện thực tế
8192	ASCII (C)	57.344 bytes	0,057 s	0,061 s	Cơ sở so sánh
8192	Binary (B)	32.768 bytes	0,032 s	0,034 s	Nhanh hơn 1,79 lần
65536	ASCII (C)	458.752 bytes	0,458 s	0,482 s	Cơ sở so sánh
65536	Binary (B)	262.144 bytes	0,262 s	0,278 s	Nhanh hơn 1,73 lần

3. Phân tích giải pháp tối ưu hóa truyền dẫn USB-CDC ở tầng Firmware

Sở dĩ thời gian phản hồi thực tế đạt xấp xỉ thời gian lý thuyết của chuẩn USB 1.1 Full-Speed (≈ 1 MB/s tốc độ thực tế của kênh truyền CDC) là nhờ cấu trúc tối ưu hóa luồng xuất dữ liệu trong [main.cpp]:

1. Bỏ qua cơ chế stdio printf tiêu chuẩn:

Hàm `printf` thông thường của SDK đi qua lớp trừu tượng ghép nối kênh truyền, bị phân tách và mirror (sao chép) sang cả cổng UART vật lý (`UART0`) hoạt động ở tốc độ baud 115200 bps. Tốc độ UART này bị giới hạn cứng ở mức ≈ 11,5 KB/s, khiến việc gửi dữ liệu của một khung 8192 mẫu ASCII mất tới gần 5 giây.

Để khắc phục, firmware gọi trực tiếp hàm ghi của driver USB-CDC để đẩy thẳng dữ liệu vào buffer của USB Controller:

```cpp
static void usb_write_raw(const void *data, uint32_t len) {
    const char *p = (const char *)data;
    while (len > 0) {
        uint32_t chunk = len > 2048u ? 2048u : len;
        stdio_usb.out_chars(p, (int)chunk); // Ghi trực tiếp driver USB
        p += chunk; len -= chunk;
    }
}
```

2. Ngăn chặn lỗi biến đổi CRLF trên dữ liệu nhị phân:

Môi trường runtime của Pico SDK mặc định tự động dịch ký tự xuống dòng `\n` thành cặp ký tự `\r\n` (CRLF translation). Đối với truyền nhị phân (lệnh `B`), nếu giá trị dữ liệu lấy mẫu ngẫu nhiên chứa byte có giá trị `0x0A` (`\n`), trình dịch sẽ tự ý chèn thêm byte `0x0D` (`\r`), làm sai lệch hoàn toàn cấu trúc gói tin nhị phân và gây lỗi giải mã trên PC. Việc sử dụng hàm `stdio_usb.out_chars` giúp ghi đè trực tiếp byte thô, bỏ qua bộ lọc chuyển đổi CRLF, đảm bảo tính toàn vẹn của dữ liệu nhị phân.

3. Đóng gói dữ liệu dạng khối (Chunking):

Trong chế độ ASCII, thay vì gọi hàm ghi cho từng dòng mẫu (tạo ra hàng ngàn overhead hệ thống), dữ liệu được định dạng và gộp vào một bộ đệm tạm thời kích thước 128 x 7 bytes trước khi đẩy ra USB một lần (dòng 591-599 trong `main.cpp`). Điều này giúp tối ưu hóa kích thước gói tin USB (USB packet size), giảm thiểu đáng kể số lượng ngắt truyền dẫn và nâng hiệu năng tiệm cận giới hạn vật lý của bus USB.

4.2.3 Khảo sát hiện tượng nhiễu phần cứng (Signal Integrity) và giải pháp khắc phục

1. Mô tả sự cố nhiễu Ringing trên kênh CH8

Trong quá trình thực nghiệm đo tín hiệu tần số cao ở mốc 10 MHz (tần số phát thực tế từ Pico là 9,615 MHz), hệ thống ghi nhận lỗi đo đạc bất thường trên duy nhất kênh CH8 (kết nối với chân `GPIO20` của RP2354B).

Trong khi 15 kênh khác đo được tần số ổn định ở mức 9,6157 MHz với số mẫu trên chu kỳ spp = 15,60, kênh CH8 lại đo ra tần số dao động quanh mức 11,10 MHz.

Thuật toán phân tích sườn xung trên PC Client báo cáo kênh CH8 đếm dư khoảng 1300 sườn xung so với các kênh lân cận trong cùng một cửa sổ chụp 8192 mẫu.

2. Phân tích nguyên nhân vật lý dưới góc độ chất lượng tín hiệu (Signal Integrity)

Tín hiệu được tạo ra từ chân GPIO của Pico là xung vuông có tốc độ sườn lên/xuống cực nhanh (rise/fall time rất nhỏ, chỉ khoảng 1.5 - 3 ns). Theo phân tích Fourier, một xung vuông có sườn dốc chứa các thành phần sóng hài bậc cao có tần số lên đến hàng trăm megahertz.

Khi truyền dẫn tín hiệu này qua một sợi dây đơn (jumper wire) dài khoảng 20 cm cắm lỏng lẻo kết nối giữa hai bo mạch:

- Sợi dây hoạt động như một đường truyền sóng không phối hợp trở kháng đầu cuối, đồng thời có độ tự cảm ký sinh lớn (L_wire).
- Khi xảy ra sự kiện chuyển cạnh xung, dòng điện biến thiên nhanh tạo ra hiện tượng dội tín hiệu (ringing) tạo các sóng nhấp nhô ở đỉnh và đáy xung vuông.
- Ngõ vào GPIO của RP2354B sử dụng bộ đệm Schmitt Trigger có khoảng hysteresis chống nhiễu nhỏ (≈ 0,2V ở mức điện áp 3,3V). Điện áp dội tín hiệu của CH8 do tiếp xúc kém và dây dài đã dao động mạnh vượt quá ngưỡng điện áp chuyển trạng thái ($V_{IH} \geq$ 2,0V và $V_{IL} \leq$ 0,8V).
- Hệ quả là một sườn xung thực tế bị vi điều khiển nhận nhầm thành nhiều sườn xung chuyển trạng thái liên tiếp ở tần số cực cao, làm sai lệch kết quả đếm sườn và tính toán tần số trên ứng dụng PC Client.

3. Giải pháp khắc phục triển khai thực tế và kết quả đạt được

Để giải quyết hiện tượng dội tín hiệu này, nhóm thử nghiệm đã thực hiện các cải tiến phần cứng truyền dẫn như sau:

1. Rút ngắn dây truyền tín hiệu: Thay thế sợi dây nối jumper đơn dài bằng dây chất lượng tốt hơn có chiều dài ngắn nhất có thể (< 10 cm) để giảm điện cảm ký sinh L_wire, đẩy tần số cộng hưởng dội tín hiệu lên rất cao vượt ngoài tầm đáp ứng của chip đo.
2. Cải thiện kết nối đất (GND Shielding): Chạy một đường dây đất (GND) song song sát cạnh đường tín hiệu CH8 (tạo thành một cặp dây xoắn nhẹ). Việc này làm giảm diện tích

vòng lặp dòng điện (loop area), từ đó giảm thiểu đáng kể điện cảm vòng lặp và hạn chế hiện tượng xuyên nhiễu (crosstalk) từ các kênh lân cận.
3. Đảm bảo chắc chắn cơ học: Cắm chặt lại các đầu connector pin header để loại bỏ điện trở tiếp xúc chập chờn.

Kết quả sau khắc phục:
Hiện tượng dội tín hiệu trên kênh CH8 biến mất hoàn toàn. Tiến hành đo lại ở mốc tần số cao nhất là 15,625 MHz, toàn bộ 16 kênh đo (bao gồm cả CH8) đều cho kết quả đo đồng nhất, sạch sẽ ở mức 15,6254 - 15,6256 MHz với sai số chu kỳ tiệm cận 0%, khẳng định giải pháp khắc phục nhiễu phần cứng đạt hiệu quả tuyệt đối.

4.3, Tối ưu hóa cấu hình ưu tiên Bus (Bus Priority)

4.3.1 Đánh giá tổng quan hiệu năng hệ thống hiện tại

Hệ thống Logic Analyzer 24 kênh song song dựa trên vi điều khiển RP2354B đã hoàn thành toàn bộ các giai đoạn nghiên cứu, chế tạo thử nghiệm phần cứng và tối ưu hóa phần mềm.
Về mặt phần cứng: Chuỗi liên kết vật lý bao gồm board phát tín hiệu mẫu Pico, 3 IC dịch mức bảo vệ SN74LVC8T245, board mạch chính RP2354B và màn hình LCD ST7789 hoạt động đồng bộ, ổn định. Các vấn đề về chất lượng tín hiệu (Signal Integrity) như hiện tượng dội tín hiệu (ringing) đã được khảo sát kỹ lưỡng và khắc phục triệt để bằng các biện pháp bọc cáp tiếp đất GND.
Về mặt phần mềm (Firmware & Client): Thiết kế tối ưu hóa dual-core phân tách nhiệm vụ (Core 1 chuyên vẽ LCD, Core 0 xử lý lấy mẫu và giao tiếp) giúp giao diện người dùng mượt mà và không bị treo đứng hệ thống. Khối PIO kết hợp DMA đạt tốc độ lấy mẫu cực đại 150 MSa/s thật sau khi cấu hình ưu tiên bus và điều chỉnh tần suất polling của CPU. Các tính năng nâng cao như trigger cạnh ở tầng phần cứng (post-trigger), điều chỉnh linh hoạt sample rate và depth, truyền tải nhị phân tốc độ cao đều đã hoạt động chính xác.

Mặc dù hệ thống đạt đầy đủ các thông số kỹ thuật cam kết (24 kênh, 150 MSa/s, đo sạch tín hiệu <= 15 MHz), nhóm nghiên cứu đề xuất ba giải pháp cải tiến lớn dưới đây nhằm khai thác sâu hơn nữa tiềm năng phần cứng vượt trội của dòng vi xử lý mới RP2350/RP2354B.

4.3.2 Đề xuất giải pháp PIO Interleave nâng tốc độ lấy mẫu lên 300 MSa/s

Hiện tại, tốc độ lấy mẫu cực đại của hệ thống bị giới hạn ở mức 150 MSa/s do một State Machine PIO chạy bằng xung nhịp hệ thống mặc định fs = sysclk = 150 MHz chỉ có thể thực thi một lệnh chụp in pins, 24 sau mỗi chu kỳ xung nhịp.

Để nâng tốc độ lấy mẫu hiệu dụng lên gấp đôi mà không cần thay đổi tần số hoạt động của chip, giải pháp PIO Interleave (Lấy mẫu xen kẽ) được đề xuất:

```mermaid
gantt
    title Sơ đồ lấy mẫu xen kẽ (PIO Interleave) lệch pha 180 độ
    dateFormat  X
    axisFormat %s
    section Clock hệ thống (150 MHz)
    Chu kỳ 1 (T0)          :active, clk1, 0, 2
    Chu kỳ 2 (T1)          :clk2, 2, 4
    Chu kỳ 3 (T2)          :active, clk3, 4, 6
    Chu kỳ 4 (T3)          :clk4, 6, 8
    section PIO State Machine 0
    Chụp Mẫu 0 (Cạnh lên CLK)   :crit, sm0_1, 0, 1
    Chụp Mẫu 2 (Cạnh lên CLK)   :crit, sm0_2, 4, 5
    section PIO State Machine 1
    Chụp Mẫu 1 (Cạnh xuống CLK) :sm1_1, 2, 3
    Chụp Mẫu 3 (Cạnh xuống CLK) :sm1_2, 6, 7
```

Nguyên lý thiết lập phần cứng:
Vi xử lý RP2354B sở hữu cấu trúc PIO mạnh mẽ (3 khối PIO độc lập với tổng cộng 12 State Machine). Ta cấu hình hai State Machine (ví dụ SM0 và SM1 của khối PIO0) cùng giám sát và đọc song song từ một cụm chân GPIO4-27.
Cả hai State Machine được thiết lập bộ chia clkdiv ở giá trị 2.0. Tuy nhiên, thời điểm kích hoạt của SM1 sẽ được làm trễ hơn SM0 đúng 1 chu kỳ xung nhịp hệ thống (lệch pha đúng 180 độ). Việc khởi động lệch pha chính xác này được kiểm soát bằng cách ghi trực tiếp vào thanh ghi kích hoạt chung của PIO.
Khi hoạt động, SM0 sẽ lấy mẫu ở các thời điểm lẻ (T0, T2, T4...), còn SM1 lấy mẫu ở các thời điểm chẵn (T1, T3, T5...). Mỗi SM đẩy dữ liệu vào một kênh DMA riêng biệt.

Thuật toán ghép nối (Interleave Merging) trên CPU:

Sau khi quá trình chụp burst hoàn tất, hai kênh DMA sẽ điền đầy dữ liệu vào hai bộ đệm SRAM độc lập. CPU Core 0 sẽ thực hiện một vòng lặp ghép xen kẽ (merging loop) để sắp xếp lại dữ liệu thu được:
Buffer_Final[2i] = Buffer_SM0[i] và Buffer_Final[2i+1] = Buffer_SM1[i]
Kết quả cuối cùng cho ra bộ đệm mẫu có tốc độ lấy mẫu hiệu dụng đạt 300 MSa/s (khoảng cách giữa hai mẫu liên tiếp giảm xuống chỉ còn 3,33 ns), nâng dải đo tín hiệu sạch lên xấp xỉ 30 MHz.

Để tránh xung đột truy cập bus khi hai kênh DMA hoạt động đồng thời ở tốc độ rất cao, hai bộ đệm DMA sẽ được phân bổ nằm ở hai bank SRAM độc lập (trong số 10 bank SRAM của RP2354B), tận dụng tối đa băng thông đa luồng của cấu trúc bus crossbar.

4.3.3 Đề xuất giải pháp Ép xung chip hệ thống (Overclocking sysclk)

Vi điều khiển RP2350/RP2354 được sản xuất trên quy trình công nghệ bán dẫn tiên tiến, cho phép vận hành cực kỳ ổn định ở các mức xung nhịp cao hơn nhiều so với tần số mặc định 150 MHz. Thực tế thực nghiệm trên cộng đồng nhà phát triển chỉ ra chip có thể ép xung dễ dàng lên dải từ 200 MHz đến 300 MHz mà không gặp lỗi nhiệt độ hay mất ổn định phần cứng.

Cách thức triển khai trên Firmware:
Sử dụng thư viện SDK cấu hình lại bộ khóa pha PLL (pll_sys) để nâng tần số sysclk. Ví dụ thiết lập lên mốc 200 MHz ổn định bằng cách ghi nhận lại các tham số chia tần trong bộ sinh xung nhịp của SDK:

```cpp
// Thiết lập sysclk lên 200 MHz
set_sys_clock_khz(200000, true);
```

Việc tăng xung nhịp hệ thống lên 200 MHz sẽ trực tiếp nâng tốc độ lấy mẫu của một State Machine PIO lên 200 MSa/s, kéo dài băng thông Nyquist lên 100 MHz và dải đo sạch >= 10x oversampling lên mức 20 MHz.
Nếu kết hợp đồng thời giải pháp lấy mẫu xen kẽ PIO Interleave ở trên, hệ thống có thể đạt tốc độ lấy mẫu hiệu dụng lên đến 400 MSa/s hoặc 600 MSa/s (ở xung nhịp ép tối đa 300 MHz).

Các lưu ý kỹ thuật khi ép xung:

Bảo vệ ngoại vi: Khi thay đổi sysclk, cần cấu hình lại các bộ chia tần số cấp cho khối USB (48 MHz) và LCD SPI để ngăn ngừa lỗi mất kết nối USB hoặc màn hình bị hiển thị rác.
Điện áp và nhiệt độ: Mạch nguồn LDO nội của chip có thể cần được cấu hình tăng nhẹ điện áp lõi (core voltage VREG) từ mức mặc định 1.1V lên 1.2V hoặc 1.3V để duy trì trạng thái chuyển mạch tốc độ cao ổn định của các transistor trong nhân ARM và khối PIO.

4.3.4 Phát triển tính năng Pre-trigger sử dụng bộ đệm vòng tròn (Ring Buffer DMA)

Trong kiến trúc firmware Giai đoạn 3 hiện tại, hệ thống mới chỉ hỗ trợ cơ chế Post-trigger (Kích hoạt sau). Tức là khi phát hiện cạnh sườn tín hiệu mong muốn tại mẫu số 0, hệ thống mới kích hoạt DMA ghi và lưu trữ N mẫu xảy ra sau thời điểm đó.
Hạn chế của phương pháp này là người dùng hoàn toàn không biết trạng thái của các kênh tín hiệu đo đạc ở các chu kỳ ngay trước thời điểm xảy ra sự kiện trigger (ví dụ: các dữ liệu chuẩn bị, hoặc nguyên nhân dẫn đến lỗi chuyển mạch đột ngột của mạch đích).

Để giải quyết vấn đề này, tính năng Pre-trigger (Kích hoạt trước) được đề xuất triển khai dựa trên cơ chế bộ đệm vòng tròn (Circular Ring Buffer):

```mermaid
stateDiagram-v2
    [*] --> Continuous_Sampling : Khởi tạo & Chạy DMA Vòng tròn
    Continuous_Sampling --> Continuous_Sampling : DMA liên tục ghi đè SRAM (Ghi xoay vòng)
    Continuous_Sampling --> Trigger_Detected : Phát hiện Cạnh Trigger (Chân Wait chuyển tiếp)
    Trigger_Detected --> Post_Capture : DMA ghi nốt M mẫu Post-Trigger (M < N)
    Post_Capture --> Send_To_PC : Dừng DMA & PIO
    Send_To_PC --> [*] : Gửi N mẫu về PC (gồm N-M mẫu trước cạnh và M mẫu sau cạnh)
```

Nguyên lý hoạt động của Ring Buffer DMA:
1. Lấy mẫu liên tục: Cấu hình bộ điều khiển DMA ở chế độ ghi vòng tròn (Ring Buffer) vào một mảng đệm có kích thước cố định N trong SRAM. Khi bắt đầu chạy, PIO và

DMA liên tục lấy mẫu và ghi đè tuần hoàn vào mảng đệm này mà không dừng lại, ngay cả khi chưa có trigger.
2. Giám sát sự kiện Trigger: Sử dụng một State Machine PIO chạy song song làm nhiệm vụ phát hiện cạnh sườn chuyển tiếp (Rising/Falling). Khi phát hiện sự kiện trigger, State Machine này sẽ phát một xung ngắt (IRQ) đến CPU hoặc kích hoạt một kênh DMA thứ hai.
3. Ghi nốt dữ liệu Post-trigger: Khi nhận được tín hiệu trigger, bộ điều khiển DMA được cấu hình ghi nốt thêm đúng M mẫu (với M < N, đại diện cho lượng mẫu Post-trigger muốn giữ lại) rồi lập tức dừng hẳn quá trình capture.
4. Tái cấu trúc bộ đệm dữ liệu: Dữ liệu trong SRAM lúc này là một mảng vòng tròn. CPU sẽ dựa vào con trỏ ghi cuối cùng của DMA (Write Pointer) để sắp xếp lại mảng:
N - M mẫu đứng trước con trỏ ghi đại diện cho trạng thái tín hiệu trước trigger (Pre-trigger).
M mẫu đứng sau đại diện cho trạng thái sau trigger (Post-trigger).

Tính năng này sẽ giúp thiết bị Logic Analyzer nâng tầm hiệu năng, tiệm cận các thiết bị đo đạc chuyên dụng thương mại trên thị trường, hỗ trợ kỹ sư phân tích sâu sắc các sự cố phần cứng phức tạp.

4.4, Thiết kế giao thức truyền tin Serial

4.4.1 Kết luận kết quả đạt được

Đồ án nghiên cứu đã thiết kế, chế tạo phần cứng và xây dựng thành công hệ thống Logic Analyzer 24 kênh song song, tốc độ lấy mẫu cực đại đạt 150 MSa/s dựa trên chip vi điều khiển thế hệ mới RP2354B (nhân RP2350). Trải qua các giai đoạn đo kiểm nghiệm tích hợp với nguồn phát xung mẫu thực tế (Raspberry Pi Pico) và phân tích dữ liệu qua ứng dụng máy tính, hệ thống đã chứng minh độ tin cậy và đạt được đầy đủ các chỉ tiêu kỹ thuật cam kết:

1. 1. Hoạt động ổn định ở hai chế độ:
    a. Chế độ Standalone Mode: Cho phép hệ thống tự vận hành độc lập. Core 1 đảm nhận nhiệm vụ vẽ dạng sóng của 2 kênh đo (CH1 và CH2) lên màn hình LCD ST7789 với tần số quét mượt mà (150 – 220 ms), trong khi Core 0 liên tục giám sát nút bấm điều khiển và cập nhật trạng thái đo đạc với chu kỳ quét live 100 Hz.
    b. Chế độ PC Mode: Chuyển đổi linh hoạt bằng nút bấm vật lý (giữ SW1 1 giây), cho phép kết nối máy tính qua cổng ảo USB-CDC. PC Client có thể gửi lệnh điều chỉnh cấu hình đo (Sample Rate từ 2.3 kHz đến 150 MHz, Depth từ 16 đến 65.536 mẫu) và nhận dữ liệu chính xác tuyệt đối.

2. 2. Khả năng lấy mẫu tốc độ cao: Đạt tốc độ lấy mẫu thực tế 150 MSa/s, đáp ứng hoàn hảo spec đo sạch tín hiệu số có tần số 15 MHz (đảm bảo oversampling tối thiểu 9,6 lần).
3. 3. Tính năng Trigger nâng cao: Tích hợp thành công cơ chế trigger cạnh sườn (post-trigger) ở cấp độ phần cứng thông qua việc vá trực tiếp mã lệnh PIO ở runtime. Hệ thống hỗ trợ khôi phục sạch sau khi gặp sự cố timeout (5 giây) mà không bị treo phần cứng.
4. 4. Tối ưu hóa đường truyền dữ liệu: Phát triển song song hai định dạng truyền dữ liệu là ASCII và Binary. Trong đó, định dạng Binary (truyền thô Little-Endian 32-bit qua driver USB) giúp giảm dung lượng gói tin tải và tăng tốc độ phản hồi thực tế lên gấp ~1,75 lần, đạt thời gian truyền toàn bộ gói mẫu cực đại 65.536 mẫu trong chưa đầy 0,28 giây.

4.4.2 Bài học kinh nghiệm rút ra

1. Tối ưu hóa đường truyền dữ liệu phần cứng tốc độ cao (DMA, PIO)

Bộ xử lý PIO (Programmable I/O) trên dòng chip RP2000/RP2300 là một công cụ cực kỳ mạnh mẽ để giao tiếp with các bus ngoại vi tốc độ cao. Bài học kinh nghiệm lớn nhất khi sử dụng PIO là việc cấu hình tối ưu hóa luồng dữ liệu (data flow) ngay tại phần cứng:

- Cấu hình dịch bit tự động (autopush) kết hợp gộp bộ đệm FIFO truyền/nhận (sm_config_set_fifo_join với PIO_FIFO_JOIN_RX) giúp tạo ra một bộ đệm tạm thời 8 mức sâu cho dữ liệu nhận.
- Điều này giúp giảm tần suất yêu cầu truyền của DMA, tạo ra một khoảng đệm an toàn đề phòng trường hợp DMA bị trễ nhịp do bận xử lý các giao dịch bus khác, từ đó hạn chế tối đa lỗi mất mẫu (FIFO overflow).

2. Kỹ thuật cấu hình phân bổ và ưu tiên bus hệ thống (Bus Priority & Bus Contention)

Tranh chấp bus hệ thống (bus contention) là một vấn đề sống còn khi xây dựng các ứng dụng nhúng thời gian thực tốc độ cao.

- Khi DMA phải chuyển dữ liệu ở băng thông cực lớn (600 MB/s ở tốc độ lấy mẫu 150 MSa/s), bất kỳ hành động tranh chấp bus nào từ phía CPU (như việc chạy vòng lặp kiểm tra trạng thái DMA liên tục với tần số cao) cũng có thể kéo tụt hiệu năng thực tế của DMA xuống 17% (chỉ đạt 125 MSa/s), gây ra sai lệch nghiêm trọng trên trục thời gian đo được.

- Giải pháp khắc phục: Cần cấu hình thanh ghi ưu tiên bus (bus_ctrl_hw->priority) để cấp quyền ưu tiên tuyệt đối cho DMA. Đồng thời, thiết kế các vòng lặp kiểm tra của CPU có chèn các khoảng trễ thưa (như busy_wait_us_32(10)) để CPU giải phóng đường truyền cho DMA hoạt động độc quyền.

3. Chất lượng tín hiệu phần cứng truyền dẫn tần số cao (Signal Integrity)

Khi tần số tín hiệu số vượt quá ngưỡng 10 MHz, chất lượng truyền dẫn dây kết nối vật lý đóng vai trò quyết định đến độ chính xác của phép đo.

- Xung vuông tần số cao có sườn xung rất dốc (tốc độ sườn lên/xuống cực nhanh, rise/fall time khoảng 1,5 – 3 ns) chứa các thành phần sóng hài bậc cao lên tới hàng trăm MHz. Cáp kết nối dài, không bọc chống nhiễu hoặc tiếp xúc lỏng lẻo sẽ tạo ra điện cảm ký sinh lớn, gây ra hiện tượng dội tín hiệu (ringing) vượt quá ngưỡng điện áp hysteresis của Schmitt trigger đầu vào MCU.

Giải pháp khắc phục: Bắt buộc phải sử dụng dây cáp truyền tín hiệu ngắn nhất có thể (< 10 cm), bọc tiếp đất (GND) song song sát cạnh các đường tín hiệu nhạy cảm để giảm diện tích vòng lặp dòng điện (loop area), triệt tiêu nhiễu dội và xuyên nhiễu (crosstalk).

4.5. Ứng dụng trên máy tính (PC Client)

Bên cạnh khối firmware, ứng dụng PC Client là giao diện trung tâm để người dùng cấu hình, gửi lệnh kích hoạt đo, tiếp nhận dữ liệu và hiển thị trực quan giản đồ xung 24 kênh. Ứng dụng được phát triển bằng ngôn ngữ Python, triển khai dưới dạng công cụ dòng lệnh (CLI) giúp tối ưu hóa hiệu năng xử lý dữ liệu thô.

Ứng dụng PC Client được phát triển bằng ngôn ngữ lập trình Python và triển khai dưới dạng công cụ dòng lệnh (CLI) mẫu thông qua hai tập lệnh chính: *capture_burst.py* (phục vụ chế độ chụp burst tốc độ cao) và *la_viewer.py* (phục vụ chế độ Live View tần số thấp). Thiết kế của ứng dụng bám sát các đặc tả kỹ thuật trong *PC_APP_SPEC.md* và giao thức truyền thông đã chốt trong *PROTOCOL.md*.

4.5.1 Lập trình xử lý cổng nối tiếp (PySerial) và Giao thức truyền thông

Ứng dụng sử dụng thư viện *pyserial* để thiết lập kết nối với cổng COM ảo USB-CDC. Cơ chế tự động dò tìm thiết bị được tích hợp qua hàm *find_port()*, giúp xác định đúng bo mạch RP2354B giữa các thiết bị ngoại vi khác.

Ứng dụng PC Client sử dụng thư viện *pyserial* để thiết lập kết nối và trao đổi dữ liệu với cổng COM ảo (Virtual COM Port) được tạo ra bởi driver USB-CDC trên RP2354B.

1. Quét tìm thiết bị tự động (Auto-discovery)

Để đơn giản hóa quá trình kết nối cho người sử dụng, PC Client được tích hợp cơ chế tự động dò tìm cổng serial khả dụng. Hàm *find_port()* sử dụng thư viện *glob* để quét các mẫu cổng đặc trưng trên hệ điều hành Linux:

Đối với hệ điều hành Windows, cơ chế này có thể mở rộng bằng cách liệt kê danh sách các cổng *COMx* có sẵn trong registry hoặc thông qua thư viện *serial.tools.list_ports*.

2. Thiết lập cấu hình và gửi lệnh điều khiển

Giao thức USB-CDC v1.2 quy định tập lệnh điều khiển dạng ký tự ASCII (1 byte). Trước khi tiến hành chụp tín hiệu, PC Client gửi các lệnh cấu hình tham số đo đạc thông qua luồng serial và kiểm tra phản hồi *OK* từ thiết bị:

- **Đặt tốc độ lấy mẫu (R<hz>\n):** Ví dụ gửi lệnh R150000000\n để yêu cầu lấy mẫu ở tốc độ 150 MSa/s.
- **Đặt độ sâu mẫu (D<depth>\n):** Ví dụ gửi lệnh D8192\n để yêu cầu chụp 8192 mẫu dữ liệu.
- **Cài đặt Trigger (T<ch><edge>\n hoặc T0\n):** Ví dụ gửi lệnh T3R\n để cấu hình trigger cạnh lên trên kênh CH3, hoặc T0\n để tắt trigger và chụp ngay.

Hàm *send_cmd_expect_ok()* đảm bảo các cấu hình này được gửi và xác nhận thành công trước khi bắt đầu chu trình chụp burst:

```

**3. Tiếp nhận và giải mã khung dữ liệu Burst**

PC Client hỗ trợ hai chế độ tiếp nhận dữ liệu tương ứng với hai lệnh `'C'` (ASCII) và `'B'` (Binary) nhằm tối ưu hóa tốc độ truyền dẫn:
```

Chế độ truyền dạng văn bản ASCII (Lệnh 'C')

Khi gửi lệnh 'C', vi điều khiển chụp tín hiệu và chuyển đổi từng mẫu dữ liệu 24-bit thành chuỗi mã Hex 6 ký tự viết hoa. Dòng dữ liệu được bao bọc trong một khung văn bản được định nghĩa bởi từ khóa khởi đầu *BURST_BEGIN* và kết thúc *BURST_END*.

- **Luồng xử lý:** Client đọc từng dòng bằng hàm readline(), sử dụng biểu thức chính quy (regular expression) để trích xuất các giá trị số và kiểm tra tính toàn vẹn của khung dữ liệu:

Chế độ truyền nhị phân tốc độ cao (Lệnh 'B')

Để vượt qua giới hạn về tốc độ định dạng chuỗi của CPU và tối đa hóa băng thông USB-CDC, giao thức hỗ trợ chế độ nhị phân thông qua lệnh 'B'. Khung truyền bao gồm một tiêu đề ASCII dạng BURST_BIN depth=<depth> rate=<rate>\n, ngay sau đó là khối dữ liệu nhị phân thô có kích thước đúng bằng depth × 4 bytes (mỗi mẫu được lưu trữ dưới dạng số nguyên không dấu 32-bit Little-Endian, trong đó 24-bit thấp chứa trạng thái logic của các kênh). Khung truyền kết thúc bằng chuỗi văn bản BIN_END\n.

- **Luồng xử lý:** Hàm request_burst_bin() thực hiện đọc trực tiếp khối dữ liệu nhị phân từ bộ đệm nhận của Serial với độ dài cố định, giúp giảm đáng kể thời gian truyền dẫn thực tế xuống khoảng 3.5 lần so với chế độ ASCII:

```

**4. Cơ chế xử lý lỗi chuẩn hóa**

Ứng dụng PC Client phân tích các mã lỗi nhận được từ thiết bị thông qua hàm *_raise_if_err()* để đưa ra chỉ dẫn rõ ràng cho người dùng:

- *ERR_MODE_LCD*: Xảy ra khi PC gửi lệnh chụp trong khi thiết bị vẫn ở chế độ màn hình độc lập (MODE_LCD). PC Client sẽ đưa ra cảnh báo yêu cầu người dùng *"Giữ nút SW1 ≥ 1s trên board để chuyển sang PC mode"*.
- *ERR_TRIG_TIMEOUT*: Xảy ra khi bật trigger nhưng không xuất hiện sườn tín hiệu kích hoạt trên chân đo trong vòng 5 giây.

## 4.5.2 Xử lý dữ liệu nhị phân tốc độ cao bằng NumPy

Sau khi nhận mảng dữ liệu mẫu 24-bit, PC Client sử dụng thư viện **NumPy** để giải mã. Thay vì dùng vòng lặp Python, các phép toán vector hóa được áp dụng để trích xuất trạng thái logic của từng kênh CHk dựa trên vị trí bit. Thuật toán này giúp tính toán tự động tần số, số cạnh xung và chu kỳ nhiệm vụ (Duty Cycle) cho 65.536 điểm mẫu trong chưa đầy 1.5 ms.
```

Sau khi nhận được mảng dữ liệu mẫu samples (mỗi phần tử là một số nguyên không dấu 24-bit biểu diễn trạng thái của 24 kênh tại một thời điểm lấy mẫu), PC Client sử dụng thư viện xử lý số liệu **NumPy** để giải mã và phân tích dữ liệu một cách hiệu quả nhờ các phép toán vector hóa (vectorized operations) thay vì sử dụng vòng lặp Python thông thường.

1. Mô hình dữ liệu và giải mã kênh

Mỗi phần tử thứ i trong mảng mẫu đại diện cho trạng thái của 24 chân GPIO đo đạc tại thời điểm ti = i / fs. Kênh đo CHk (k $\in$ [1, 24]) tương ứng với bit thứ 24 - k của mẫu dữ liệu. Giá trị logic của kênh CHk tại điểm mẫu thứ i được tính theo công thức:

bit(CHk, i) = (samples[i] / 2^(24-k)) mod 2

Trong mã nguồn, phép toán này được vector hóa hoàn toàn trên toàn bộ mảng NumPy:

2. Thuật toán tự động phân tích và đo đạc thông số tín hiệu

Với mỗi kênh đo, PC Client thực hiện tính toán tự động số lượng cạnh sườn (edges), tần số (frequency) và chu kỳ nhiệm vụ (duty cycle).

Đếm số cạnh sườn chuyển mức

Sử dụng hàm sai phân *np.diff()* để xác định sự thay đổi logic giữa hai mẫu liên tiếp:

- Mức d[i] = 1 đại diện cho sườn lên (Rising edge).
- Mức d[i] = -1 đại diện cho sườn xuống (Falling edge).

Tính chu kỳ nhiệm vụ (Duty Cycle)

Chu kỳ nhiệm vụ được ước lượng dựa trên giá trị trung bình logic của toàn bộ mảng dữ liệu (tương ứng với tỷ lệ mẫu ở mức cao trong cửa sổ đo):

Duty Cycle (%) = mean(bit) × 100%

Đo tần số thông qua khoảng cách sườn xung (Samples Per Period - spp)

Thuật toán tìm kiếm tất cả các vị trí xảy ra sườn lên (Rising edges) trong mảng sai phân: Nếu tìm thấy tối thiểu hai sườn lên (M ≥ 2), khoảng cách trung bình giữa các sườn lên (đơn vị tính bằng số điểm mẫu - spp) được xác định bằng cách lấy sai phân giữa các chỉ số sườn lên liên tiếp:

spp = [1 / (M-1)] * Σ (rising[j] - rising[j-1]) (từ j=1 đến M-1)

Tần số đo đạc thực tế của tín hiệu (f_measured) được tính bằng tỷ lệ giữa tần số lấy mẫu thực tế của thiết bị (fs) và số mẫu trung bình trên một chu kỳ tín hiệu (spp):

$$f_{measured} = fs / spp$$

Nhờ thực hiện tính toán song song trên mảng NumPy, thời gian phân tích và đưa ra kết quả đo đạc 24 kênh đối với 65,536 điểm mẫu chỉ mất dưới 1.5 ms, đảm bảo tính tương tác thời gian thực cao.

4.5.3 Hiển thị dạng sóng và giao diện trực quan (Matplotlib)

Để quan sát trực quan, PC Client dùng thư viện **Matplotlib** để vẽ giản đồ xung. Dạng sóng được biểu diễn kiểu bậc thang (Step Plot) với kỹ thuật dịch chuyển biên độ (Vertical Offsetting) để hiển thị đồng thời 24 kênh mà không bị chồng chéo. Hệ thống cũng hỗ trợ xuất dữ liệu ra file CSV để phân tích chuyên sâu trên các phần mềm như Excel hay Matlab.

Để người dùng có thể quan sát trực quan giản đồ xung logic của các kênh, PC Client sử dụng thư viện vẽ đồ thị **Matplotlib** để mô phỏng hiển thị giống như giao diện của các máy đo Logic Analyzer chuyên nghiệp.

1. Thiết lập trục thời gian tự động

Trục hoành biểu thị thời gian thực tế của quá trình lấy mẫu. Khoảng thời gian giữa các điểm mẫu được tính chính xác bằng công thức ti = (i / fs) × 10^6 (µs). Nhờ vậy, đơn vị hiển thị trên trục hoành tự động co giãn chính xác theo tham số rate nhận được từ tiêu đề khung dữ liệu, tránh hiện tượng co giãn sai lệch trục thời gian.

2. Biểu diễn dạng sóng bậc thang (Step Plot)

Tín hiệu logic số chỉ có hai trạng thái HIGH và LOW, do đó đồ thị dạng sóng phải được vẽ dưới dạng bậc thang (giữ nguyên mức logic của điểm mẫu hiện tại cho đến khi chuyển tiếp sang điểm mẫu kế tiếp). Matplotlib hỗ trợ việc này thông qua hàm step với tùy chọn where="post":

3. Kỹ thuật xếp chồng dọc các kênh đo (Vertical Offsetting)

Để hiển thị đồng thời nhiều kênh đo trên cùng một đồ thị mà không bị chồng chéo tín hiệu lên nhau, PC Client áp dụng một kỹ thuật dịch chuyển biên độ (vertical offset). Mỗi kênh đo CHk (k = 1 ... Nchannels) được gán một mức nền (offset base) riêng biệt:

$$base_k = (Nchannels - k) \times 1.5$$

Tín hiệu logic của kênh (sau khi nhân với hệ số biên độ 0.9 để tạo khoảng hở thẩm mỹ 0.6 giữa các dòng) được cộng trực tiếp với giá trị base_k này. Trục tung sau đó được cấu hình ẩn các giá trị số và thay thế bằng các nhãn tên kênh tương ứng (CH1 đến CH24).

4. Tính năng xuất bản dữ liệu (Export)

Đáp ứng các yêu cầu kiểm thử và lập báo cáo kỹ thuật, ứng dụng PC Client cung cấp các cơ chế xuất bản dữ liệu:

- **Xuất đồ thị dạng ảnh:** Hỗ trợ lưu trữ giản đồ xung dưới định dạng ảnh chất lượng cao PNG thông qua Matplotlib.

Xuất dữ liệu số dạng CSV: Hỗ trợ chuyển đổi toàn bộ mảng dữ liệu logic của các kênh thành bảng dữ liệu CSV để mở trên các phần mềm bảng tính hoặc các công cụ phân tích dữ liệu chuyên sâu khác (như Matlab, Excel, Python Pandas). Định dạng file CSV tuân thủ quy tắc: dòng đầu tiên chứa tên các tiêu đề cột (index, CH1, CH2, ..., CH24), các dòng tiếp theo chứa chỉ số mẫu và giá trị logic nhị phân tương ứng của từng kênh.

5. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

5.1, Thiết lập môi trường thử nghiệm

Để tiến hành thực nghiệm, đo kiểm và đánh giá toàn diện năng lực hoạt động của thiết bị Logic Analyzer 24 kênh, nhóm nghiên cứu đã thiết lập một hệ thống thử nghiệm tích hợp đồng bộ bao gồm: nguồn phát xung mẫu (DUT), khối đệm dịch mức logic bảo vệ, thiết bị đo chính (Logic Analyzer) và giao diện máy tính phân tích dữ liệu.

5.1.1 Cấu hình các khối thiết bị trong hệ thống

1. Nguồn phát tín hiệu mẫu (Device Under Test - DUT)

Nguồn phát xung thử nghiệm là một bo mạch Raspberry Pi Pico sử dụng vi điều khiển RP2040 độc lập.

Tầng cứng: Phát 24 kênh xung độc lập thông qua các chân GPIO từ GP0 đến GP22 và chân GP26.

Tầng mềm: Chương trình phát xung được tối ưu hóa chạy trên Pico bằng ngôn ngữ C sử dụng Pico SDK. Để đảm bảo tần số xung test ra hoàn toàn sạch và không bị ảnh hưởng bởi độ trễ thực thi của CPU, phần mềm sử dụng khối tạo xung PWM phần cứng trên chip kết hợp điều khiển phi nghẽn (non-blocking) qua hàm *to_ms_since_boot()*.

Mức tần số kiểm thử: Tần số được thay đổi theo thang đo định sẵn để kiểm tra độ tin cậy của Logic Analyzer:

- **Xung tần số thấp (Live Mode):** Khoảng 0.9 Hz đến 5.0 Hz để người dùng dễ dàng quan sát sự thay đổi logic thời gian thực bằng mắt thường trên màn hình LCD.

- **Xung tần số cao (Burst Mode):** Được đặt tại các mốc xung nhịp 1.000 MHz, 5.000 MHz, 9.615 MHz và 15.625 MHz.

2. Khối đệm dịch mức và bảo vệ (Level Shifter Front-end)
Các tín hiệu ngõ ra từ Pico được kết nối trực tiếp đến board đo Logic Analyzer qua cáp dẹt chống nhiễu. Trên board Logic Analyzer, tín hiệu được dẫn qua 3 IC dịch mức **SN74LVC8T245** cấp nguồn kép:
Nguồn ngõ vào (V_{CCA}): Nối với mức điện áp tín hiệu đo từ thiết bị phát (3.3V cấp từ chân nguồn Pico).
Nguồn ngõ ra (V_{CCB}): Nối với nguồn logic của RP2354B (3.3V).
Chân điều khiển xuất ngõ ra (*OE*): Kết nối trực tiếp với chân GPIO28 của RP2354B. Khi bắt đầu khởi động hoặc chuyển đổi các chế độ, chân *OE* được kích hoạt mức thấp (LOW) để bật bộ đệm dịch mức. Khi chuyển chế độ bảo vệ hoặc reset, *OE* chuyển HIGH để đưa các ngõ vào của RP2354B về trạng thái trở kháng cao (Hi-Z).

3. Vi điều khiển xử lý chính và LCD ST7789
Bo mạch chính sử dụng chip xử lý **RP2354B** (nhân ARM Cortex-M33, xung nhịp mặc định 150 MHz) chịu trách nhiệm điều khiển toàn bộ hệ thống.
Bộ kênh đo: GPIO4 đến GPIO27 nhận dữ liệu 24 kênh song song.
Màn hình hiển thị độc lập: LCD ST7789 độ phân giải 320 x 240 pixel kết nối qua giao tiếp SPI0 tốc độ cao (12.5 MHz).
Nút bấm điều khiển: SW1 (GPIO31), SW2 (GPIO32), SW3 (GPIO33) sử dụng để di chuyển, chọn và chuyển đổi giữa các màn hình LCD.
4. Giao diện máy tính (PC Client)
PC Client chạy trên máy tính Ubuntu sử dụng kịch bản Python CLI *capture_burst.py*. Script này giao tiếp với Logic Analyzer thông qua cổng USB ảo */dev/ttyACM0* để cấu hình, kích hoạt và nhận kết quả đo.

5.1.2 Quy trình thực hiện thử nghiệm tích hợp

Quy trình kiểm thử tích hợp được tiến hành theo các bước sau:
1. Nối chung Mass đất (Common Ground): Đất (GND) của bo mạch phát tín hiệu Pico và bo mạch Logic Analyzer được kết nối chung thông qua dây dẫn tiết diện lớn. Đây là yêu cầu bắt buộc để triệt tiêu chênh lệch điện áp đất giữa hai mạch, giảm nhiễu vòng lặp đất (ground loop) và tránh sai lệch mức ngưỡng logic HIGH/LOW.
2. Khởi động hệ thống: Cấp nguồn USB cho cả hai bo mạch. Thiết bị Logic Analyzer khởi động mặc định ở chế độ hiển thị màn hình LCD Standalone (`MODE_LCD`).

3. Thực nghiệm chế độ Standalone: Sử dụng Live View để quan sát 24 kênh ở tần số thấp. Sau đó, chuyển sang chế độ Scope để cấu hình trigger cạnh, độ rộng cửa sổ thời gian, tiến hành bấm nút đo và kiểm tra khả năng Zoom/Pan giản đồ xung trên màn hình LCD.
4. **Thực nghiệm chế độ PC Mode:** Giữ nút SW1 trên board trong thời gian ≥ 1 giây để chuyển sang *MODE_PC*. Trên terminal của máy tính, chạy Python CLI để gửi cấu hình tốc độ lấy mẫu (f_s), độ sâu bộ nhớ (N) và kích hoạt quá trình chụp burst (ASCII qua lệnh C và Binary qua lệnh B).
5. Ghi nhận và xử lý số liệu: Thu thập dữ liệu từ PC Client, phân tích tần số đo được để đối chiếu với tần số phát thực tế, lập bảng thống kê và đánh giá sai số trục thời gian.

5.2, Đánh giá chế độ Standalone (LCD)

Chế độ Standalone (MODE_LCD) cho phép thiết bị Logic Analyzer tự vận hành độc lập mà không cần kết nối với máy tính. Toàn bộ thông tin trạng thái và dạng sóng tín hiệu được xử lý và hiển thị trực tiếp lên màn hình LCD ST7789 TFT 320×240 pixel.

5.2.1 Kiến trúc Dual-Core phục vụ hiển thị mượt mà

Một trong những thách thức lớn nhất của các thiết bị đo tự chế là độ trễ vẽ màn hình làm ảnh hưởng đến quá trình lấy mẫu tín hiệu (gây mất mẫu) hoặc ngược lại, việc ưu tiên lấy mẫu làm màn hình bị giật lag. Để giải quyết vấn đề này, phần mềm hệ thống khai thác tối đa kiến trúc hai nhân (Dual-Core) của RP2354B:

- **Core 0 (Điều khiển & Lấy mẫu):** Thực hiện vòng lặp điều khiển chính ở tần số 100 Hz (chu kỳ 10 ms). Nhân này đảm nhận việc quét nút nhấn chống rung (debounce), quản lý máy trạng thái (state machine) của hệ thống, thực hiện quét nhanh tín hiệu thời gian thực để ghi vào bộ đệm vòng (ring buffer) và đóng vai trò là bộ ghi duy nhất của các biến trạng thái dùng chung.
- **Core 1 (Chuyên trách hiển thị):** Chạy độc lập hoàn toàn với Core 0. Nhân này liên tục kiểm tra sự thay đổi của biến đếm thế hệ giao diện (g_ui_gen). Mỗi khi phát hiện sự thay đổi, Core 1 thực hiện các tác vụ vẽ hình học lên LCD qua bus SPI tốc độ cao. Do Core 0 chỉ ghi và Core 1 chỉ đọc trạng thái, thiết kế này hoạt động không cần khóa đồng bộ (lock-free), giúp giao diện màn hình phản hồi mượt mà trong khi tần số lấy mẫu ngõ vào vẫn được bảo toàn tuyệt đối.

5.2.2 Thực nghiệm giao diện đo trực tiếp Live View

Màn hình Live View (SCR_LIVE) cung cấp khả năng giám sát trực tiếp trạng thái logic của 24 kênh với chu kỳ quét 100 Hz.

1. Bố cục hiển thị

- **Màn hình chi tiết (12 trang):** Mỗi trang hiển thị chi tiết dạng sóng của 2 kênh đo cạnh nhau. Status bar phía trên hiển thị trạng thái Level Shifter (ON/OFF), trạng thái logic thời gian thực của 2 kênh (H/L), số trang hiện tại (Pg1/12 đến Pg12/12) và số lần đổi trạng thái.
- **Màn hình tổng quan (Overview):** Trang thứ 13 hiển thị đồng thời trạng thái logic của toàn bộ 24 kênh đo dưới dạng các thanh vạch đứng thu nhỏ, giúp người dùng bao quát nhanh hoạt động của toàn bộ bus dữ liệu.

2. Kết quả đánh giá thực nghiệm

- Xung vuông tần số thấp (0.9 Hz - 5 Hz) từ Pico phát được đưa vào các chân đo. Dạng sóng hiển thị trên LCD cuộn liên tục từ phải qua trái.
- **Độ trễ vẽ hình:** Thời gian render toàn bộ khung sóng trên LCD đo được dao động từ 150 ms đến 220 ms, hoàn toàn đáp ứng độ mượt cần thiết cho mắt người quan sát.
- Khi nhấn SW2 để chuyển trang hoặc SW1 để toggle bật/tắt Level Shifter, phản hồi trên LCD xảy ra tức thì (< 50ms) và đồ thị dạng sóng của các trang mới được cập nhật lập tức nhờ dữ liệu được "phát lại" liên tục từ bộ đệm vòng tĩnh 280 mẫu (g_ring) được Core 0 duy trì trong SRAM.

5.2.3 Thực nghiệm giao diện Scope Viewer tĩnh (Zoom & Pan)

Khi chuyển sang màn hình thiết lập Scope, người dùng có thể cấu hình thời gian đo từ 437 microgiây đến 5 giây (tương ứng tốc độ lấy mẫu từ 150 triệu mẫu trên giây xuống 13.1 nghìn mẫu trên giây) và cấu hình trigger. Sau khi nhấn đo và trigger thành công, thiết bị chụp 65536 mẫu vào bộ nhớ SRAM rồi hiển thị tĩnh trên giao diện xem sóng Scope Viewer.

1. Thuật toán nén điểm mẫu vẽ sóng (Glitch/Edge Preservation)

Màn hình LCD có chiều rộng vùng vẽ sóng cố định là 280 pixel, trong khi độ sâu mẫu tĩnh lên đến 65536 mẫu. Khi hiển thị toàn bộ cửa sổ ở mức thu nhỏ tối đa, mỗi pixel cột x trên màn hình phải đại diện cho một khoảng mẫu dữ liệu khá lớn. Cụ thể, khoảng dữ liệu này bắt đầu từ chỉ số mẫu bằng x nhân với nhịp mẫu chia cho 280, và kéo dài đến trước chỉ số mẫu bằng x cộng 1 nhân với nhịp mẫu chia cho 280.

- **Vách dọc (Vertical Bar):** Nếu trong khoảng mẫu có cả mức logic 0 và 1 (tín hiệu có chuyển trạng thái), chương trình sẽ vẽ một vạch đứng nối từ mức LOW đến HIGH tại cột pixel x. Kỹ thuật này giúp bảo toàn các gai nhiễu (glitches) và sườn xung tần số cao, ngăn ngừa hiện tượng mất xung hay méo dạng sóng khi zoom out.
- **Đoạn ngang (Horizontal Segment):** Nếu trong khoảng mẫu chỉ có duy nhất mức logic 0 hoặc 1 (tín hiệu phẳng), chương trình sẽ gộp các cột pixel liên tiếp cùng mức lại và vẽ một đường ngang bằng lệnh fill_rect đơn lẻ của driver ST7789. Điều này giúp giảm thiểu số lượng lệnh ghi SPI gửi đến LCD, tăng tốc độ vẽ đồ thị.

2. Thử nghiệm tính năng Zoom và Pan bằng phím vật lý

Giao diện Scope Viewer hiển thị cùng lúc 8 kênh đo trên màn hình. Người dùng tương tác thông qua 3 nút nhấn vật lý:

- **SW3 (Chọn công cụ):** Luân phiên thay đổi giữa 3 công cụ: Zoom (Thu phóng), Pan (Dịch chuyển cửa sổ thời gian) và Page (Đổi nhóm hiển thị: CH1-8 đến CH9-16 đến CH17-24).
- **SW2 (+ / Tăng) và SW1 (- / Giảm):**
    - Trong công cụ Zoom: SW2 thu hẹp cửa sổ hiển thị (span giảm) để phóng to dạng sóng đến mức tối đa. SW1 mở rộng cửa sổ hiển thị (span tăng) để thu nhỏ dạng sóng.
    - Trong công cụ Pan: SW2 dịch chuyển cửa sổ xem về phía sau (start tăng). SW1 dịch chuyển cửa sổ xem về phía trước (start giảm).
- **Thanh cuộn trạng thái (Scrollbar):** Một thanh vạch màu vàng hiển thị ở phía cuối màn hình, trực quan hóa vị trí bắt đầu (start) và độ rộng cửa sổ xem (span) hiện tại so với tổng độ sâu bộ nhớ 65536 mẫu.

3. Kết quả đánh giá hiệu năng Scope Viewer

- Thuật toán nén dữ liệu hoạt động cực kỳ hiệu quả. Thời gian quét phân tích bit và vẽ lại toàn bộ 8 kênh trên LCD khi người dùng thực hiện thao tác Zoom/Pan đo được chỉ từ 8 ms đến 15 ms.

Dạng sóng hiển thị sắc nét, các sườn xung được căn lề thẳng đứng chính xác. Vạch chỉ thị trigger màu tím hiển thị đúng vị trí điểm bắt đầu trigger (mẫu số 0), hỗ trợ đắc lực cho việc phân tích các xung sườn chuyển tiếp nhanh.

5.3, Đánh giá sai số lấy mẫu

Mục tiêu cốt lõi của giai đoạn thực nghiệm này là xác thực tốc độ lấy mẫu cực đại (maximum sample rate) của hệ thống Logic Analyzer đạt mức 150 MSa/s thật ở chế độ chụp Burst và đánh giá sai số trên trục thời gian đo được so với nguồn phát xung chuẩn.

5.3.1 Kết quả đo đạc thực nghiệm ở tốc độ 150 MSa/s

Thực hiện quá trình đo kiểm nghiệm với các mốc tần số phát khác nhau từ nguồn phát xung PWM 24 kênh của Raspberry Pi Pico, thu được dữ liệu thô và tiến hành phân tích thông qua PC Client ở tốc độ lấy mẫu cấu hình fs = 150 MSa/s, độ sâu mẫu N = 8192 mẫu. Kết quả đo đạc thực tế được ghi nhận chi tiết trong bảng dưới đây:

Bảng 5.1: Đối chiếu tần số phát thực tế và tần số đo được bằng PC Client ở tốc độ lấy mẫu 150 MSa/s

Tần số phát gốc fgen (MHz)	Tần số đo trên PC Client fmeasured (MHz)	Số mẫu / chu kỳ (spp)	Sai số tương đối (%)	Trạng thái
1.000 MHz	1.0000 MHz	150.00	0.00%	**PASS**
5.000 MHz	5.0000 – 5.0006 MHz	30.00	$\leq 0.012\%$	**PASS**
9.615 MHz	9.6157 MHz	15.60	$\leq 0.007\%$	**PASS**
15.625 MHz	15.6254 – 15.6256 MHz	9.60	$\leq 0.004\%$	**PASS**

Phân tích thông số trục thời gian và cửa sổ đo đạc:

- Cửa sổ thời gian đo đạc (Twindow): Được tính toán bằng tỷ số giữa độ sâu bộ đệm mẫu (N) và tần số lấy mẫu (fs):

$$T\text{window} = N / f\text{s} = 8192 / 150.000.000 \text{ Hz} \approx 54{,}61 \text{ µs}$$

- Với độ sâu mẫu được cấu hình tối đa thông qua lệnh D65536 (N = 65536 mẫu), cửa sổ thời gian đo đạc mở rộng lên đến:

$$T\text{window_max} = 65536 / 150.000.000 \text{ Hz} \approx 436{,}91 \text{ µs}$$

- Số mẫu trên một chu kỳ xung (spp - Samples Per Period): Được tính theo công thức:

$$spp = f\text{s} / f\text{gen}$$

  - Ở tần số phát 1.000 MHz, một chu kỳ tín hiệu kéo dài 1.000 ns, hệ thống thu được đúng 150.00 mẫu. Điều này có nghĩa là mỗi mức logic (HIGH hoặc LOW) chiếm đúng 75 mẫu trong bộ đệm.
  - Ở tần số phát cực đại thử nghiệm là 15.625 MHz, một chu kỳ tín hiệu kéo dài 64 ns, hệ thống thu được 9.60 mẫu/chu kỳ. Con số này xấp xỉ quy tắc oversampling 10 lần (đạt 9.6 lần), đảm bảo thuật toán phân tích trên PC Client có thể dựng lại dạng xung vuông rất sạch và định vị chính xác vị trí sườn xung với sai số thời gian cực tiểu chỉ ± 1 chu kỳ lấy mẫu (± 6.67 ns).

5.3.2 Phân tích lỗi sụt giảm tốc độ DMA và giải pháp tối ưu

1. Hiện tượng và vấn đề sụt giảm tốc độ ban đầu

Trong lần chạy thử nghiệm đầu tiên với xung phát 1.000 MHz, hệ thống gặp phải một sai lệch đo đạc nghiêm trọng. Thay vì đo được tần số 1.0000 MHz, PC Client hiển thị kết quả đo là 1.203 MHz.

Khi phân tích dữ liệu mẫu thô trong bộ đệm, số mẫu thực tế thu được trên một chu kỳ xung (spp) chỉ đạt trung bình 124.7 mẫu thay vì 150.0 mẫu như tính toán lý thuyết. Áp dụng công thức tính tốc độ lấy mẫu thực tế (fs_actual):

$$f\text{s_actual} = f\text{gen_real} \times spp = 1.000.000 \text{ Hz} \times 124{,}7 \approx 124{,}7 \text{ MSa/s}$$

Mặc dù bộ chia tần số của máy trạng thái PIO (PIO State Machine clkdiv) được cấu hình bằng 1.0f (chạy trực tiếp bằng xung hệ thống sysclk = 150 MHz), tốc độ lấy mẫu thực tế

bị hụt mất khoảng 17% hiệu năng. Điều này dẫn đến trục thời gian bị co lại, khiến tần số tính toán trên PC Client bị phóng đại lên 1.2 lần so với thực tế.

2. Nguyên nhân kỹ thuật sâu sắc (Hardware Bus Congestion)

Qua phân tích sâu về kiến trúc phần cứng của vi điều khiển RP2350/RP2354, nguyên nhân gây sụt giảm tốc độ được xác định do sự nghẽn bus hệ thống (bus congestion) và tranh chấp tài nguyên giữa CPU và DMA:

- Tốc độ PIO cực đại: Ở tần số 150 MHz, khối PIO thực hiện lệnh in pins, 24 đẩy dữ liệu vào FIFO RX với tốc độ 1 word (32-bit chứa 24-bit dữ liệu) sau mỗi chu kỳ xung nhịp 6.67 ns. Băng thông yêu cầu để duy trì hoạt động liên tục là:

  Băng thông = 150.000.000 samples/s x 4 bytes/sample = 600 MB/s
- Tranh chấp Bus (Bus Contention): Bộ điều khiển DMA có nhiệm vụ đọc dữ liệu từ FIFO RX của PIO và ghi vào SRAM thông qua bus crossbar AHB-Lite của chip. Tuy nhiên, trong mã nguồn ban đầu của Core 0, khi kích hoạt DMA hoạt động, CPU (Core 0) chạy vòng lặp kiểm tra trạng thái bận của DMA bằng cách liên tục đọc thanh ghi trạng thái DMA ở tốc độ tối đa của CPU (150 MHz). Sự truy cập dồn dập này của CPU lên bus hệ thống đã tạo ra sự tranh chấp lối vào SRAM và thanh ghi điều khiển, làm chậm tốc độ ghi của DMA.
- PIO Stall: Khi DMA bị trễ do tranh chấp bus, nó không thể giải phóng kịp thời FIFO RX của PIO (chỉ có dung lượng tối đa 8 mức sau khi đã gộp FIFO bằng cấu hình PIO_FIFO_JOIN_RX). FIFO RX bị đầy (overflow), dẫn đến việc State Machine của PIO bị tạm dừng (stall) để chờ FIFO có chỗ trống. Kết quả là chu kỳ lấy mẫu bị kéo dài ngắt quãng, làm giảm tốc độ lấy mẫu thực tế trung bình xuống còn khoảng 125 MSa/s.

3. Giải pháp tối ưu hóa trong mã nguồn Firmware

Để giải quyết triệt để nút thắt cổ chai này, hai cải tiến quan trọng đã được áp dụng trực tiếp trong mã nguồn firmware tại tệp capture_burst.cpp:

Giải pháp A: Cấu hình ưu tiên bus tuyệt đối cho DMA

Nhóm phát triển đã cấu hình lại thanh ghi điều khiển bus của hệ thống (bus_ctrl_hw->priority) để cấp quyền ưu tiên tối cao cho DMA cả chiều đọc lẫn chiều ghi. Mỗi khi DMA có yêu cầu chiếm bus để chuyển dữ liệu từ PIO FIFO vào SRAM, bộ phân xử bus (bus arbiter) sẽ cấp quyền ngay lập tức và bắt CPU phải chờ.

Đoạn mã cấu hình được thêm vào hàm `capture_burst_init`:
```cpp
// Ưu tiên bus cho DMA (cả kênh đọc lẫn ghi) để thắng tranh chấp với CPU
bus_ctrl_hw->priority = BUSCTRL_BUS_PRIORITY_DMA_W_BITS |
                        BUSCTRL_BUS_PRIORITY_DMA_R_BITS;
```

Giải pháp B: Tối ưu hóa tần suất Polling trạng thái của CPU

Thay vì để CPU chạy vòng lặp kiểm tra liên tục không có độ trễ (busy-polling) làm nghẽn bus, nhóm phát triển đã chèn một khoảng trễ nhỏ 10 µs bằng hàm busy_wait_us_32(10) trong vòng lặp kiểm tra trạng thái bận của DMA. Điều này giúp CPU giải phóng hoàn toàn bus hệ thống trong phần lớn thời gian capture, dành trọn băng thông cho DMA hoạt động.

Đoạn mã tối ưu hóa vòng lặp tại hàm capture_burst_run:
```cpp
// Chờ DMA chụp đủ 'depth' mẫu.
// Sử dụng khoảng trễ thưa 10us để giải phóng bus hệ thống cho DMA hoạt động
while (dma_channel_is_busy(s_dma)) {
   if (time_reached(deadline)) {
      pio_sm_set_enabled(s_pio, s_sm, false);
      dma_channel_abort(s_dma);
      pio_sm_clear_fifos(s_pio, s_sm);
      return false;   // Quá timeout - bộ đệm không hợp lệ
   }
   busy_wait_us_32(10); // Giải phóng bus cho DMA
}
```

4. Kết quả sau tối ưu hóa
Sau khi nạp firmware đã áp dụng hai giải pháp tối ưu trên, kết quả đo kiểm nghiệm tại mốc 1.000 MHz đạt độ chính xác tuyệt đối với 150.00 mẫu/chu kỳ (fs_actual = 150.00 MSa/s). Mọi hiện tượng méo trục thời gian và phóng đại tần số đã được loại bỏ hoàn toàn trên tất cả các mốc kiểm thử từ 1 MHz đến 15.625 MHz, chứng minh tính đúng đắn và

hiệu quả vượt trội của giải pháp tối ưu hóa băng thông bus hệ thống trên vi xử lý RP2354B.

5.4, Đánh giá độ chính xác

Cơ chế kích hoạt lấy mẫu (Triggering) là một tính năng nâng cao quan trọng giúp Logic Analyzer chỉ ghi lại dữ liệu kể từ thời điểm xảy ra một sự kiện chuyển đổi trạng thái logic mong muốn (cạnh lên hoặc cạnh xuống trên một kênh chỉ định), thay vì chụp một cách ngẫu nhiên.

5.4.1 Kịch bản thực nghiệm đo và kiểm tra Trigger

Thử nghiệm được thực hiện để đánh giá độ chính xác của cơ chế trigger cạnh sườn (post-trigger) ở cấp độ phần cứng và tính ổn định của hệ thống khi gặp sự cố không có tín hiệu trigger (timeout).

1. Thiết lập kịch bản
Tín hiệu ngõ vào: Nguồn phát Pico cấp xung vuông 1.000 MHz vào kênh CH3 (GPIO25 của LA).
Cấu hình trên PC Client: Gửi lệnh cài đặt trigger cạnh lên trên kênh CH3: T3R\n. Thiết bị phản hồi OK trig=CH3R.
* Trên màn hình LCD lúc này hiển thị trạng thái chờ: `Trig: CH3 RISING` và `WAITING TRIG...`.
* Gửi lệnh chụp dữ liệu nhị phân 'B' từ PC Client.

2. Hai tình huống kiểm thử chính
Tình huống A (Trigger thành công): Cắm dây kết nối CH3 bình thường. Khi xung phát từ Pico xuất hiện, hệ thống nhận diện cạnh lên, lập tức kích hoạt DMA chụp đủ 8192 mẫu và chuyển dữ liệu về PC Client.
Tình huống B (Trigger Timeout): Rút dây kết nối kênh CH3 (để hở chân đo). Khởi động lệnh chụp từ PC Client. Hệ thống sẽ đứng chờ tín hiệu. Đo thời gian chờ thực tế đến khi trả về lỗi ERR_TRIG_TIMEOUT trên PC Client. Kiểm tra khả năng tự khôi phục tài nguyên hệ thống sau timeout.

5.4.2 Phân tích cơ chế hoạt động trong Firmware

Cơ chế trigger cạnh được thực hiện trực tiếp ở tầng phần cứng thông qua sự kết hợp giữa chương trình PIO (capture.pio) và hàm điều khiển trong capture_burst.cpp:

1. Kỹ thuật vá mã lệnh PIO ở Runtime (Runtime Instruction Patching)
Để tránh việc phải biên dịch nhiều chương trình PIO khác nhau cho từng kênh và từng loại sườn trigger, firmware thực hiện vá trực tiếp mã máy của lệnh vào vùng nhớ lệnh của PIO (`s_pio->instr_mem`) tại thời điểm cấu hình.
Chương trình PIO dự phòng 3 ô nhớ lệnh đầu cho cơ chế trigger:
wait 0 gpio 0; slot0: Chờ mức logic NGƯỢC của cạnh mục tiêu
in pins, 24; slot1: Chụp 1 mẫu pre-trigger ở mức logic ngược này
wait 1 gpio 0; slot2: Chờ mức logic ĐÍCH của cạnh mục tiêu

Nếu trigger được bật, hàm capture_burst_run sẽ tính toán chân GPIO tương ứng của kênh đo (GPIO = 28 - k) và mã hóa lệnh wait để ghi đè vào slot0 và slot2:
if (s_trig_en) {
s_pio->instr_mem[s_offset + 0] = pio_encode_wait_gpio(!s_trig_rising, s_trig_gpio);
s_pio->instr_mem[s_offset + 2] = pio_encode_wait_gpio(s_trig_rising, s_trig_gpio);
}
Sau đó, con trỏ lệnh của State Machine được nhảy về địa chỉ đầu (offset + 0). Nếu trigger tắt, State Machine nhảy thẳng về địa chỉ .wrap_target (offset + 3) để chụp ngay lập tức.
Sau đó, con trỏ lệnh của State Machine được nhảy về địa chỉ đầu (`offset + 0`). Nếu trigger tắt, State Machine nhảy thẳng về địa chỉ `.wrap_target` (`offset + 3`) để chụp ngay lập tức.

2. Bảo toàn sườn chuyển tiếp (Pre-trigger)
Nhờ cơ chế chụp 1 mẫu ở mức logic ngược tại slot1 trước khi lệnh wait 1 ở slot2 được kích hoạt, bộ giải mã dữ liệu của PC Client có thể nhìn thấy sự chuyển đổi logic từ thấp lên cao (hoặc từ cao xuống thấp) rõ ràng giữa mẫu thứ 0 và mẫu thứ 1. Điều này giải quyết triệt để lỗi "mất sườn chuyển tiếp" đặc trưng của các hệ thống logic analyzer giá rẻ, giúp các bộ giải mã giao thức (UART, I2C, SPI) sau này định vị chính xác điểm Start bit hoặc Start condition.

3. Công thức tính Timeout và khôi phục sạch bộ đệm
Thời gian timeout được tính toán động dựa trên sample rate và depth, cộng thêm một khoảng chờ trigger cố định là 5000 ms:
Timeout (ms) = (N x 1000) / fs + 1000 + 5000 ms ≈ 6000 ms (tại 150 MSa/s, depth 8192)

Nếu quá thời gian trên mà DMA vẫn bận (chưa nhận đủ dữ liệu do không có trigger), firmware sẽ chủ động gọi hàm giải phóng bộ nhớ để tránh treo cứng vi xử lý:
pio_sm_set_enabled(s_pio, s_sm, false); // Dừng State Machine của PIO
dma_channel_abort(s_dma); // Hủy truyền nhận DMA
pio_sm_clear_fifos(s_pio, s_sm); // Xóa sạch bộ đệm FIFO tránh dữ liệu rác

5.4.3 Kết quả đánh giá độ chính xác Trigger

Ở tình huống A (Trigger thành công): PC Client nhận dữ liệu và phân tích dạng sóng của kênh CH3. Mẫu thứ 0 có giá trị logic là 0 (LOW) và mẫu thứ 1 mang giá trị 1 (HIGH). Dạng sóng xung vuông khôi phục được đạt tần số chính xác 1.0000 MHz, chứng tỏ thời điểm trigger bắt đúng cạnh lên của xung. Sai số thời gian trễ post-trigger từ lúc xảy ra sườn xung thực tế đến khi mẫu đầu tiên được ghi chỉ là 1 chu kỳ PIO (6.67 ns), hoàn toàn không đáng kể.
Ở tình huống B (Trigger Timeout): Khi rút dây cắm CH3, PC Client đợi đúng 6.0 giây (bao gồm 5.0 giây chờ trigger vật lý và 1.0 giây bù trừ cửa sổ thời gian) rồi lập tức nhận được chuỗi báo lỗi ERR_TRIG_TIMEOUT từ cổng serial và dừng chương trình một cách an toàn. Vi điều khiển RP2354B tự động giải phóng tài nguyên DMA và PIO. Khi cắm lại dây đo và thực hiện lệnh chụp tiếp theo, hệ thống hoạt động bình thường ngay lập tức mà không cần khởi động lại mạch, chứng minh giải pháp khôi phục bộ đệm sau timeout hoạt động rất tin cậy.

5.5, Đánh giá tốc độ truyền dữ liệu

Để tối ưu hóa trải nghiệm người dùng trên PC Client và đảm bảo khả năng đáp ứng nhanh chóng khi đo đạc liên tục ở các độ sâu mẫu lớn, tốc độ truyền tải dữ liệu từ bộ nhớ đệm SRAM của vi điều khiển RP2354B về máy tính qua kết nối USB-CDC là một thông số cực kỳ quan trọng.

5.5.1 Kịch bản đánh giá hiệu năng truyền tải

Thực nghiệm đo đạc và đối chiếu thời gian truyền dẫn thực tế từ lúc PC Client gửi lệnh kích hoạt chụp (Command) cho đến khi nhận và giải mã toàn bộ dữ liệu mẫu (Frame) thành công qua cổng COM ảo. Quá trình kiểm thử được thực hiện qua hai chế độ truyền tải dữ liệu và các độ sâu mẫu khác nhau:

- Chế độ truyền dạng văn bản ASCII (Lệnh C): Mỗi mẫu dữ liệu 24-bit được chuyển đổi thành chuỗi Hex 6 ký tự viết hoa và kết thúc bằng ký tự xuống dòng (n). Mỗi mẫu tiêu tốn đúng 7 bytes dữ liệu truyền dẫn trên bus.
- Chế độ truyền nhị phân Binary (Lệnh B): Mỗi mẫu dữ liệu được truyền nguyên bản dưới dạng số nguyên không dấu 32-bit Little-Endian (u32le). Mỗi mẫu tiêu tốn đúng 4 bytes dữ liệu truyền dẫn trên bus.

5.5.2 Kết quả đo đạc thực tế thời gian truyền dẫn

Thời gian phản hồi thực tế được đo đạc chính xác bằng hàm thời gian hệ thống trên PC Client, tổng hợp chi tiết trong bảng dưới đây:

Độ sâu mẫu N (mẫu)	Định dạng truyền	Dung lượng USB (bytes)	Thời gian lý thuyết (s)	Thời gian thực tế (s)	Hiệu năng cải thiện
8192	ASCII (C)	57.344	0,057	0,061	Cơ sở so sánh
8192	Binary (B)	32.768	0,032	0,034	**Nhanh hơn 1,79 lần**
65536	ASCII (C)	458.752	0,458	0,482	Cơ sở so sánh
65536	Binary (B)	262.144	0,262	0,278	**Nhanh hơn 1,73 lần**

Phân tích kết quả:

- Ở độ sâu mẫu cực đại 65536 mẫu, chế độ truyền Binary chỉ mất 0.278 s để chuyển xong toàn bộ gói tin 262.144 bytes về PC Client. Tốc độ truyền dẫn thực tế đạt được là: Tốc độ thực tế = 262144 bytes / 0.278 s ≈ 943 KB/s ≈ 7,54 Mbps.
- Con số này tiệm cận sát giới hạn băng thông vật lý thực tế của chuẩn USB 1.1 Full-Speed (≈ 1 MB/s hoặc 8 Mbps sau khi trừ đi hao phí giao thức lớp CDC).
- So với chế độ truyền ASCII, chế độ Binary giúp tăng tốc độ truyền dẫn thực tế lên từ 1.73 đến 1.79 lần (khớp chính xác với tỷ lệ giảm dung lượng gói tin vật lý từ 7 bytes/mẫu xuống 4 bytes/mẫu, tương đương giảm 43% băng thông).

5.5.3 Giải pháp tối ưu hóa truyền dẫn USB-CDC ở tầng Firmware

Đạt được hiệu suất truyền tải tốc độ cao tiệm cận giới hạn vật lý này là nhờ nhóm phát triển đã áp dụng ba giải pháp tối ưu hóa luồng xuất dữ liệu trong firmware tại file main.cpp.

1. Loại bỏ hoàn toàn cơ chế Stdio Printf tiêu chuẩn

Hàm xuất chuẩn printf của thư viện Pico SDK đi qua lớp trừu tượng ghép nối kênh truyền, bị phân tách ký tự và tự động sao chép sang cả cổng UART vật lý (UART0) hoạt động ở tốc độ baud mặc định 115200 bps để phục vụ mục đích debug. Tốc độ UART này bị giới hạn cứng ở mức ≈ 11,5 KB/s, khiến việc gửi dữ liệu của một khung 8192 mẫu ASCII mất tới gần 5 giây.
Giải pháp khắc phục: Firmware gọi trực tiếp hàm ghi của driver USB-CDC để đẩy thẳng dữ liệu vào buffer của USB Controller, bỏ qua lớp UART debug:

```cpp
static void usb_write_raw(const void *data, uint32_t len) {
    const char *p = (const char *)data;
    while (len > 0) {
        uint32_t chunk = len > 2048u ? 2048u : len;
        stdio_usb.out_chars(p, (int)chunk); // Ghi trực tiếp driver USB
        p += chunk; len -= chunk;
    }
}
```

Nhờ việc bỏ qua lớp UART, tốc độ truyền dữ liệu tăng lên hơn 17 lần so với việc xuất bằng printf tiêu chuẩn.

2. Ngăn chặn lỗi tự động biến đổi CRLF trên dữ liệu nhị phân

Môi trường runtime của Pico SDK mặc định tự động dịch ký tự xuống dòng n (0x0A) thành cặp ký tự rn (CRLF translation) trên tất cả các kênh xuất chuẩn. Đối với chế độ nhị phân (Binary mode), nếu dữ liệu lấy mẫu ngẫu nhiên chứa byte có giá trị 0x0A (n), trình dịch sẽ tự ý chèn thêm byte 0x0D (r), làm sai lệch hoàn toàn cấu trúc gói tin nhị phân và gây lỗi giải mã trên PC. Việc sử dụng hàm stdio_usb.out_chars giúp ghi đè trực tiếp byte thô, bỏ qua bộ lọc chuyển đổi CRLF, đảm bảo tính toàn vẹn của dữ liệu nhị phân.

3. Đóng gói dữ liệu dạng khối (Chunking)

Trong chế độ ASCII, thay vì gọi hàm ghi cho từng dòng mẫu (tạo ra hàng ngàn overhead hệ thống), dữ liệu được định dạng và gộp vào một bộ đệm tạm thời kích thước lớn trước khi đẩy ra USB một lần. Điều này giúp tối ưu hóa kích thước gói tin USB (USB packet size), giảm thiểu số lượng ngắt truyền dẫn và nâng hiệu năng truyền tiệm cận giới hạn.

5.6, Phân tích giới hạn hệ thống & các trade-off

Trong quá trình nghiên cứu, thiết kế và kiểm thử thực tế hệ thống Logic Analyzer 24 kênh trên chip RP2354B, nhóm phát triển đã phân tích sâu sắc các giới hạn vật lý của hệ thống và đưa ra các quyết định thiết kế đánh đổi (trade-offs) quan trọng để đạt được sự cân bằng tối ưu giữa hiệu năng và chi phí phần cứng.

5.6.1 Giới hạn Nyquist và Hiện tượng răng cưa (Aliasing)

Tốc độ lấy mẫu tối đa của hệ thống được giới hạn bởi tần số xung nhịp hệ thống cực đại mặc định không overclock của vi điều khiển RP2354B là f_{sys} = 150 MHz.

1. Lý thuyết Nyquist và quy tắc lấy mẫu dư (Oversampling)

Theo Định lý lấy mẫu Nyquist-Shannon, tần số giới hạn trên trục lý thuyết để khôi phục một tín hiệu tuần hoàn (Tần số Nyquist $f_{Nyquist}$) bằng một nửa tốc độ lấy mẫu:

$$f_{Nyquist} = f_s / 2 = 150 \text{ MHz} / 2 = 75 \text{ MHz}$$

Tuy nhiên, giới hạn này chỉ áp dụng đối với các tín hiệu dạng sin tuần hoàn thuần túy. Xung logic số bản chất là các xung vuông, về mặt toán học được cấu thành từ sóng sin cơ bản và các sóng hài bậc lẻ cao tần (3f, 5f, 7f, ...) theo chuỗi Fourier. Để khôi phục chính xác vị trí sườn xung vuông và tránh hiện tượng méo dạng xung, tần số lấy mẫu thực tế phải lớn hơn nhiều lần tần số tín hiệu đo:

- Quy tắc lấy mẫu dư tối thiểu: Hệ thống yêu cầu tốc độ lấy mẫu phải gấp ít nhất 10 lần tần số tín hiệu lớn nhất cần đo ($f_s \geq 10 \times f_{in}$) để đảm bảo sai số thời gian xác định sườn nằm trong khoảng ≤ 10%.
- Dải tần đo sạch cam kết: Với f_s = 150 MSa/s, dải tần số đo sạch tối đa mà thiết bị cam kết ghi nhận chính xác dạng xung là:

$$f_{limit} = f_s / 10 = 15 \text{ MHz}$$

Kết quả thực nghiệm tại Bảng 5.1 đã chứng minh mốc tần số phát 15.625 MHz (đạt 9.6 lần lấy mẫu dư) là giới hạn cuối cùng thiết bị còn giữ được dạng xung vuông ổn định.

2. Hiện tượng răng cưa (Aliasing) khi vượt ngưỡng Nyquist

Nếu đưa tín hiệu đo có tần số vượt quá tần số Nyquist (f_{in} > 75 MHz) vào các kênh nhận của Logic Analyzer, hệ thống sẽ gặp hiện tượng răng cưa (aliasing):

- Tần số cao của tín hiệu sẽ bị gập gương (folded back) trở lại dải tần thấp và xuất hiện dưới dạng các xung giả tần số thấp trên PC Client.
- Ví dụ, nếu đo một xung có tần số 80 MHz ở tốc độ lấy mẫu 150 MSa/s, tần số đo được trên đồ thị sẽ hiển thị sai lệch thành:

$$f_{alias} = |f_s - f_{in}| = |150\ MHz - 80\ MHz| = 70\ MHz$$

Đây là giới hạn vật lý không thể tránh khỏi của bất kỳ hệ thống lấy mẫu rời rạc nào nếu không trang bị mạch lọc thông thấp chống răng cưa (Anti-aliasing filter) ở ngõ vào (điều cực kỳ khó thực hiện trên các đường tín hiệu số dải rộng).

5.6.2 Giới hạn băng thông truyền thông USB 1.1 và Kiến trúc Burst

Vi điều khiển RP2354B tích hợp sẵn bộ điều khiển USB tương thích chuẩn USB 1.1 Full-Speed, có băng thông lý thuyết cực đại là 12 Mbps (băng thông truyền dữ liệu thực tế sau khi trừ đi hao phí giao thức lớp CDC đạt tối đa khoảng 1 MB/s).

1. Nút thắt cổ chai băng thông truyền dữ liệu

Khi hệ thống Logic Analyzer thực hiện lấy mẫu đồng thời 24 kênh ở tốc độ cực đại 150 MSa/s, lượng dữ liệu số sinh ra sau mỗi giây là vô cùng lớn:

$$\text{Băng thông dữ liệu đo} = 150.000.000\ \text{samples/s} \times 4\ \text{bytes/sample} = 600\ \text{MB/s}$$

Rõ ràng, việc truyền luồng dữ liệu thời gian thực (Real-time Streaming) trực tiếp về máy tính là hoàn toàn bất khả thi vì băng thông dữ liệu đo lớn gấp 600 lần băng thông thực tế của cổng truyền USB 1.1 (600 MB/s $\gg$ 1 MB/s).

2. Thiết kế đánh đổi: Kiến trúc chụp Burst (Burst Capture)

Để giải quyết nút thắt cổ chai truyền dẫn này, nhóm thiết kế bắt buộc phải áp dụng kiến trúc chụp Burst (chụp gián đoạn tốc độ cao):

- Khi có lệnh chụp, khối PIO0 kết hợp với bộ điều khiển DMA ghi trực tiếp dữ liệu lấy mẫu từ các chân GPIO vào phân vùng SRAM nội bộ của RP2354B ở tốc độ tối đa 150 MSa/s mà hoàn toàn không đi qua CPU.
- Dung lượng bộ nhớ SRAM tĩnh phân bổ tối đa cho bộ đệm capture là 256 KB (tương ứng độ sâu tối đa 65536 mẫu). Cửa sổ thời gian chụp cực đại đạt 436.9 µs.
- Sau khi bộ đệm SRAM đã được ghi đầy, DMA dừng hoạt động. Lúc này, CPU mới thực hiện tác vụ đọc dữ liệu tĩnh từ SRAM và truyền chậm về PC Client qua cổng USB-CDC với tốc độ khoảng 1.3 giây cho gói dữ liệu nhị phân cực đại.

Đánh đổi: Người dùng chấp nhận không thể giám sát tín hiệu số liên tục vô hạn thời gian thực trên PC Client, đổi lại hệ thống có thể chụp được các sự kiện tín hiệu cực nhanh ở tần số hàng chục MHz chỉ với phần cứng vi điều khiển giá thành thấp.

5.6.3 Nhiễu phần cứng (Signal Integrity) và Hiện tượng Ringing

Khi tần số hoạt động của tín hiệu đo đạt ngưỡng từ 10 MHz trở lên, chất lượng truyền dẫn tín hiệu vật lý giữa các bo mạch đóng vai trò quyết định đến độ chính xác của phép đo.

1. Hiện tượng dội tín hiệu (Ringing) và đếm dư cạnh

Tín hiệu xung vuông tốc độ cao có sườn xung rất dốc (thời gian chuyển mức logic từ 1.5 - 3 ns), tương ứng chứa sóng hài tần số cao hàng trăm MHz. Cáp kết nối dài (như dây jumper đơn 20 cm) không được bọc chống nhiễu sẽ đóng vai trò như một đường truyền sóng không phối hợp trở kháng đầu cuối, có điện cảm ký sinh lớn (L_{wire}).

- Sự kiện chuyển cạnh xung tạo ra dòng điện biến thiên nhanh, gây ra hiện tượng dội tín hiệu (ringing) tạo các đỉnh sóng nhấp nhô quá ngưỡng trên đỉnh và dưới đáy xung.
- Bộ đệm đầu vào Schmitt Trigger trên các chân GPIO của RP2354B có khoảng hysteresis chống nhiễu nhỏ (≈ 0.2V). Các đỉnh nhiễu dội vượt quá ngưỡng logic V_{IH} và V_{IL} khiến vi điều khiển nhận diện nhầm một sườn xung thực tế thành nhiều sườn xung chuyển trạng thái liên tiếp ở tần số cực cao.
- Trong thực nghiệm ở mốc 10 MHz, kênh CH8 đo ra tần số sai lệch 11.10 MHz và đếm dư 1300 sườn xung chính là do hiện tượng này gây ra.

2. Biện pháp thiết kế phần cứng khắc phục dội tín hiệu

Để đảm bảo tính toàn vẹn tín hiệu khi đo đạc tần số cao, các quy tắc thiết kế phần cứng sau đây bắt buộc phải được tuân thủ:

- Rút ngắn tối đa chiều dài dây dẫn: Sử dụng dây cáp đo ngắn nhất có thể (< 10 cm) kết nối giữa mạch phát và mạch thu để giảm thiểu trị số điện cảm ký sinh L_{wire}, từ

đó đẩy tần số dao động dội tín hiệu lên rất cao vượt ngoài tầm đáp ứng thu nhận của cổng GPIO vi điều khiển.

Bọc đất song song (GND Shielding): Chạy các đường dây đất (GND) song song sát cạnh các đường tín hiệu đo (tốt nhất là kết nối xen kẽ 1 đường tín hiệu - 1 đường GND). Việc này làm giảm diện tích vòng lặp dòng điện (loop area), triệt tiêu nhiễu dội sườn và giảm hiện tượng xuyên nhiễu (crosstalk) giữa các kênh đo cạnh nhau trên bus dữ liệu.

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

Đồ án đã thiết kế và chế tạo thành công thiết bị Logic Analyzer 24 kênh song song dựa trên vi điều khiển RP2354B, đáp ứng đầy đủ các mục tiêu đề ra:

- Hiệu năng phần cứng: Đạt tốc độ lấy mẫu thời gian thực 150 MSa/s, đảm bảo khả năng đo sạch tín hiệu ở tần số 15 MHz với cơ chế oversampling tối ưu.
- Kiến trúc phần mềm: Xây dựng thành công kiến trúc dual-core (Core 0 xử lý thu thập dữ liệu và giao tiếp, Core 1 chuyên trách hiển thị LCD), đảm bảo giao diện luôn mượt mà.
- Độ tin cậy và chính xác: Khắc phục triệt để các vấn đề về nghẽn băng thông bus hệ thống (bus contention) bằng cách cấp ưu tiên tuyệt đối cho DMA và tối ưu hóa polling CPU. Các sự cố nhiễu (ringing) được giải quyết qua việc cải thiện thiết kế PCB và cáp kết nối.
- Giao thức truyền tin: Hoàn thiện giao thức USB-CDC với hai định dạng ASCII và Binary, giúp tốc độ truyền dữ liệu nhanh chóng, đáp ứng yêu cầu phân tích thời gian thực.

6.2 Hướng phát triển

Để khai thác tối đa tiềm năng phần cứng của dòng chip RP2350/RP2354B, nhóm nghiên cứu đề xuất các hướng phát triển trong tương lai:

- Nâng cao tốc độ lấy mẫu: Triển khai kỹ thuật PIO Interleave để đạt tốc độ lấy mẫu hiệu dụng 300 MSa/s, nâng dải đo tín hiệu sạch lên 30 MHz.
- Ép xung hệ thống: Nghiên cứu cấu hình ép xung hệ thống (overclocking) lên mốc 200-300 MHz để tăng băng thông Nyquist.

- Phát triển tính năng Pre-trigger: Thiết kế lại luồng dữ liệu sử dụng bộ đệm vòng tròn (Ring Buffer DMA) nhằm lưu trữ và phân tích trạng thái tín hiệu trước thời điểm kích hoạt.
- Tối ưu hóa PC Client: Xây dựng giao diện đồ họa GUI thay cho công cụ dòng lệnh hiện tại, tích hợp các công cụ phân tích sâu hơn cho kỹ sư.

TÀI LIỆU THAM KHẢO

[1] Raspberry Pi Ltd., RP2350 Datasheet — A microcontroller by Raspberry Pi, tài liệu số RP-008373-DS, 2024.

[2] Raspberry Pi Ltd., Raspberry Pi Pico-series C/C++ SDK — Libraries and tools for C/C++ development on RP2040 and RP2350 microcontrollers, 2024.

[3] Texas Instruments, SN74LVC8T245 — 8-Bit Dual-Supply Bus Transceiver with Configurable Voltage Translation and 3-State Outputs, datasheet.

[4] Sitronix Technology Corp., ST7789V — 240RGB x 320 dot 262K Color TFT-LCD Single Chip Driver, datasheet.

[5] Winbond Electronics Corp., W25Q128JV — 3V 128M-bit Serial Flash Memory with Dual/Quad SPI, datasheet.

[6] AP Memory Technology Corp., APS6404L-3SQR — 64Mb Quad Serial Peripheral Interface (QSPI) Pseudo-SRAM, datasheet.

[7] Monolithic Power Systems, MP2307 — 3A, 23V, 340KHz Synchronous Rectified Step-Down Converter, datasheet.

[8] Diodes Incorporated, AP2112 — 600mA CMOS LDO Regulator With Enable, datasheet.

[9] USB Implementers Forum, Universal Serial Bus Specification, Revision 2.0, 2000.

[10] C. E. Shannon, “Communication in the Presence of Noise”, Proceedings of the IRE, vol. 37, no. 1, tr. 10–21, 1949.

[11] sigrok project, PulseView — Qt-based logic analyzer GUI for sigrok, https://sigrok.org (truy cập 2026).